

基于可行域更新与协同规划的干扰源定位清除

摘要

快速定位并清除无线电干扰源，是恢复通信秩序和保障危险区域作业安全的重要环节。题目要求机器狗在同一地点测角误差固定、接收范围有限且源数未知的条件下，自主完成圆域内干扰源排查。本文从定位区域的几何性质出发，建立主动测向、路径规划与清除协同的决策方法。

针对问题一，本文建立半平面交与直径圆覆盖判定模型，把每次测向允许的方位范围表示为两个半平面的交集，使多次观测转化为统一的线性不等式约束。先用线性规划判断交集是否为空或无界，再枚举满足全部约束的边界交点，由最远顶点对计算区域直径。对于同直径圆的覆盖问题，提出并证明最远顶点对夹角判据：其余每个顶点到最远两点的连线夹角均不小于 90° ，是该圆能够覆盖整个区域的充要条件。构造三次合法测向形成的等边三角形，其直径为 38 m ，第三个顶点处的夹角为 60° ，最小覆盖半径为 21.94 m ，大于直径的一半，说明同直径圆未必能够覆盖定位区域。

针对问题二，本文建立基于贝叶斯更新的加权选点模型。首次观测不仅限定方位，还要求源位于目标圆域及可接收范围内；不同可能源位置对应的可能接收半径也不同，因此观测后的位置分布未必均匀。在首次观测的基础上，通过贝叶斯更新预测第二次检测可能得到的反馈，并据此评价各候选点的定位效果。评价同时考虑两个方面：一是检测后剩余区域的最小包围圆的期望半径，二是该半径不超过 20 m 、从而无需补测即可确定保证清除位置的概率（完成概率）。将期望半径按 20 m 归一化，与未完成概率加权后最小化，采用分段高斯求积、网格初筛和多初值 Nelder-Mead 搜索求解。固定权重确定选点，改变权重汇集候选区域。特别地，在第一检测点为原点、示向度正东的算例中，提高完成概率权重使优选点趋近首次点，同时增大期望半径；等权下优选点为 $(813.8, \pm 521.0)\text{ m}$ ，期望半径为 25.67 m ，完成概率为 37.34% 。

针对问题三，建立全程协同策略，将搜索站点与已发现源的定位清除任务统一纳入时间规划。根据观测更新可行区域，结合光学定位距离判断能否清除，并据此调整任务次序，通过沿途补测以及继续测向与提前试清的耗时比较，减少重复行程。以逐频道覆盖和已发现源全部清除共同判定任务完成。共 80 场模拟器演练测试均完成全部清除，场均每源耗时约为 229.70 s ，较基线谨慎测向策略降低约 31.14% 。三次正式测试分别清除 15、12、10 个源，平均定位清除时间分别为 195.399 、 276.748 、 248.719 s 。

针对问题四，在第三问基础上建立联合域规划策略，处理定向源无信号可能由超距或背向引起的歧义。利用站点凸包与最小接收半径条件构造 22 站搜索布局，保证任意朝向下的源均可被发现。定位时联合约束源位置、接收半径和朝向，将无法由超距解释的无信号转化为背向约束，仅排除与观测历史不相容的位置块；再按历史相容程度确定代表位置，协调路线与测点选择，并以最小包围圆判据决定可靠清除。30 场模拟器演练测试全部清除 390 个源，场均每源耗时为 458.37 s ，较基线谨慎测向降低 29.68% 。

关键词：半平面交 最小包围圆 贝叶斯选点 协同路径规划 朝向鲁棒覆盖

一、问题背景与重述

1.1 问题背景

无线电干扰源的定位与清除是无线电频谱管理的重要任务。实际排查通常经历初步监测、区域性排查和精准定位三个阶段，其中前两个阶段确定干扰源的分布区域，而具体位置仍需进一步通过精准定位设备查明。为了尽快消除干扰，需要尽可能缩短这一过程的耗时。

针对危险环境下的排查需求，某公司拟研发搭载于机器狗的干扰源自动定位清除系统。该系统先通过无线电测向获取干扰源的方位信息，并在接近目标后利用光学探测设备实现精确定位和清除。然而，测向结果存在误差，信号接收受到距离和发射方向的限制，各项设备操作也需要耗时。因此，如何在有限观测信息下完成干扰源定位，并在确保全部清除的前提下提高作业效率，是本题研究的核心问题。

1.2 问题重述

已知干扰源分布在为半径 1800m 的圆形区域，以圆心为坐标原点，正东方向和正北方向分别为 x 轴与 y 轴正向。各干扰源占用固定、互不相同且互不影响的频道，编号为 1 至 20。干扰源分为全向与定向两类，有效接收半径在 1000m 至 1500m 之间。

测向机每次检测一个频道，测得的示向度与真实方位角之间的误差在 $[-1^\circ, 1^\circ]$ 内，同一地点重复检测不会改变误差。机器狗以 5m/s 的速度沿直线移动，停止移动后可进行检测。频道切换和单次检测分别耗时 1s 和 5s；机器狗距干扰源不超过 20m 时，可进行光学精确定位与清除，分别耗时 3s 和 2s。依据上述条件，需要完成以下四项任务。

问题一：已知若干检测点的坐标及其对同一干扰源测得的示向度，给出采用交会定位法计算多边形定位区域直径的算法，并判断以该直径为直径的圆能否覆盖整个定位区域。

问题二：已知某全向干扰源在一个检测点处的示向度，提出第二个检测点的选择策略，使进一步检测能够取得较好的定位效果，并给出第二个检测点的候选区域。

问题三：目标区域内有 10 至 16 个全向干扰源，具体数量未知。机器狗从原点出发，测向机初始频道为 1。需要制定相应策略与算法，使机器狗自主完成搜索、定位和清除，并在确保所有干扰源被清除的前提下尽可能缩短任务时间。

问题四：目标区域内同时存在全向和定向干扰源，总数仍为 10 至 16 个，但具体数量、定向干扰源的数量及其定向方向均未知。其余条件与问题三相同，在此情形下提出检测与清除策略及算法。

二、模型假设

为预测第二次检测可能出现的反馈，并计算不同检测点的期望定位损失与定位完成概率，本文采用以下假设。

假设一：源位置采用面积均匀先验。在获得首次观测之前，假设干扰源位于目标圆域内各处的可能性仅与区域面积有关，即等面积区域具有相同的先验概率。

假设二：接收半径采用均匀先验。对固定但未知的有效接收半径，假设其在 1000–1500 米范围内服从均匀先验分布，用于计算不同源距下能够接收信号的概率。

假设三：地点误差采用均匀先验。对检测地点对应的固定未知测角误差，假设其在 $[-\alpha, \alpha]$ 内服从均匀先验分布，其中 $\alpha = \pi/180$ 。该分布描述观测前对误差取值的认识，不表示同一地点重复检测时误差重新随机产生。

假设四：未知状态在先验中相互独立。在获得首次观测之前，假设源位置、接收半径以及两个不同检测地点的测角误差相互独立。获得观测后，按观测条件更新其联合分布，不再假定更新后的未知量仍相互独立。

上述假设用于计算第二问的期望定位损失与完成概率。第四问在规划代表位置时另设朝向均匀、两种类型等权的工作先验，具体定义见第四问；这些先验只用于行动估价。

第一问的几何判据，以及第三、四问的保守可行域、保证清除与搜索覆盖判据，不依赖这些概率假设。

三、符号说明

符号	定义	单位
Ω	以原点为圆心、半径为 1800 m 的目标圆域	—
g	干扰源的位置，固定但未知	m
s_i	第 i 个检测点的位置	m
q	待选择的检测点或行动位置	m
R	干扰源的固定有效接收半径，取值为 1000–1500 m	m
θ_i	在检测点 s_i 获得的正常示向度读数	rad
α	测角误差上界， $\alpha = \pi/180$	rad
r_{strong}	强信号距离阈值，取值为 5 m	m
r_{opt}	光学精确定位距离，取值为 20 m	m
W_i	第 i 次正常测向对应的正向角域，半角为 α	—
P	纯测角交会区域，即各角域 W_i 的交集	—
K	综合观测信息与物理条件得到的源位置可行区域	—
$\text{diam}(K)$	区域 K 的直径	m
$c(K)$	区域 K 的最小包围圆圆心	m
$\rho(K)$	区域 K 的最小包围圆半径	m
$d_{\max}(q; K)$	位置 q 到区域 K 的最远距离	m

其中，全部位置变量均为二维向量；公式推导中的角度统一转化为弧度，观测数据与结果展示可采用度并注明单位。多干扰源情形下增加源编号下标。其余局部符号在各问建模时另行定义。

四、问题分析

4.1 问题一分析

问题一研究使用交会定位法排查干扰源时定位区域的几何性质。根据题意，可将各次观测对应的方向范围表示为多个半平面约束，并将定位区域表示为其交集，进而利用凸多边形的性质，将区域直径的计算转化为顶点间最大距离的求解。对于同直径圆的覆盖问题，可从最远顶点对入手，通过其余顶点处的夹角判断该点对的直径圆能否覆盖整个区域，并构造符合测向条件的反例说明同直径圆未必能够覆盖定位区域。

4.2 问题二分析

问题二需根据首次观测选择第二个检测点，并给出候选区域。其难点在于，首次示向度对应的角域还需与目标圆域、可接收范围取交集，定位区域的形状随首次检测位置和示向度共同变化。同时，首次成功接收对未知接收半径的范围也施加了限制。因此，应联合考虑这些几何约束，判断第二次可能出现哪些反馈，以及每种反馈能够排除哪些源位置。

在此基础上，用贝叶斯方法更新对源位置和接收半径的判断，预测强信号、无信号和正常示向度的概率。以剩余区域最小包围圆半径的期望衡量平均定位效果，以其不超过 20 米的概率衡量无需补测即可确定保证清除位置的可能性。将两项指标归一化后加权，固定权重求得相应的优选位置，再汇集不同权重下的优选位置，形成候选区域。

4.3 问题三分析

问题三将前两问的“单源几何”扩展为“未知数量多源的在线闭环”：机器狗只知道已返回的观测、已清除的频道与各目标的位置可行集，需要在源数量未知（10 至 16 个）的前提下，自主完成搜索、定位与清除，并尽可能缩短总时间。与前两问相比，这里出现三个新问题：其一，如何把移动、检测、换频道、清除与末尾排除放在同一时间尺度上比较，即建立统一的时间账本；其二，如何证明“没有遗漏未发现源”，即给出可审计的完整性（终止）条件；其三，如何在一个“发现一定位一清除”的闭环里选择每一步动作。

4.4 问题四分析

第四问加入朝向未知的定向源，使无信号同时具有超距与背向两种解释。因而，第三问基于无信号圆盘排除的搜索与定位方法需要分别调整：搜索端利用站点凸包与距离条件保证任意朝向下均有接收机会；定位端联合源位置、作用半径及朝向，只有观测历史不可能同时成立时才排除相应区域。在此基础上，以代表位置和测点评分协调目标服务，继续以保守位置域判断是否能够可靠清除。

五、问题一的模型与求解

5.1 单次观测的半平面表示

设获取示向度的观测次数为 n 。以 $x \in \mathbb{R}^2$ 表示候选干扰源位置，记 $u(\phi) = (\cos \phi, \sin \phi)$ 为方向角 ϕ 对应的单位向量，并定义二维叉积 $[a, b] = a_x b_y - a_y b_x$ ，则每一次示向度观测给出了正向角域

$$W_i = \{x : [u(\theta_i - \alpha), x - s_i] \geq 0, [u(\theta_i + \alpha), x - s_i] \leq 0\}. \quad (1)$$

由二维叉积的几何意义可知，这里的第一个约束要求目标位于下边界射线的左侧，第二个约束要求目标位于上边界射线的右侧，两个半平面之交即为前向角域。

记 $u_i^- = (\cos(\theta_i - \alpha), \sin(\theta_i - \alpha))$ 、 $u_i^+ = (\cos(\theta_i + \alpha), \sin(\theta_i + \alpha))$ 。在一次观测对应的两条边界上，

$$\begin{bmatrix} u_{iy}^- & -u_{ix}^- \\ -u_{iy}^+ & u_{ix}^+ \end{bmatrix} x \leq \begin{bmatrix} u_{iy}^- & -u_{ix}^- \\ -u_{iy}^+ & u_{ix}^+ \end{bmatrix} s_i. \quad (2)$$

将全部观测的系数按行组成矩阵 A ，右端项组成向量 b ，叠加得到

$$P = \bigcap_{i=1}^n W_i = \{x : Ax \leq b\}. \quad (3)$$

该表示在 90° 、 270° 以及 $0^\circ/360^\circ$ 附近均可用，这是选择叉积形式而非斜率形式的主要原因。若真实目标落在每个 W_i 内，则真实位置 $g \in P$ 。 P 是闭凸集，且当 P 有界，且未退化为点或线段时为凸多边形。图 1 给出了定位区域作为各测向角域交集的几何形态：多次观测交会得到有界凸多边形；观测不足以闭合时，交集无界，沿方向延伸至无穷。

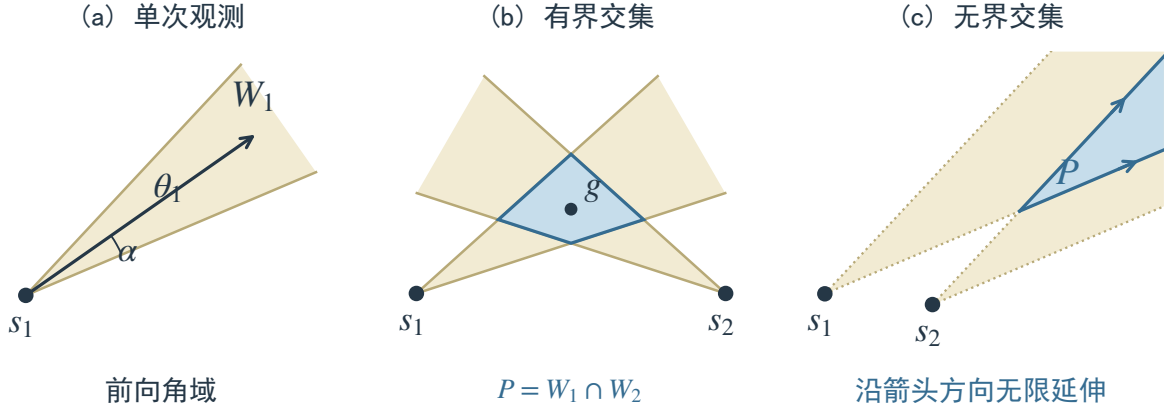


图 1 定位区域的交会几何（示意）。左：单次观测对应的前向角域；中：多次观测角域交成有界凸多边形；右：观测不足以闭合时交集无界

5.2 顶点与直径计算算法

输入检测点坐标及示向度，按式 (3) 构造半平面约束，再按以下步骤计算区域直径。

- (1) 归一化各半平面法向量，求解可行性线性规划。若不可行，则报告观测约束没有公共区域。
- (2) 将候选位置写作 $x = (x_1, x_2)$ ，在 P 上分别最大化 x_1 、 $-x_1$ 、 x_2 、 $-x_2$ 。若任一目标无界则 P 无界，直径记为 $+\infty$ ；若四个方向均有有限上界则 P 有界。
- (3) 取一个可行点作坐标平移，减轻大坐标下的数值相消。
- (4) 枚举不平行的边界直线对，求其交点，仅保留满足全部半平面约束的交点，并按容差去重。
- (5) 对二维区域按极角整理顶点顺序；对点、线段分别保留退化表示。
- (6) 计算各顶点对的欧氏距离，取最大值作为区域直径，并记录对应的最远顶点对。

若共有 $m = 2n$ 条约束，交点枚举与可行性检验的复杂度为 $O(m^3)$ ，顶点存储为 $O(h)$ (h 为去重后的顶点数)。

直径算法的依据。若 P 非空有界，将其 h 个顶点记为 $v_1, \dots, v_h$ ，则有

$$\text{diam}(P) = \max_{x, y \in P} \|x - y\| = \max_{i, j} \|v_i - v_j\|. \quad (4)$$

证明：我们知道 $x \in P$ 可写为顶点凸组合 $x = \sum_i \mu_i v_i$ ($\mu_i \geq 0$, $\sum_i \mu_i = 1$)，同理 $y = \sum_j \nu_j v_j$ 。利用两组系数之和为 1 以及三角不等式，有

$$\|x - y\| = \left\| \sum_{i, j} \mu_i \nu_j (v_i - v_j) \right\| \leq \sum_{i, j} \mu_i \nu_j \|v_i - v_j\| \leq \max_{i, j} \|v_i - v_j\|. \quad (5)$$

又因为顶点本身属于 P ，上界可以取到，故式 (4) 成立。□

上述结论说明，只需比较顶点之间的距离，就能得到整个区域的直径。在算法中，需要枚举 $h(h-1)/2$ 个点，复杂度为 $O(h^2)$ ；加上前面的交点枚举，在完成线性规划判定后，几何计算的总复杂度为 $O(m^3 + h^2)$ 。

5.3 同直径圆的覆盖判据与反例

由最小包围圆的定义，对区域 P 有

$$\rho(P) = \min_c \max_{x \in P} \|x - c\| = \min_c \max_i \|v_i - c\|. \quad (6)$$

其中第二个等号成立的原因与 (4) 同理。该最小化问题的最优圆心记为 $c(P)$ 。由于任一覆盖圆都必须容纳区域中相距最远的两点，其半径至少为两点间距的一半，因此 $\rho(P) \geq \text{diam}(P)/2$ 。下面给出这一半径下界能够达到的几何条件。

最远顶点对夹角判据：对有界多边形区域 P ，任选一对最远顶点 v_i, v_j ，以 M 表示其中点。则存在直径为 $\text{diam}(P)$ 的覆盖圆，当且仅当其余每个顶点 v_k 都满足 $\angle v_i v_k v_j \geq 90^\circ$ 。此时，最小包围圆的圆心为 M ，半径为 $\text{diam}(P)/2$ 。

证明：由于 M 是 $v_i v_j$ 的中点，向量 $\overrightarrow{Mv_i}$ 与 $\overrightarrow{Mv_j}$ 互为相反向量，易得恒等式

$$\overrightarrow{v_k v_i} \cdot \overrightarrow{v_k v_j} = \left\| \overrightarrow{v_k M} \right\|^2 - \frac{1}{4} \left\| \overrightarrow{v_i v_j} \right\|^2. \quad (7)$$

当 $\angle v_i v_k v_j \geq 90^\circ$ 时，左侧内积非正，说明 v_k 到 M 的距离不超过 $\|v_i - v_j\|/2$ 。若其余所有顶点都满足该条件，则以 $v_i v_j$ 为直径的圆盘包含全部顶点，也包含它们的凸包 P ，因而构成所需的覆盖圆。结合半径下界，可知它就是最小包围圆。

反过来，若存在直径为 $\text{diam}(P)$ 的覆盖圆，最远顶点 v_i, v_j 的间距恰好等于圆的直径，因此它们只能是该圆的一对直径端点，圆心必须为 M 。其余顶点均在圆盘内，由式 (7) 可得对应内积非正，即夹角均不小于 90° 。□

从恒等式(7)可以知道，利用这一判据计算时不要求角度，只需逐个检查 $\overrightarrow{v_k v_i} \cdot \overrightarrow{v_k v_j} \leq 0$ 。若 P 退化为单点，其直径与最小包围圆半径均为 0，可直接处理。

下面构造一个符合题意的反例，说明与定位区域同直径的圆未必能够覆盖整个区域。如图 2 所示，取边长为 38 米的等边三角形 T 。在每条边的反向延长线上，距起始顶点 1000 米处设置一个检测点，并使测向角域的一条边界沿该边、另一条边界向三角形内部偏转 2° 。从各检测点看，整个三角形的张角均小于 2° ，因此三个测向角域的交集恰好为该三角形。

这一构造可以在题目情境下实现。取三角形重心为真实源位置，各次示向度读数取相应角域的中间方向，则测角误差均不超过 1° 。源到三个检测点的距离均约为 1019 米，选取共同接收半径为 1200 米即可正常接收；将三角形放在目标圆域中心附近，也满足源位置的范围要求。

等边三角形的任意一对顶点都是最远点对，但第三个顶点处的夹角为 $60^\circ < 90^\circ$ ，不满足上述夹角判据。因此，以任意一条边为直径的圆都无法覆盖第三个顶点，也就不存在与该三角形同直径的覆盖圆。具体地，该区域的直径为 38 米，而最小包围圆是以重心为圆心的外接圆，其半径为

$$\rho(T) = \frac{38}{\sqrt{3}} \text{ m} \approx 21.94 \text{ m} > 19 \text{ m} = \frac{\text{diam}(T)}{2}. \quad (8)$$

因此，区域直径为 38 米时，覆盖整个区域所需的最小圆直径约为 43.88 米，同直径圆不能覆盖该区域。

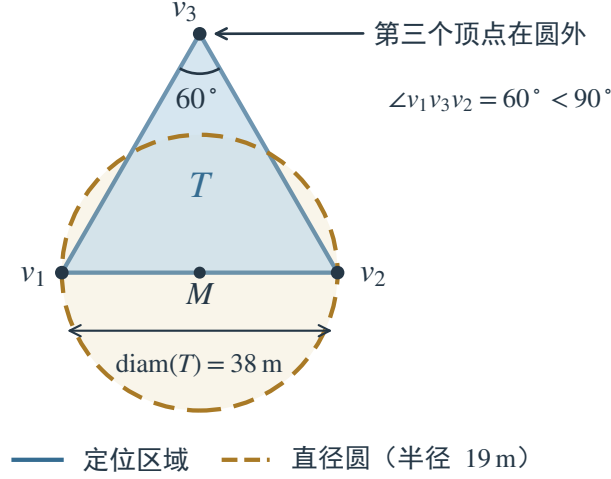


图 2 等边三角形的直径圆无法覆盖整个定位区域。

5.4 数值计算示例与核验

5.4.1 数值计算示例

题目未给出问题一的具体观测数据，下面构造一组输入，说明算法的计算过程。取两个检测点 $s_1 = (-1000, 0)$ m、 $s_2 = (0, -1000)$ m，示向度分别为 0° 、 90° ，测角误差界均为 $\pm 1^\circ$ 。该观测可由位于原点的源产生，此时测角误差为零，源到两个检测点的距离均为 1000 米，满足正常接收条件。

首先，以 $g = (x_g, y_g)$ 表示待求源位置，将上述数据代入测向角域约束，得到四个半平面不等式，可合并写为

$$\begin{aligned} -(x_g + 1000) \tan 1^\circ &\leq y_g \leq (x_g + 1000) \tan 1^\circ, \\ -(y_g + 1000) \tan 1^\circ &\leq x_g \leq (y_g + 1000) \tan 1^\circ. \end{aligned} \quad (9)$$

线性规划判定该交集非空且有界。随后将四条边界直线两两联立，保留满足全部不等式的交点，去重并按边界顺序排列，得到四个顶点。其坐标近似为

$$\begin{aligned} v_1 &= (-17.1556, -17.1556), & v_2 &= (17.1452, -17.7543), \\ v_3 &= (17.7652, 17.7652), & v_4 &= (-17.7543, 17.1452), \end{aligned} \quad (10)$$

坐标单位为米。计算四个顶点之间的六个距离，得到最远点对为 v_1, v_3 ，因此区域直径为 $\text{diam}(P) = \|v_1 - v_3\| \approx 49.3854$ m。

最后，检验另外两个顶点处的夹角，均为 90° ，满足前述夹角判据。因此，以 v_1v_3 为直径的圆能够覆盖整个区域，并且就是最小包围圆。其圆心为最远点对的中点，约为 $(0.3048, 0.3048)$ m，半径为 24.6927 m。

图 3 展示了上述计算得到的定位区域：四条角域边界围成四边形，虚线连接最远顶点 v_1, v_3 ，以其中点为圆心的最小包围圆覆盖全部顶点。

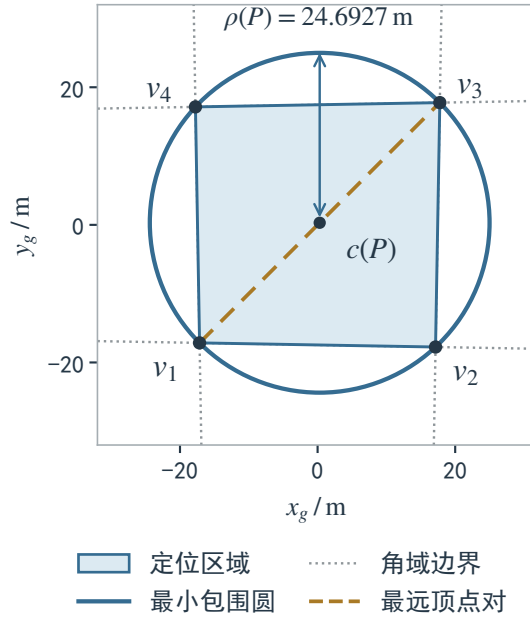


图 3 数值计算示例的定位区域与最小包围圆。最远点对为 v_1, v_3 ，区域直径为 49.3854 米，最小包围圆半径为 24.6927 米

5.4.2 算法的数值核验

为检查半平面表示是否正确，在跨越 $0^\circ/360^\circ$ 及接近坐标轴的方向上共取 3500 个测试点，直接角度判断与半平面不等式判断的结果全部一致；以独立的斜率直线求交复核上述算例，所得直径也一致。另对 60 组包含 3 至 8 个检测点的随机观测，用线性规划计算区域在给定方向上的最远投影，与所提取顶点的最大投影比较，最大差约为 4.55×10^{-13} m，支持顶点提取的数值一致性。

六、问题二的模型与求解

本文采用期望定位损失与完成概率评价第二次检测的效果，分别考察检测后还剩下多少位置不确定性，以及无需补测即可确定保证清除位置的可能性。

其中，“定位损失”指第二次无线电检测后剩余可行区域的最小包围圆半径，用来衡量源位置还有多大的不确定性。将各种反馈下的半径按概率求平均，就得到期望定位损失，数值越小，表示平均定位范围越小。“完成概率”则表示第二次检测后，剩余区域的包围圆半径在 20 米内的概率；此时无需继续无线电测向，移动到该圆圆心即可完成光学定位与清除。

需要注意的是，源位置、接收半径和各地点的测角误差均为固定未知量。本文采用贝叶斯方法，以主观概率描述在已有信息下对这些未知量取值的认识。获得新观测后，通过贝叶斯公式更新其概率分布，改变的是我们对未知量的判断，其实际取值保持不变，因此与题目规定的同一地点误差固定相容。

在此基础上，本文建立基于贝叶斯更新的加权选点模型。先根据首次观测更新对固定未知量的判断，再预测第二次反馈，计算上述两个指标，并构造定位效果综合评价函数。权重确定后，最小化该函数即可作出选点决策；改变权重，则得到不同定位偏好下的一组候选位置。

6.1 准备工作：由观测信息构造定位区域

设第二个检测点为 q ，它的位置是本问需要决定的量。为简化坐标表示，利用圆的旋转对称性，不妨将首次检测点写成 $s_1 = (a, 0)$ ，首次示向度记为 β ，其中 $a \geq 0$ 、

$\beta \in [-\pi, \pi)$ 均为确定的参数。若在 q 处得到正常示向度 θ ，则令 $s_2 = q$ 、 $\theta_1 = \beta$ 、 $\theta_2 = \theta$ ，并沿用问题一的角域记号 W_1, W_2 。源位置 g 到两次检测点的距离分别简记为 $d_1(g) = \|g - s_1\|$ 、 $d(g; q) = \|g - q\|$ 。

首次测得正常示向度，说明源位于对应角域内，能够被接收，并且尚未近到触发强信号。因此，首次观测后的源位置可行区域为 $K_1 = \{g \in \Omega \cap W_1 : r_{\text{strong}} < d_1(g) \leq 1500\}$ 。对于其中一个可能位置 g ，接收半径还必须满足 $\max\{1000, d_1(g)\} \leq R \leq 1500$ 。

在 q 处进行第二次检测，可能得到强信号（源距 q 不超过 5 米）、无信号（源距大于接收半径）或正常示向度，分别记为 H、N 和 θ 。在这里，无信号也能提供信息，这是由于首次探测已成功接收，且两次检测对应同一个固定接收半径，第二次无信号表明源距 q 比源距 s_1 更远，同时超过 1000 米。将这些新信息与 K_1 取交集，得到三种反馈后的剩余区域

$$\begin{aligned} K_H(q) &= \{g \in K_1 : d(g; q) \leq r_{\text{strong}}\}, \\ K_N(q) &= \{g \in K_1 : d(g; q) > \max\{1000, d_1(g)\}\}, \\ K_\theta(q) &= \{g \in K_1 \cap W_2 : r_{\text{strong}} < d(g; q) \leq 1500\}. \end{aligned} \quad (11)$$

其中， W_2 由检测点 q 和读数 θ 确定。若某种反馈对应的区域为空，它就不可能在已有观测条件下出现，后续不计入评价。

沿用问题一中最小包围圆的定义，用半径 $\rho(K)$ 衡量区域 K 还有多大，半径越小，源的位置就确定得越精细。第二次反馈为 z 时，将剩余区域 $K_z(q)$ 的这一半径简记为 $\rho_z(q)$ 。

若第二个检测点距 K_1 超过 1500 米，则必然只能得到无信号，因而无法缩小已有区域。因此，将距 K_1 不超过 1500 米、且不同于首次检测点的位置作为搜索范围，记为 Q （允许位于目标圆域 Ω 外）。

6.2 定位损失的定义

第二次反馈为 z 时，将剩余区域的最小包围圆半径定义为定位损失，即 $L_z(q) = \rho_z(q)$ ，单位为米。三种反馈均按这一规则计算。

- (1) **强信号**。剩余区域 $K_H(q)$ 位于检测点的 5 米范围内，取 $L_H(q) = \rho_H(q) \leq 5$ 米。
- (2) **无信号**。根据未能接收信号的信息排除部分源位置，取剩余区域的半径 $L_N(q) = \rho_N(q)$ 。
- (3) **正常示向度**。将新读数对应的角域与已有区域相交，取更新后区域的半径 $L_\theta(q) = \rho_\theta(q)$ 。

6.3 基于贝叶斯更新的反馈预测

对于同一个检测点，不同反馈留下的区域大小不同，能否达到清除条件也不同。为了计算期望定位损失和完成概率，需要先用首次观测更新对源位置和接收半径的判断，再预测第二次各种反馈出现的可能性。

6.3.1 未知量的先验描述

按照前文模型假设，源位置 G 在 Ω 内按面积均匀分布，接收半径 $R \sim U[1000, 1500]$ ，两个不同检测点的误差 $E_1, E_2 \sim U[-\alpha, \alpha]$ ，观测前认为这些未知量相互独立。均匀性和独立性是为了计算反馈概率而增加的假设，不能仅由题目给出的误差界与接收范围推出。后续的期望定位损失与完成概率均基于上述先验假设和观测信息计算，其结果反映这些条件下的预测评价。

同一地点的多次检测始终共享同一个误差，不能重新抽样，也不能靠重复测量取平均消除误差。本模型只预测不同于首次检测点的新位置。对于观测后产生的关联，例如较远的源位置必须对应较大的接收半径，后续计算会一并保留。

6.3.2 首次观测后的概率更新

将首次正常示向度信息记为 $I_1 = (s_1, \beta)$ 。可行区域 K_1 告诉我们哪些位置没有被排除，但其中的位置未必同样可能。例如，源距不超过 1000 米时，题目允许的任何接收半径都能解释首次接收；距离超过 1000 米后，只有较大的接收半径才能解释这一观测。因此，应根据接收条件调整不同位置的概率。

贝叶斯更新将原有概率与观测的似然相乘，再归一化。似然表示假定某个源位置成立时，得到已有观测的可能程度。在 K_1 内，均匀测角误差对应的角度似然相同，位置权重的差别来自能否接收到信号。距离为 d 时，这一接收概率为

$$p_{\text{rec}}(d) = \mathbb{P}(R \geq d) = \begin{cases} 1, & d \leq 1000, \\ (1500 - d)/500, & 1000 < d < 1500, \\ 0, & d \geq 1500. \end{cases} \quad (12)$$

将上述权重用于 K_1 内的各个位置，并使总概率等于 1，得到首次观测后的源位置密度

$$p_1(g) = \frac{\mathbf{1}_{K_1}(g)p_{\text{rec}}(d_1(g))}{C_1}, \quad C_1 = \int_{K_1} p_{\text{rec}}(d_1(g)) dg > 0. \quad (13)$$

其中 C_1 是归一化系数， $\mathbf{1}$ 表示集合的示性函数。该结果说明，距离超过 1000 米后，越远的位置得到的权重越低，因为能解释首次接收的半径取值越来越少。

图 4 固定首次检测点为 $s_1 = (1200, 0)$ 米，比较 $\beta = 30^\circ$ 与 90° 时的后验分布。示向度决定保留哪个角域，而圆域边界进一步限制源的位置： 30° 方向的角域较早遇到边界，后验集中在较小的区域； 90° 方向保留的距离范围更长，其中超过 1000 米的部分还因接收概率下降而降低权重。这说明，当圆域边界参与截断时，示向度会改变后验分布的形状与密度大小。两幅热力图采用同一色标，并按源距与相对示向角展开显示；颜色仍表示单位面积上的位置概率密度。

接收半径本身也需要更新。若源位于 g ，首次接收已经排除了小于 $d_1(g)$ 的半径。因此，对于后验密度为正的位置，有 $R | G = g, I_1 \sim U[\max\{1000, d_1(g)\}, 1500]$ 。预测第二次检测时，要使用更新后的范围，不能把 R 再当作与源位置无关的量。

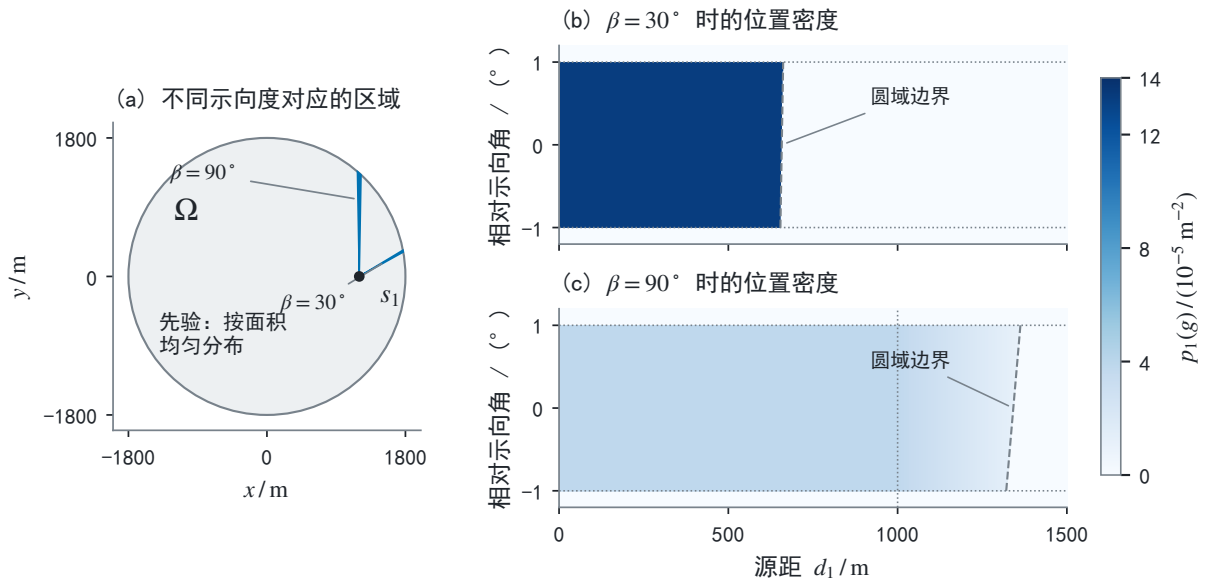


图 4 非圆心检测点处，不同首次示向度对应的后验位置密度

6.3.3 第二次反馈的概率预测

现在固定一个待评价的检测点 q ，考虑源的每个可能位置 g 。因为接收半径固定，两次都能接收要求半径至少达到两次源距中较大的一个。记这个距离为 $d_v(g; q) = \max\{d_1(g), d(g; q)\}$ ，则两次均能接收的概率权重为 $p_{\text{rec}}(d_v)$ 。从首次能够接收的权重 $p_{\text{rec}}(d_1)$ 中减去它，就得到首次接收、第二次无信号的权重。

再结合三种反馈各自的位置约束，得到用于计算反馈概率的权重函数

$$\begin{aligned} b_H(g; q) &= \mathbf{1}_{K_H(q)}(g) p_{\text{rec}}(d_1(g)), \\ b_N(g; q) &= \mathbf{1}_{K_1}(g) [p_{\text{rec}}(d_1(g)) - p_{\text{rec}}(d_v(g; q))], \\ b_\theta(g; q) &= \frac{\mathbf{1}_{K_\theta(q)}(g)}{2\alpha} p_{\text{rec}}(d_v(g; q)). \end{aligned} \quad (14)$$

其中，正常示向度的读数还受新地点测角误差影响。按照不同地点误差独立且均匀的假设，允许的角度范围内密度为 $1/(2\alpha)$ ，因此第三项包含这个因子。将各个可能源位置的权重积分，并除以首次观测的归一化系数，得到

$$\begin{aligned} P_H(q) &= \frac{1}{C_1} \int_{K_1} b_H(g; q) dg, \quad P_N(q) = \frac{1}{C_1} \int_{K_1} b_N(g; q) dg, \\ f_\Theta(\theta; q) &= \frac{1}{C_1} \int_{K_1} b_\theta(g; q) dg. \end{aligned} \quad (15)$$

$P_H(q)$ 和 $P_N(q)$ 分别是强信号与无信号出现的概率。正常示向度可以连续变化，因此用密度 $f_\Theta(\theta; q)$ 描述；将它在某个角度区间内积分，就得到读数落入该区间的概率。在全部角度上积分所得的正常反馈概率，与前两种概率相加等于 1。

6.4 定位效果的综合评价与选点决策

至此，对于每个待选位置 q ，我们既知道不同反馈会留下多大的定位损失，也知道这些反馈出现的概率。以 Z 表示尚未得到的第二次反馈，将损失按预测概率求平均，得到期望定位损失

$$\begin{aligned} J(q) &= \mathbb{E}[\rho_Z(q) \mid I_1, q] \\ &= P_H(q) \rho_H(q) + P_N(q) \rho_N(q) + \int_0^{2\pi} f_\Theta(\theta; q) \rho_\theta(q) d\theta. \end{aligned} \quad (16)$$

右端依次累加强信号、无信号和正常示向度对应的剩余半径，其中前两项按反馈概率加权，正常示向度按预测密度积分。

另一方面，题目允许在距源 20 米以内进行光学定位。只要剩余区域能被一个半径 20 米的圆完全覆盖，移动到圆心后，无论源位于区域中的何处，都能完成光学定位与清除，无需继续无线电测向。按最小包围圆的定义，这一条件等价于 $\rho_Z(q) \leq 20$ m。将这一事件的概率记为 $P_{\text{finish}}(q)$ ，则

$$\begin{aligned} P_{\text{finish}}(q) &= \mathbb{P}(\rho_Z(q) \leq 20 \mid I_1, q) \\ &= P_H(q) + P_N(q) \mathbf{1}_{\{\rho_N(q) \leq 20\}} \\ &\quad + \int_0^{2\pi} f_\Theta(\theta; q) \mathbf{1}_{\{\rho_\theta(q) \leq 20\}} d\theta. \end{aligned} \quad (17)$$

强信号将源限制在检测点的 5 米范围内，一定满足完成条件；无信号和正常示向度则按更新后区域的最小包围圆半径判断。

图 5 具体说明了 20 米包围圆条件。这里取首次点为原点、首次示向正东，第二检测点为 $q = (813.82, 521.02)$ 米。若新读数为 270° ，剩余区域的包围圆半径约为 16.87 米，

移动至其圆心即可保证清除，无需补测；若读数为 300° ，半径约为 30.43 米，任何 20 米圆都无法覆盖整个区域，因而仍需补测。两幅局部图使用相同的长度尺度，虚线圆的半径均为 20 米。

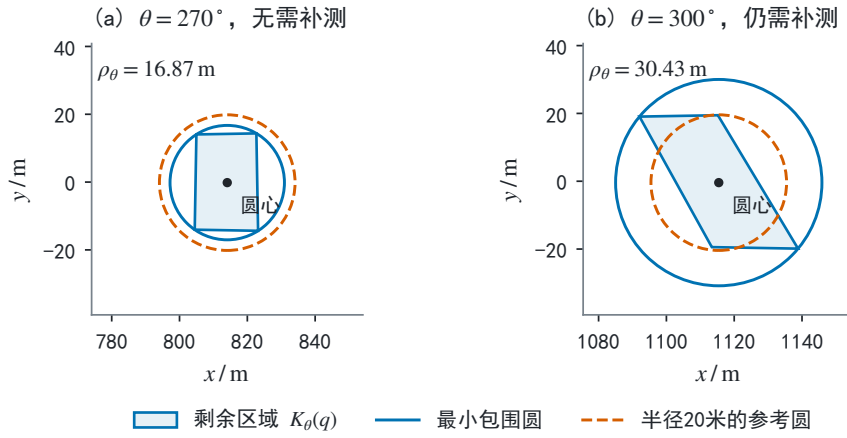


图 5 两种示向度反馈下的剩余区域与完成条件

上述两个指标分别反映平均剩余不确定性和本次检测后结束测向的可能性。例如，在首次点位于原点、示向正东的算例中，将完成概率纳入权衡后，数值优选点的期望损失可由 25.08 米增至 25.67 米，而完成概率由 26.62% 提高至 37.34%。这说明较小的平均损失增量可能换来明显更高的完成概率，因此有必要同时考虑两个指标。

最后，以任务给定的 20 米光学定位距离作为尺度，并将 $J(q)$ 除以 20，消除其与概率之间的量纲差异。设 $w \in [0, 1]$ 为完成概率指标的权重，构造定位效果综合评价函数 $F_w(q)$ ，并将第二检测点的选择写为

$$\min_{q \in Q} F_w(q) = (1 - w) \frac{J(q)}{20} + w [1 - P_{\text{finish}}(q)]. \quad (18)$$

F_w 越小，综合定位效果越好。 $w = 0$ 时，只最小化期望定位损失； $w = 1$ 时，只最大化完成概率；中间取值表示两者之间的权衡。权重反映对定位效果的偏好，不是题目给定的物理参数。固定 w 后，求解式 (18) 即得到该偏好下的选点决策。

沿用图 5 的原点算例，该检测点是 $w = 0.5$ 时得到的数值优选点。图 6 给出其第二次示向度的预测密度，并用阴影标出满足完成条件的反馈。本例强信号与无信号的概率均为 0，因此阴影面积就是完成概率，约为 37.34%。密度较高的读数未必能使区域缩小到 20 米圆内，这说明选点时需要同时考虑反馈出现的可能性及其留下的区域大小。

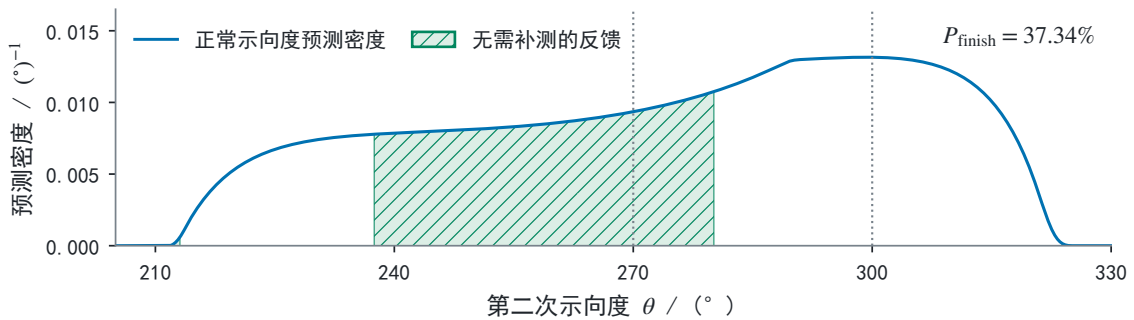


图 6 第二次示向度的预测密度及满足完成条件的反馈

6.5 由权重变化确定候选区域

题目没有指定上述两项指标的相对重要程度，因此不必预先限定唯一的权重。让 w 在 0 至 1 之间变化，汇集各个权重下的最优位置，便得到第二检测点的候选区域

$$Q_{\text{cand}} = \bigcup_{w \in [0,1]} \arg \min_{q \in Q} F_w(q). \quad (19)$$

每个入选位置都对应至少一种定位偏好。使用时先确定权重，再选取该权重对应的位置即可。若出现并列最优点，则保留这些等效选择；例如，首次点位于圆心时，关于首次示向方向对称的位置具有相同评价。在只重视完成概率且出现并列解时，可优先报告期望损失较小的代表点。

实际计算时，对若干权重分别求解并汇集优选点。具体求解方法与结果如下。

6.6 数值求解方法

对一般的 (a, β) ，反馈区域随检测点改变，评价函数同时包含分段概率积分和最小包围圆计算，难以写成统一、简洁的显式选点公式。因此，本文采用**确定性数值积分与网格、多初值局部搜索相结合的方法**，输入首次观测及权重，输出检测点和两项评价指标。

第一步：构造区域与搜索范围。由 (a, β) 计算 K_1 和归一化系数 C_1 ，检查首次观测是否可行。计算时以首次点为临时原点、首次示向为横轴，便于处理狭窄角域；最终将选点坐标平移、旋转回题目的坐标系。若首次区域已满足 20 米包围圆条件，则直接报告无需再次测向。

第二步：计算一个检测点的评价值。固定 q ，分别构造强信号、无信号和各正常读数对应的区域，用边界交点与圆弧计算最小包围圆。位置积分在极坐标下进行，包含面积因子 r ；在接收权重和区域约束变化处划分积分区间，再用高斯求积计算反馈概率。对正常示向度的外层积分，还要在区域形状变化及包围圆半径穿过 20 米的位置分段，并自适应细分，从而分别累加 $J(q)$ 与 $P_{\text{finish}}(q)$ 。这种确定性计算避免了随机抽样波动干扰相近检测点的比较。

第三步：搜索优选位置。先在搜索范围的外包矩形内布置 100 米网格，再在可行区域附近补充较密的网格与沿示向方向的采样点。每次评价均检查 $q \neq s_1$ 且距 K_1 不超过 1500 米，不合格的位置不参与比较。对固定权重，选取 5 个位置相互分开的较优网格点，并加入初始区域圆心作为参考起点，分别进行 Nelder-Mead 单纯形搜索。该方法只比较函数值，适合本模型中导数不易稳定计算的情形；随后对两个较优结果提高积分精度并再次细化。局部细化以坐标收缩至厘米量级、目标值极差不超过 2×10^{-9} 为停止条件，最多迭代 130 次。同一点的两项指标可供不同权重复用。

第四步：复算与核验。最终候选点采用更高阶求积和更密的阈值扫描复算，并检查 0.1、1、5 米邻域内是否存在更优点。各组优选点的期望损失在提高精度前后变化小于 2×10^{-6} 米，概率归一化残差小于 10^{-7} 。另以独立射线交区间和支撑圆计算核查区域几何，并对 8 个代表选点分别进行 20000 次联合先验抽样，核对期望损失与完成概率。下文报告经过上述搜索与核验得到的数值优选点。

6.7 求解结果与第二检测点的选择

6.7.1 不同首次观测下的典型结果

为分辨首次位置与示向度的影响，表 1 统一取 $w = 0.5$ ，比较圆心、非圆心的不同朝向及靠近边界的情形。表中均使用以目标圆心为原点的坐标，存在等效选点时列出一个代表。

表 1 说明，首次点到圆心的距离相同，并不必然对应相近的定位效果。例如， $a = 1200$ 米时， 30° 方向的初始区域受圆域边界截断较多，第二次检测的完成概率可达 100%； 90° 方向保留范围更长，完成概率约为 40.38%。 150° 方向的首次角域在接收范围内完整落

表 1 典型首次观测下的数值优选点 ($w = 0.5$)

首次观测点横坐标 a / m	首次观测点示向度 $\beta / ^\circ$	第二检测点 q / m	期望定位损失 $J(q) / \text{m}$	完成概率 $P_{\text{finish}}(q) / \%$
0	0	(813.8, 521.0)	25.67	37.34
1200	30	(1459.2, 467.7)	12.29	100.00
1200	90	(695.7, 798.1)	24.12	40.38
1200	150	(755.7, 858.1)	25.67	37.34
1700	0	(1770.1, 42.0)	1.88	100.00

入圆域，其定位几何与圆心算例等价，因此平移、旋转选点后得到相同的两项指标。靠近边界的 $a = 1700$ 、 $\beta = 0$ 情形则具有更小的剩余不确定性。

上述结果表明，首次位置与示向度共同决定第二次检测能够达到的定位精度和完成概率。

6.7.2 首次位于原点、示向正东时的权重比较

当 $a = 0, \beta = 0$ 时，关于横轴对称的两个检测点具有相同评价。表 2 列出 11 个权重对应的数值优选点，以 $\pm$ 同时表示两支对称候选位置。

表 2 原点、正东算例中不同权重的选点与定位效果

权重 w	第二检测点 q / m	$J(q) / \text{m}$	$P_{\text{finish}}(q) / \%$
0	(906.6, ± 572.8)	25.08	26.62
0.05	(895.8, ± 562.3)	25.09	28.77
0.1	(885.9, ± 554.0)	25.11	30.40
0.2	(867.4, ± 541.8)	25.20	32.84
0.35	(840.9, ± 529.6)	25.39	35.42
0.5	(813.8, ± 521.0)	25.67	37.34
0.65	(785.1, ± 513.1)	26.08	38.81
0.8	(754.0, ± 502.4)	26.65	39.89
0.9	(731.4, ± 490.7)	27.20	40.37
0.95	(719.1, ± 482.2)	27.56	40.51
1	(705.5, ± 470.7)	28.02	40.57

由表 2，只考虑期望损失时，优选点约为 (906.6, ± 572.8) 米。两项归一化指标等权时，选取 $w = 0.5$ ，得到 $q \approx (813.8, \pm 521.0)$ 米；相较 $w = 0$ ，期望损失增加约 0.60 米，完成概率提高约 10.71 个百分点。若只追求完成概率，则取 $w = 1$ ，优选点约为 (705.5, ± 470.7) 米，完成概率约为 40.57%，相应期望损失增至 28.02 米。

图 7 将这些结果画在位置平面和指标平面上。左图显示权重变化对应的两支镜像候选点；右图的曲线逐渐变平，说明完成概率越接近本次搜索得到的最大值，进一步提高它所需增加的期望损失越多。由此，本问的输出是随权重变化的一组候选位置；确定定位偏好后，采用对应权重的选点。对于表外的首次观测，按前述数值流程重新求解即可。

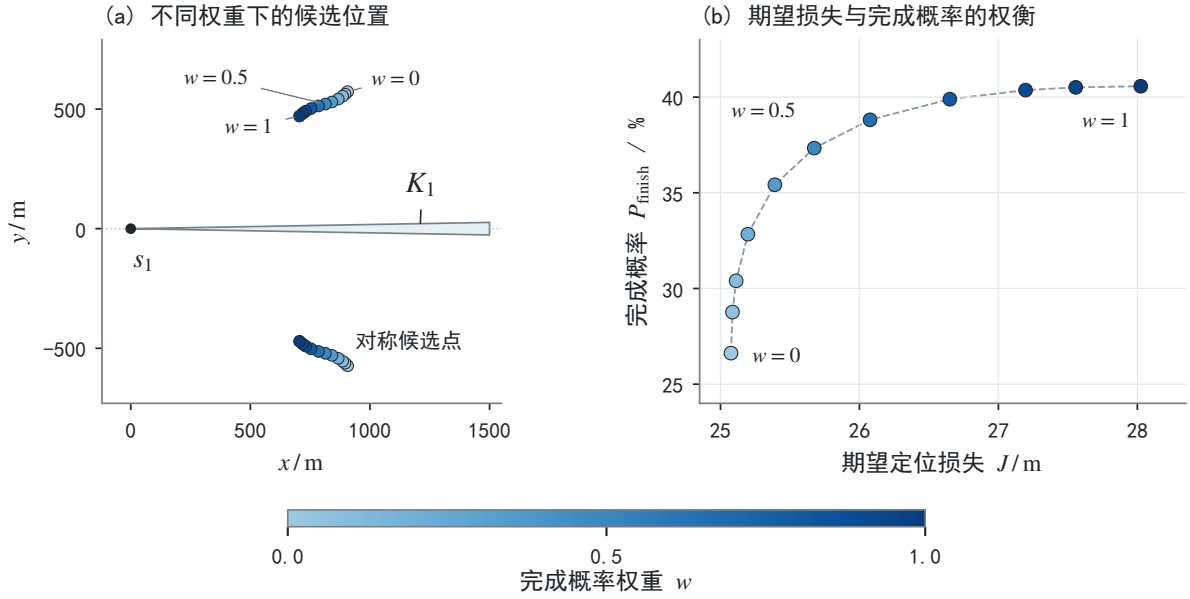


图 7 11 个权重下的候选检测点及评价指标 ($a = 0, \beta = 0$)。颜色表示完成概率权重, 虚线连接相邻权重的数值结果。

七、问题三的模型与求解

7.1 多源任务的时间目标

第三问需要在源数未知的情况下完成全部干扰源的搜索与清除。若逐源定位后再继续搜索, 容易重复经过同一区域; 若先完成全部搜索, 又会错过沿途定位的机会。为减少这两类绕行, 本文提出**全程协同策略**, 将搜索站点与已发现源一并排序, 并根据每次反馈调整后续测量和行进路线。

Vander Hook 等将移动与测量时间共同纳入主动定位评价 [4]。结合本题计费规则, 记总路程为 D_{move} , 无线电检测、实际换频、成功清除和失败清除次数依次为 n_{meas} 、 n_{sw} 、 n_{ok} 和 n_{fail} , 则总虚拟时间为

$$T_{\text{total}} = \frac{D_{\text{move}}}{5} + 5n_{\text{meas}} + n_{\text{sw}} + 5n_{\text{ok}} + 3n_{\text{fail}}. \quad (20)$$

成功清除的 5 秒包括光学定位 3 秒与清除 2 秒; 清除不改变测向频道, 无信号检测与末尾排查也须计费, 任务结束后无需返回原点。

记 $K_{j,t}$ 为第 t 次更新后频道 j 的位置可行区域, p_t 为机器狗当前位置, $\mathcal{D}_t$ 、 $\mathcal{C}_t$ 分别为已发现、已清除频道集合, $\mathcal{J}_t$ 为尚未发现且未被完整排除的频道集合。策略只使用检测与清除反馈, 真实位置 g_j 和源数 N 仅用于事后评价。以下沿用第一问的角域求交表示和第二问的观测后区域评价, 并结合光学定位距离, 建立多源任务的行动规则。

7.1.1 谨慎测向与协同改进

首先, 以 Vander Hook 等的主动定位方法 [4] 构造**谨慎测向策略**作为基线。该方法在估计协方差主轴的法向上生成两个对称测点, 选择离当前位置较近的一侧; 随着不确定性减小, 测点逐渐靠近估计位置。原文采用高斯噪声与单目标设定, 本文保留其选点规则, 以本题的有界几何约束更新位置。

具体地, 由角域交集的均匀面积矩计算中心、主轴与尺度, 取谨慎参数 $\beta_c = 0.1$, 并将步长限制在能够保证接收的范围内, 排除重复测点。面积矩用于选点, 清除仍以 19.5 米最远距离为准。搜索站点取原点及半径 1200 米圆周上的六个等分点; 优先就近处理已发现源, 再按覆盖收益和距离选择扫描点, 清除后不返回原站点。基线与本文策略采

用相同的逐频道整单元覆盖停止条件。基线的适配步骤及参数均在演练前确定，不沿用原文的滤波器与性能保证。

谨慎测向将各源的定位任务分开处理，所选测点主要改善当前源的交会几何，未同时考虑定位结束后到下一扫描点的路程。针对这一不足，先构造**局部协同策略**，通过调整扫描位置、共享站点和比较局部测向时间减少费用；再提出**全程协同策略**，将任务的进入、离开位置纳入路线比较，并增加沿途补测和提前试清。前者主要改进单次选点，后者进一步协调前后任务的衔接。

7.2 由观测约束到可靠清除

7.2.1 融合测向、接收与清除反馈

对全向源，正常测向给出正向角域 W_i 与 1500 米接收上界；强信号给出 r_{strong} 距离约束；无信号至少排除 1000 米范围。在检测点 s_i 取得反馈后，分别更新为

$$\begin{aligned} K_{j,t+1} &= K_{j,t} \cap W_i \cap B(s_i, 1500), & \text{正常示向度,} \\ K_{j,t+1} &= K_{j,t} \cap B(s_i, r_{\text{strong}}), & \text{强信号,} \\ K_{j,t+1} &= K_{j,t} \setminus B(s_i, 1000), & \text{无信号.} \end{aligned} \quad (21)$$

其中 $B(s, r)$ 为闭圆盘。同一源的接收半径固定，若 s_+ 有信号而 s_- 无信号，真实位置还满足

$$\|g_j - s_+\| \leq R_j < \|g_j - s_-\| \implies 2(s_- - s_+)^T g_j < \|s_-\|^2 - \|s_+\|^2, \quad (22)$$

从而获得一个连接正、负观测的半平面约束。

计算中以保守外包表示相容位置区域，并在其闭包上求最远距离。为容纳 0.01° 读数舍入，角域半宽取 1.0051° ，题设误差界 $\alpha = \pi/180$ 不变。圆域以外包多边形近似；普通无信号排除后允许取凸包，光学失败产生的孔洞和分裂则予以保留，以免再次将已检查位置计入试清收益。

7.2.2 根据区域尺度选择行动

根据题目的光学定位距离，若行动位置 q 到可行区域 $K_{j,t}$ 内所有位置的距离均不超过 20 m，就能在该位置完成光学定位与清除。为预留 0.5 米的数值余量，实现采用 $r_{\text{safe}} = 19.5 \text{ m}$ 。

当 $\rho(K_{j,t}) \leq r_{\text{safe}}$ 时，最小包围圆圆心一定是可选的清除位置。为减少移动，在该圆心与当前位置 p_t 之间的线段上，选择距 p_t 最近、且到整个区域的最远距离不超过 r_{safe} 的位置；若当前位置已满足要求，则原地清除。

若区域半径超过 120 米，沿区域最长方向的法向，在圆心两侧各生成一个检测点，侧移距离取 150 米与 $0.25\rho(K_{j,t})$ 中的较小值；区域较小时优先考虑中心。候选须满足接收的充分条件，再选移动较近者。例如，整个区域距候选点均不超过 1000 米即可保证接收；首次正观测也可用于构造接收下界。重复位置改用可接收侧移点。

7.3 以逐频道覆盖保证搜索完整

对未发现频道 j ，记实际无信号检测位置集合为 $\mathcal{P}_{j,t}^-$ ，其尚未排除区域为

$$\mathcal{U}_{j,t} = \Omega \setminus \bigcup_{s \in \mathcal{P}_{j,t}^-} B(s, 1000). \quad (23)$$

只有 $\mathcal{U}_{j,t}$ 为空，才有依据认定该频道不存在源。经过某点但未检测该频道，或只检测其他频道，均不能计入覆盖记录。

计算时，将包围 Ω 的正方形划分为边长 28.125 米的网格，保留所有与圆域相交的单元，包括边界单元。只有单元四角均距该频道的同一个实际观测点不超过 999.99 米，才标记整单元已排除。圆盘为凸集，四角位于其中足以保证整个矩形被覆盖，略小于 1000 米的半径提供数值裕量；这也避免了仅检查网格中心造成漏判。

正常停止需同时满足两个条件：已发现源全部清除，每个未发现频道的所有网格单元均已排除。若已清除 16 个不同源，则由题设数量上限直接结束。覆盖记录只计入实际完成的检测，因此该停止条件能够排除正常终止时的漏源；程序能否顺利运行至终止，另由实验检验。

7.4 联合安排扫描与目标处理任务

Engin 和 Isler[5] 通过联合考虑多个目标的信息需求减少定位费用。对于本题，一次移动也可同时服务未知频道搜索和已知源定位。因此，将扫描与目标处理作为两类任务，统一比较其访问次序。

起点先扫描 20 个频道，再根据已发现目标方向设置 250 米侧向基线并筛选补测。在原点和半径 1200 米圆周六点的基础上，比较 0° 至 50° 、间隔 10° 的六种旋转布局；目标的预计处理位置也可作为共享扫描位置。依据实际覆盖记录删除冗余扫描任务，并在保留必需网格角点覆盖约束的范围内调整站点，缩短相邻任务间路程。

扫描任务在站点执行；目标任务可能先测向，再移动到清除点。记任务 b 的预计离开位置为 e_b ；扫描任务取站点，目标任务以 $c(K_{j,t})$ 近似。其进入位置 $q_b(e_a)$ 由前一任务离开位置 e_a 决定，取可行清除点或下一测向点。候选序列 π 以当前位置为起点，按

$$\hat{T}_{\text{plan}}(\pi) = \frac{1}{5} \sum_{(a,b) \in \pi} [\|q_b(e_a) - e_a\| + \|e_b - q_b(e_a)\|] + 6 \# \mathcal{J}_t m_{\text{scan}}(\pi) \quad (24)$$

比较，其中 m_{scan} 为扫描站数，6 秒是一次测量与换频的保守估价。进入位置随前序任务变化，故预计路程通常不对称。通过多起点近邻构造和路径片段反转比较序列，只执行首项任务，再据反馈重规划；实际费用仍按式(20)累计。

行程中的补测按位置间隔和预计精度收益筛选。行程至少 250 米时，检查四分之一、二分之一和四分之三处；仅考虑区域半径至少 100 米、距目标圆心不超过 1000 米，且与已有测点和最近尝试点均相隔至少 150 米的候选。以指向圆心的方位及 $\pm 1^\circ$ 试算三种读数，预计半径平均下降至少 100 米才补测。若多个位置满足条件，则比较各入选频道“预计半径下降量减 30 米”的总和。

到达站点后，补测还要求历史观测少于 100 次、距圆心不超过 1200 米，且与历史测点及最近尝试点相隔至少 150 米。已有两次观测、距圆心超过 100 米时，三种读数须均达到 19.5 米精度，或使平均半径降至原值的 40% 以内。上述阈值用于控制补测数量，所有实际补测仍按频道计费。

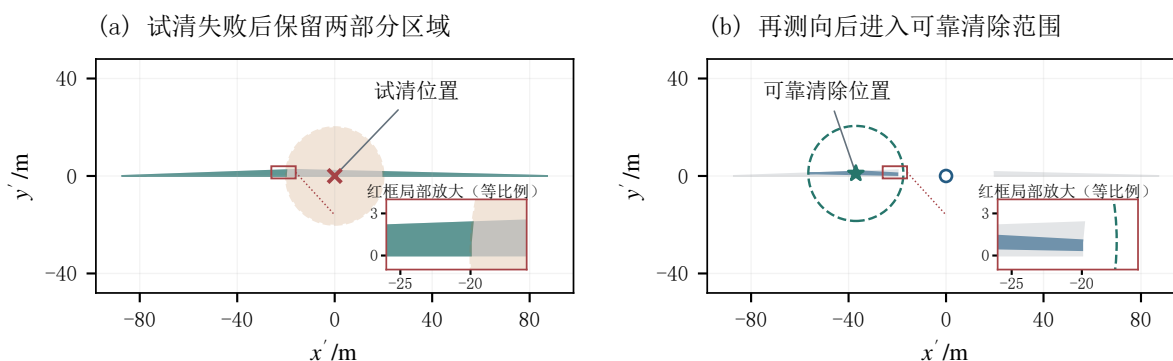
7.5 继续测向与提前试清的局部时间比较

当 $19.5 < \rho(K_{j,t}) \leq 100$ 米时，试清有可能比继续测向更快，但不能只比较失败 3 秒与测量 5 秒，还须考虑移动和失败后的后续定位。算法从当前位置、区域中心及邻近几何位置生成少量试清点，剔除重复或覆盖面积过小者。将区域代表位置与 $-\alpha, 0, \alpha$ 误差组合为有限情形集合 $\Xi_{j,t}$ 。对情形 $\xi = (x, e)$ ，试清点 q 的局部完成时间估计为

$$\hat{T}_{\text{try}}(q; \xi) = \frac{\|q - p_t\|}{5} + \begin{cases} 5, & \|x - q\| \leq r_{\text{opt}}, \\ 3 + \hat{T}_{\text{cont}}(K_{j,t}^-(q), q; \xi), & \|x - q\| > r_{\text{opt}}. \end{cases} \quad (25)$$

其中 $K_{j,t}^-(q)$ 为试清失败后的区域。后续过程最多试算五次测向，未完成则在估计时间中追加 1000 秒。动作按“80% 平均时间加 20% 最大时间”评分；试清评分比继续测向至少低 1 秒时才采用，每源最多两次。该评分为有限情形下的近似评价。

实际试清失败后，源距 q 大于 20 米。从可行区域中去除半径 19.999 米圆的内接多边形，保守保留非凸或分裂结构，再继续选择动作。图8中，试清前区域半径 87.43 米，试清与继续测向评分分别为 24.90、29.59 秒。失败后区域分裂，半径仍约 87.43 米；随后一次测向使其降至 18.28 米，再移至最远距离 19.5 米的位置成功清除。面积减少未必立即降低包围圆半径，失败反馈仍可用于后续定位。



主图与插图分别保持等比例；插图刻度单位为米

图 8 实际试清失败与后续定位。主图保持等比例，插图为明确标注的局部放大。蓝色表示观测位置域，绿色表示试清失败后保留部分，灰色为后续测向前区域；圆圈分别为 20 米试清和 19.5 米可靠清除范围。局部坐标仅由平移、旋转得到，未使用真实源坐标。

7.6 测试结果与策略分析

7.6.1 模拟器演练测试比较

谨慎测向进行了 30 场模拟器演练测试，局部协同与全程协同各采用此前 50 场完整记录。每场以总虚拟时间除以实际源数，再对各场结果取平均，结果见表3。三组均完成全部清除，全部结果纳入统计。由于场景与批次不同，组间差异按独立样本分析。

表 3 谨慎测向、局部协同与全程协同的模拟器演练测试

策略	全清场数	源总数	均值	标准差	最大值
谨慎测向	30/30	373	333.59	37.21	396.52
局部协同	50/50	640	238.44	35.21	302.30
全程协同	50/50	645	231.71	31.20	303.41

后三列单位为 s/源；仅比较本题实现，不代表原文献算法排名。

全程协同较谨慎测向平均节省 101.88 秒/源 (30.54%)，节省量的 Welch 95% 置信区间为 [85.63, 118.13] 秒/源。图9显示，移动时间由 277.35 降至 170.72 秒/源，检测时间则由 43.40 增至 46.70 秒/源。增加的少量补测改善了后续定位条件，使扫描与清除能沿较短路线完成，因而总时间下降。按两组共同源数层的合并频数标准化后，均值为 329.64 与 234.74 秒/源，改善方向不变；这一处理控制了源数构成差异，地图、接收半径与批次的影响仍然存在。

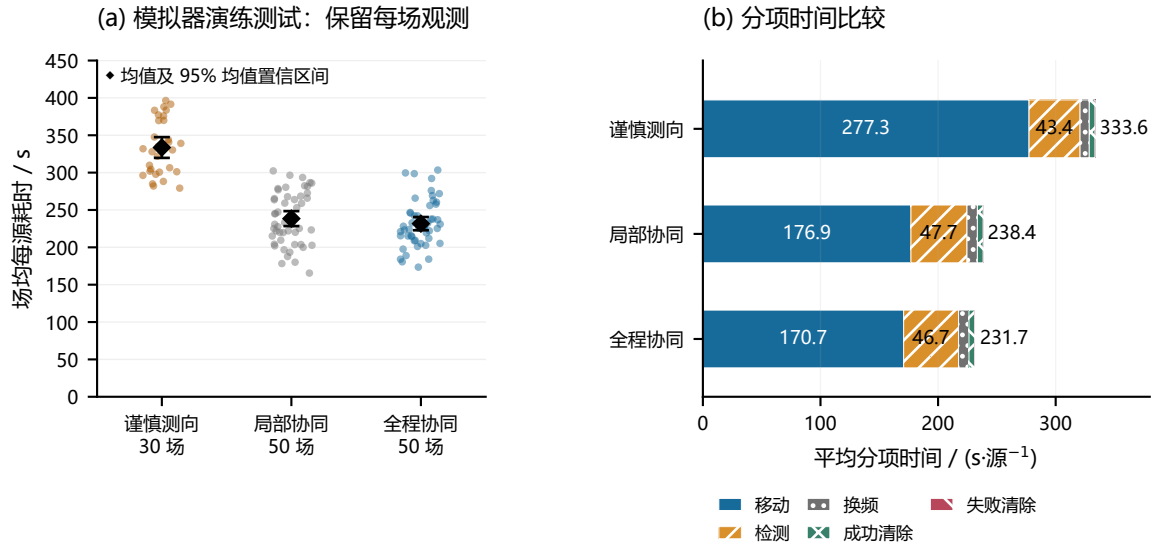


图 9 模拟器演练测试的逐场耗时与费用分解。左图保留 30、50、50 场观测，菱形与误差线为均值及其 95% 置信区间；右图各分项先除以本场源数，再对场次平均。独立案例比较不单独识别某一模块的因果收益。

全程协同较局部协同平均节省 6.72 秒/源 (2.82%)，其中移动时间减少 6.22 秒/源，但节省量区间 $[-6.48, 19.93]$ 包含零。另有 30 场独立的全程协同演练均完成全部清除，均值为 226.34 秒/源。因此，现有数据已显示其相对谨慎测向的明显改善，对局部协同的优势则仍需更多同场景对照检验。

7.6.2 同场景路线比较

为进一步控制场景差异，在统一校准本地模拟器中固定 30 组源位置、接收半径和同位置误差场，分别运行谨慎测向与全程协同。两组均完成全部清除，平均耗时由 328.54 降至 231.89 秒/源，30 组均有下降；配对平均节省 96.65 秒/源，95% 置信区间为 $[87.67, 105.63]$ 。本地场景分布是对官方环境的近似，这组结果用于比较相同行动条件下的策略差异。

图10选取节省量最接近配对中位数的场景。对于相同的 16 个源，谨慎测向逐源侧向定位后再前往固定扫描点；全程协同随任务路线调整扫描位置，将检测和清除穿插在连续行程中。路程由 20.87 降至 12.68 千米，节省 1639.11 秒；虽然多检测 12 次、多换频 23 次、试清失败 5 次，共增加 98 秒，整场仍节省 1541.11 秒。该场景说明，以少量观测费用换取较大的行程缩短，是协同安排的主要作用。

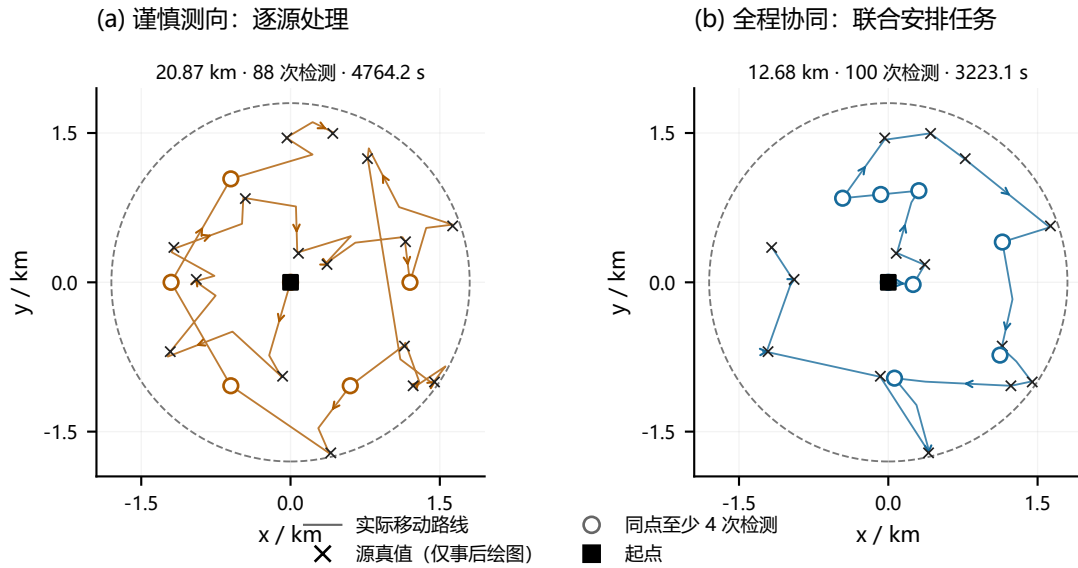


图 10 同一场景的本地路线对照（种子 2026092410）。坐标、比例与源真值相同，连线来自实际动作，箭头示方向；圆点为至少执行四次检测的位置。真值只用于事后绘图，不输入策略。

除上述完整实验外，历史本地测试还发现一次异常：在估计试清失败后的后续耗时，假想续跑区域发生数值退化，估价异常传播至主程序并导致退出；当时实际观测对应的位置域仍有效。该记录属于未完成任务，不能计为低耗时结果。后续仍需完善退化区域处理，并检验先验、误差场和参数变化的影响。

7.6.3 三次正式测试记录

全程协同的三次正式测试结果见表4。正式元数据未提供真实源总数，故表中报告成功清除数。

表 4 第三问已提交的正式测试记录

测试案例编码	清除源数	平均定位清除时间/s	程序运行时间/s
BXUJ-W9QZ-KQ54-NKCU	15	195.399	8.619
QNXC-8U8P-SQBF-A2A6	12	276.748	6.149
89UC-2392-TZ5M-ZUTB	10	248.719	6.076

平均定位清除时间计入搜索和失败操作；程序运行时间取官方接受进入与退出请求的时间差。前者反映题目规定的动作费用，后者反映实际运行耗时。

八、问题四的模型与求解

8.1 定向接收与观测约束

第三问以测角反馈缩小源的位置区域，并协调搜索、定位与清除。加入定向源后，几何上有利的测点可能位于辐射背面，无法获得方位信息；无信号也同时具有超距和背向两种解释。若仍沿用全向源的排除规则，就可能删去真实位置。因此，本问构造适用于任意朝向的搜索布局，并结合有信号与无信号的历史观测，联合约束位置、半径和朝向，形成联合域规划策略。

Tokekar 与 Isler[1] 用有界测角误差构造几何可行域，为前文的扇形求交提供了依据。Vander Hook 等 [4] 同时考虑移动与测量时间，沿主要不确定方向的垂线谨慎接近目标，可用于建立测点选择基线。其高斯估计与方位歧义设定不同于本题，应用时需改写状态更新方式，并另行处理定向源的接收条件。

针对方向性，赵倩倩等 [7] 在无人机部署中考虑目标发射天线的方向，提示应将源位置与朝向共同作为未知量；与其连续增益模型不同，本文采用题设的半圆可见性条件。Song 等 [6] 依据多个未知无线电源的空间概率分布引导机器人搜索，提供了利用更新后的位置分布规划运动的思路。其 RSS 似然依赖信号强度，本题仅反馈方位和接收状态，故下文重新构造相容条件。

沿用第三问的区域 Ω 、位置域 K 、最小包围圆中心 $c(K)$ 和半径 $\rho(K)$ 。以 x 表示未知源位置 g 的候选值，作用半径 $R \in [1000, 1500]$ m，类型 $z = 0, 1$ 分别表示全向与定向，单位向量 n 表示发射朝向。机器人在 q 处的可接收状态为

$$h(q; x, R, z, n) = \mathbf{1}\{\|q - x\| \leq R\} \mathbf{1}\{z = 0 \text{ 或 } n^\top(q - x) \geq 0\}. \quad (26)$$

有效接收后还要满足测角误差或 5 m 近场条件。这里的发射朝向不同于测得的源方位，图11说明了这种差异。

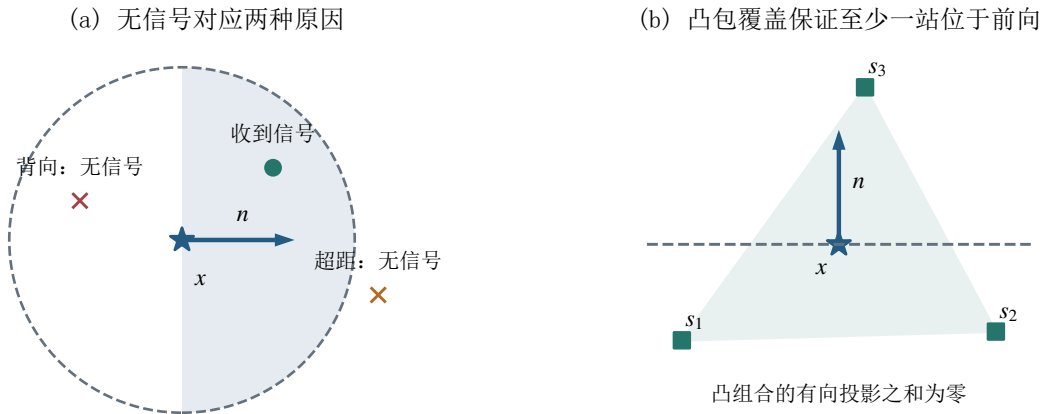


图 11 无信号歧义与方向鲁棒搜索原理。左图中背向和超距均产生无信号；右图中源位于站点凸包内，任意朝向下至少一站位于前向。此图为机制示意。

8.2 从谨慎测向到联合域规划

沿用第三问的文献适配思路，构造**谨慎测向策略**。以位置域凸包的均匀面积均值 μ 和协方差近似原文估计，令 $\sigma_{\max}$ 为最大特征值的平方根， v 为相应特征向量的垂直方向。按文献 [4] 的 Algorithm 1，在 $\mu \pm av$ 中选择离当前位置较近者，其中

$$a = \frac{\sigma_{\max}}{\sqrt{\sigma_{\beta_c}^2 - \sigma_s^2}}, \quad \sigma_{\beta_c} = \frac{\pi}{2\Phi^{-1}(1 - \beta_c/2)}. \quad (27)$$

本问在测试前固定 $\beta_c = 0.05$ ， σ_s^2 取 $\pm 1^\circ$ 均匀误差的二阶矩。位置域仍由有界几何约束更新，清除仍按包围圆判定。若无信号使区域不变，且两个候选点都已测量，则以 30° 步长旋转同半径候选；无可测点时转入有限光学覆盖。该策略保留原文的谨慎选点思想，以适应本题的固定测角误差和定向接收条件。

谨慎测向只比较交会几何和移动距离，未评价候选点的可接收性。为此，先构造**可见性评分策略**：以包围圆中心代表目标，用有限的半径、朝向假设估计接收可能性，将失联后的恢复时间纳入测点评分，并共同安排目标与搜索站点的访问次序。这样可避开部分低效测点，但无信号仅影响行动评分，没有直接缩小连续位置域；包围圆中心也未区分候选位置与观测历史的相容程度。

联合域规划策略进一步将位置、半径、类型与朝向共同代入历史观测，删除已不可能成立的位置块，再按相容程度计算用于路线规划的代表位置。表5概括了三种策略的差别。三者均使用 22 站搜索布局、路线组合、顺路测量、数量上界停止规则和有限清除回退。各策略代码与参数在相应演练前固定，比较重点是接收历史如何参与选点和位置更新。

表 5 对照策略与逐步解决的问题

策略	决策依据	仍存在的问题或新增作用
谨慎测向	沿主要不确定方向的垂线测量，选较近侧	有利交会角未必可接收，可能反复失联后进入光学覆盖
简洁可见性评分	有限假设估计可见性与行动费用，包围圆中心用于规划	无信号不直接裁剪连续位置域，目标代表位置较粗
联合域规划	保留可见性评分框架，加入联合域证书和后验代表位置	将接收历史同时用于几何排除与运动规划；保证条件与启发式排序分开

8.3 任意朝向下的搜索覆盖

仅以接收圆盘覆盖目标区域不足以保证定向源被发现。对任意候选源位置 x ，若存在站点子集 S_x 满足

$$x \in \text{conv}(S_x), \quad \max_{s \in S_x} \|s - x\| \leq 1000 \text{ m}, \quad (28)$$

则任意朝向下均能在至少一站收到信号。证明只需将 x 写成站点的凸组合：各站相对 x 的有向投影的加权和为零，因此不可能所有正权重站点均严格位于背向；至少一站前向且不超出最小作用半径。

采用中心、半径 970 m 的七点等角内环及半径 1850 m 的十四点等角外环，共 22 个站点，两环均从正横轴起始。外环略超出源分布圆域，为边界源提供外侧观测位置。对覆盖半径 1800 m 圆域的 4,988 个矩形单元，逐一检查四角位于见证站点凸包内，且到各见证站点距离均小于 1000 m；最大见证距离为 999.975 m。矩形的凸性与距离函数的凸性将四角检查推广到整个单元，因而这是一份连续区域证书，而非离散源位置的抽样检验。该布局是可行配置，不主张站点数最少。

每站扫描尚未发现的频道，并保存包括无信号在内的全部反馈。访问完全部站点后，覆盖证书保证搜索完整；若提前发现 16 个不同源，则由题目数量上界提前结束搜索。随后继续处理已发现未清除源，直至全部完成。搜索结束由覆盖完成或达到数量上界判定。

8.4 无信号约束下的联合区域裁剪

对每个频道，将与全部观测历史相容的参数组成联合域 Θ_t ，并以 K_t 保守外包其位置投影。有效方位和近场反馈仍按第三问分别与扇形、5 m 圆盘求交；无信号反馈则须结合此前接收位置，判断超距与背向能否由同一组参数解释。

设有效接收位置集合为 $\mathcal{S}^+$ 、无信号位置集合为 $\mathcal{S}^-$ 。考虑一个由圆盘 $B(c, r)$ 外包的位置块。有效接收给出整块共享的半径下界，并确定无法由超距解释的无信号点：

$$L_B = \max\{1000, \max_{p \in \mathcal{S}^+} (\|p - c\| - r)\}, \quad \mathcal{S}_B^- = \{q \in \mathcal{S}^- : \|q - c\| + r < L_B\}. \quad (29)$$

若 $L_B > 1500$ ，该块不可能包含源。若 $\mathcal{S}_B^-$ 非空，该块内任何相容源都必须为定向型，且对 $\mathcal{S}_B^-$ 中各点背向，同时对 $\mathcal{S}^+$ 中各点前向。这些限制共享同一个朝向，不能为每次观测分别任意选择解释。

对有效接收点取 $d = p - c$ ，对被迫背向点取 $d = c - q$ 。当 $\|d\| > r$ 时，块内位置变化引起的向量方向偏移至多为 $\arcsin(r/\|d\|)$ ；因此相容朝向必在以 $\arg d$ 为中心、半宽为 $\pi/2 + \arcsin(r/\|d\|)$ 的闭圆周区间 $A(d, r)$ 内。当 $\|d\| \leq r$ 时不加入该项限制。若 $\mathcal{S}_B^- \neq \emptyset$ 且

$$\bigcap_{p \in \mathcal{S}^+} A(p - c, r) \cap \bigcap_{q \in \mathcal{S}_B^-} A(c - q, r) = \emptyset, \quad (30)$$

则整个位置块均可排除。由于各角区间由必要条件放宽得到，只有交集为空才能删除；非空或退化为单点的角区间均保留。这样便可利用无信号反馈，检验全部观测是否与同一接收半径和朝向相容。

实现沿最小旋转外接矩形的长边二分，块尺度降至 10 m 或处理数达到 4,096 时停止；未证伪区域全部保留。矩形和圆盘留微米量级外扩，距离判据留 10^{-5} m 余量，角区间再放宽 10^{-9} rad，避免边界误删。图12给出实际日志中的裁剪。所示包围圆半径由 22.42 m 降至 17.40 m，低于 19.5 m 的无需补测清除阈值，因此可移至包围圆圆心完成清除。

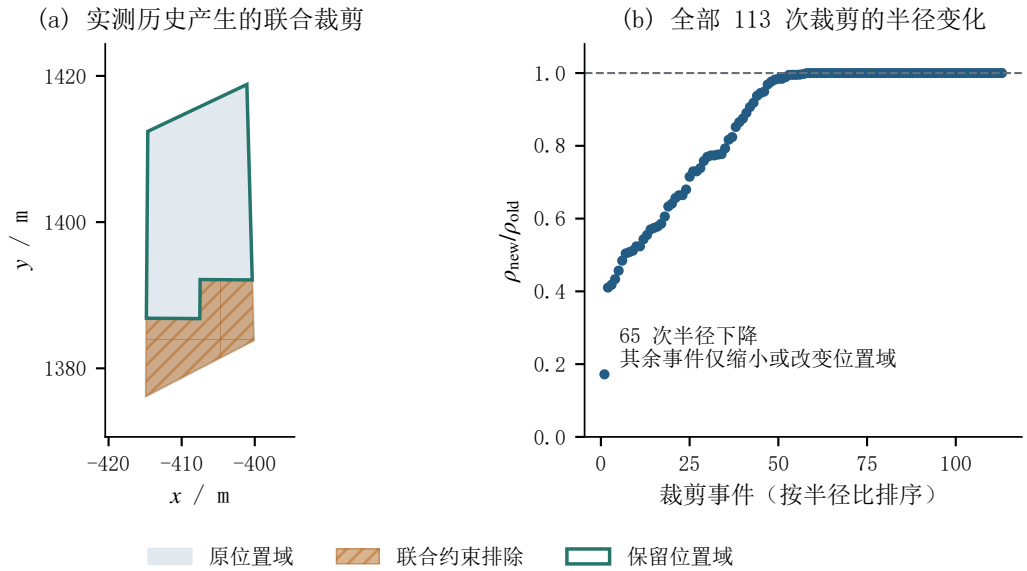


图 12 联合约束如何促成可靠清除。左图取全部 65 次半径严格下降事件按相对降幅排序的第 33 次，橙色斜线区为排除部分；右图展示联合域规划 30 场模拟器演练测试的全部 113 次裁剪，65 次降低包围圆半径，半径比中位数为 0.9972。事件非独立场景，不将 113 当作性能样本量。

8.5 路线、测点与清除位置的选择

路线规划还需要一个能够代表目标位置的点。联合域规划在区域内取均匀位置、均匀半径与朝向，并对全向、定向类型赋予相同先验权重。对每个候选位置，将相容的半

径和朝向积分后赋权，以加权平均位置生成路线节点和测点。位置积分用 128 个确定性 Sobol 面积样本近似；权重不足时退回几何代表点。上述先验服务于行动比较，不影响连续位置域的保守排除。

将剩余搜索站点和未清除目标的代表位置组成节点集，规划从当前位置出发、终点自由的开放路线。分别由最近邻、上一轮路线更新和最远种子插入构造初始解，经 2-opt 改进后取最短者。每完成一次站点扫描或一个目标的处理即重新规划。与第三问相同，本问也联合排序搜索和目标任务，但以代表位置近似目标处理位置，路线长度仅用于比较访问次序。

专用测点围绕代表位置及接近路径生成，包括位置域主轴的侧向点和中间点，剔除距已有测点不超过 2 m 的重复点。用至多 16 个代表位置、三个半径、24 个朝向及全向类型组成的历史相容假设，估计接收比例 $\hat{p}(q)$ 和有信号后的平均包围圆半径。选择预计耗时最小的点：

$$\hat{J}(q) = t_{\text{move}}(q) + t_{\text{radio}} + t_{\text{approach}}(q) + \hat{p}(q)t_{\text{loc}}(q) + (1 - \hat{p}(q))t_{\text{recover}}(q). \quad (31)$$

各项依次为移动、一次测量、靠近目标、接收后继续定位及失联恢复的时间估计。移动速度和测量费用取题设值；继续定位时间由剩余半径及其相对清除尺度的对数估计，失联恢复时间由固定基础项和当前区域尺度估计，具体系数见配套代码。式(31)是候选点排序的启发式指标，其中有限假设等权；前述代表位置则由连续先验积分加权，两者承担不同计算任务。

到达搜索站点时，还对仍需定位、距历史测点足够远且视角变化或接近程度足够大的已知频道顺路测量。由此一次移动可以服务搜索与定位两类需求。若 $\rho(K_t) \leq 19.5$ m，则移至 $c(K_t)$ 清除，20 m 光学范围覆盖整个位置域；源的发射类型不影响这一判断。每次目标服务至多进行 10 次专用无线电测量，随后用边长 25 m 的相交方格中心实施有限光学覆盖，因为方格外接圆半径仅 17.68 m。失败后保守排除已检查圆盘，在未排除候选中继续选择。有限覆盖的完整性来自几何覆盖，顺序则按单位移动及失败费用的排除面积排序。

搜索布局与保守位置域分别提供发现和清除的几何依据，代表位置、路线和测点评分则用于降低耗时。有限光学覆盖可在测向进展缓慢时继续完成清除，但整场能否满足官方时间预算仍取决于实际行动费用。

8.6 策略比较与结果分析

8.6.1 模拟器演练测试比较

谨慎测向新增 30 次模拟器演练测试，可见性评分和联合域规划各采用既有 30 场完整记录，共 90 个不同案例。每场以总虚拟时间除以官方源数，再对各场结果取算术平均；均值区间采用 29 个自由度的 Student t 分布，策略差异采用独立样本 Welch 区间。全部案例均纳入统计，结果见表6和图13。

表 6 三种策略的模拟器演练测试比较

策略	全清场数	均值	均值 95% 区间 时间单位：s/源	第 90 百分位
谨慎测向	30/30	651.83	[587.29, 716.36]	825.11
可见性评分	30/30	496.70	[456.42, 536.98]	636.08
联合域规划	30/30	458.37	[416.05, 500.69]	588.52

联合域规划在 30 场中清除 390 个源，平均耗时 458.37 s/源。相对谨慎测向，平均节省 193.46 s/源 (29.68%)，节省量 95% 置信区间为 [117.67, 269.25] s/源；相对可见性

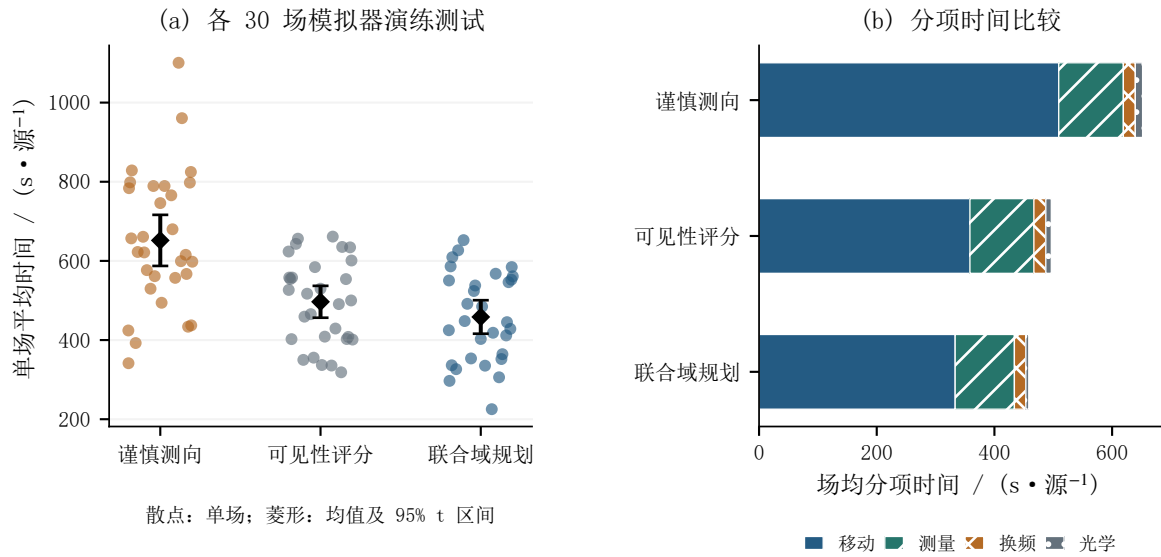


图 13 全部 90 场独立模拟器演练测试及分项计费。左图展示每场结果、场均值与 95% 均值区间；右图先对每场分项时间除以该场源数再平均。光学费用包括失败尝试，图中仅统计虚拟时间。

评分，平均节省 38.33 s/源 (7.72%)，区间为 $[-18.85, 95.51]$ s/源。前一差异较明确，后一差异仍可能由场景波动引起。

从已发现频道的后续观测看，谨慎测向出现 617 次无信号测量和 928 次失败光学尝试，可见性评分分别为 393 次、592 次，联合域规划为 194 次、0 次。谨慎测向的侧向点虽有利于交会，却可能落在辐射背面；可见性评分减少这类选择，联合域规划再利用历史反馈更新位置区域和规划代表点。

分项费用与上述变化一致：移动时间由谨慎测向的 509.27 降至联合域规划的 333.31 s/源，占平均总时间差的约 91%；相对可见性评分，移动与无线电测量分别减少 25.05 和 8.07 s/源。三组平均源数为 12.77、12.87、13.00，定向源比例也不完全相同，因此这些费用和频次主要用于分析整体行为变化，各改进环节的独立贡献还需同场景消融实验。

8.6.2 同场景路线比较

在统一校准本地模拟器中，另固定 30 组场景及同点误差，对三种策略逐场比较。三者均完成全部清除，谨慎测向、可见性评分和联合域规划的均值分别为 601.51、505.08、484.70 s/源。联合域规划较可见性评分在 25 场更快、5 场更慢，平均节省 20.38 s/源，配对差值 95% 置信区间为 $[11.38, 29.37]$ s/源。本地场景和误差分布是对官方环境的近似，这组实验用于检查相同行动条件下的改进机制。

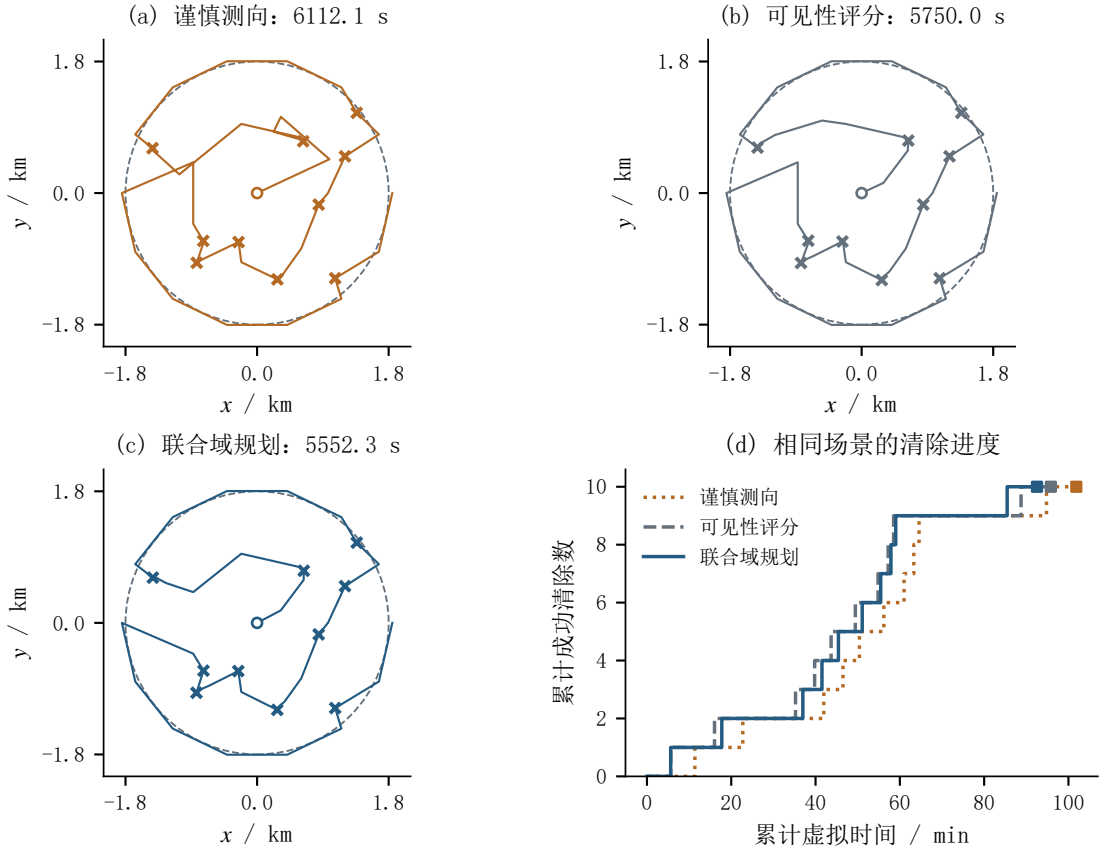


图 14 三策略在同一场景的路径与清除进度。将全部 30 个本地场景按可见性评分与联合域规划的耗时差（前者减后者，单位为秒/源）排序，取第 15 场，含 10 个源，其中 3 个为定向源；圈为起点，叉为成功清除时的停留位置，不表示源的真实位置。坐标等比例且范围相同；进度曲线延续至搜索完成，末端方块表示任务结束，时间计入全部付费动作。

图14所示场景中，可见性评分与联合域规划分别移动 20145.21 m 和 19346.71 m，测量 281 次和 273 次，换频 266 次和 268 次。联合域规划少走 798.49 m，节省 159.70 s；少测 8 次，节省 40 s；多换频 2 次，增加 2 s，合计节省 197.70 s。谨慎测向总耗时 6112.14 s，联合域规划为 5552.34 s。清除进度曲线表明，联合域规划并非始终领先，但较短的整体行程使其最终更早完成任务。

联合域规划的官方日志中共排除 1374 个位置块，113 次裁剪有 65 次降低包围圆半径，半径比中位数为 0.9972。图12说明裁剪在部分状态下可直接促成清除，但多数变化较小。另以仅采用相同后验代表位置、不进行联合裁剪的**后验规划策略**作历史对照，两者独立模拟器演练测试的均值差为 12.53 s/源，95% 置信区间 $[-41.84, 66.90]$ 跨零。因此，当前实验尚不足以单独确定联合裁剪的时间收益。

8.7 正式测验结果

联合域规划已完成三次官方正式测验，结果见表7，官方历史页面均显示已上传。平均定位清除时间取累计虚拟时间与成功清除数之比；程序运行时间取官方接受退出和进入请求的时间戳之差，包含接口交互时间。

表 7 联合域规划的三次官方正式测验结果

案例编码	成功清除数	平均定位清除时间/s	程序运行时间/s
9N77-TH84-ME6J-NRK4	10	557.293	4.325
QGEZ-KRWA-MNGB-6CMH	11	525.395	4.426
KTZ8-EZQ5-937B-QNEV	11	501.464	4.530

九、模型检验

9.1 几何计算与完成依据的核验

前文以最小包围圆半径不超过 20 米作为保证清除的统一判据并贯穿四问，检验即从这一判据的计算开始。第一问以角度判定与半平面判定对照、顶点算法与独立凸优化对照，检查交会区域、最小包围圆及最近保证清除点的数值一致性；相关样本与误差见第一问。第二问的反馈预测保留接收半径与地点误差固定，三种反馈概率之和为 1，期望损失与完成概率均在所声明的先验下解释。

第三、四问再分离局部定位与整场完成分别核验。第三问对每个未发现频道保存实际负观测，以整网格覆盖确认无遗漏，历史官方场景的七个未发现频道均通过 13,104 个含边界单元的检查。第四问用覆盖目标圆域的 4,988 个矩形单元校核 22 站搜索布局，并以放宽后的朝向必要条件删除不相容块。两问清除均沿用 19.5 米判据，计划观测与离散规划样本不替代实际完成证据。

9.2 整场表现与证据范围

第三问谨慎测向 30 场、全程协同 80 场模拟器演练测试均全清，均值分别为 333.59、231.71、226.34 秒/源；30 组同场景本地配对平均节省 96.65 秒/源。第四问谨慎测向、可见性评分、联合域规划各 30 场均全清，均值分别为 651.83、496.70、458.37 秒/源，联合域规划相对前两者平均节省 29.68% 与 7.72%。以上结果均来自各问既有记录，不将跨问题时间差解释为某一模块的因果影响，也不将独立场景比较当作同场配对实验。

计费组成显示两问移动时间均超过总均值的七成，说明减少移动是联合安排搜索与目标处理的关键。第四问联合域规划相对谨慎测向的节省量 95% 区间为 [117.67,269.25] 秒/源，相对可见性评分仍含零，整体对照尚不能单独识别联合裁剪的收益。因此，现有证据支持策略在已测场景的可用性，但不足以保证全局时间最优或所有场景预算内全部清除。

第三问三次正式测试成绩与程序耗时见表4，策略版本依据实际运行者提供的更正信息确认；第四问三次正式测验见表7，分别成功清除 10、11、11 个源，平均定位清除时间为 557.293、525.395、501.464 秒。正式元数据未给出实际源总数，故不据此宣称全清。

十、模型评价与推广

10.1 模型的优点

四问共享一致的几何依据。第一问用角域求交构造可行域并建立最小包围圆判据，第二问据此评价不同反馈留下的区域，第三、四问沿用其判断能否可靠清除，并始终区

分区域直径、最小包围圆半径与当前位置能否清除，避免仅凭直径或代表位置下强判断。

完整性判断与效率估价各有依据。搜索覆盖与保守位置域用于排除和清除判断，概率模型、有限情形试算与路线估计用于比较行动；第四问保留无信号历史，结合半径与朝向固定约束源位置，避免背向误判为超距。

连接数学条件与实际行动。策略给出站点、下一测点、清除位置、任务次序与停止条件，依据实际计费动作重建轨迹、统计耗时，以图表展示全部样本与代表场景，便于核对。

10.2 模型的局限性

第二问的期望评价与第四问的规划代表位置依赖工作先验，改变先验可能改变优选行动，数值结果不代表题目未规定的真实分布。第三问的有限步评分与第四问的测点代价含经验系数，路线为启发式求解，均无全局最优性保证。

保守几何近似可能保留多余位置、增加测量，网格与多边形分辨率还需权衡开销；第四问达到细分预算后保留未证伪区域以维持保守性，可能削弱裁剪效果；第三问冻结实现的假想续跑区域数值退化还曾异常退出，几何完成证书不保证软件无异常。本文尚未对先验、细分尺度与各评分系数完成系统敏感性分析。

现有场景有限且官方比较使用不同案例，样本全清不能覆盖所有边界情形，未触发的回退分支也不能认定已获整场性能验证。后续检验单项机制应采用相同场景的独立对照。

10.3 模型的推广

若接收半径、测角精度或清除距离变化，可替换相应观测约束与动作成本并重新校准搜索布局；存在障碍时应把直线距离换成可行路径长度并重查观测可达性，现有无障碍结论不能直接沿用。对一般定向辐射形状，可在位置、尺度与朝向的联合参数空间中构造相容条件，仍以保守域作可靠判断、以代表状态辅助规划。

参考文献

- [1] Tokekar P, Isler V. Sensor placement and selection for bearing sensors with bounded uncertainty[C]. IEEE International Conference on Robotics and Automation (ICRA), 2013. DOI: 10.1109/ICRA.2013.6630920.
- [2] Gholami M R, Gezici S, Wymeersch H, et al. Characterizing the worst-case position error in bearing-only target localization[C]. Workshop on Positioning, Navigation and Communication (WPNC), 2015.
- [3] Welzl E. Smallest enclosing disks (balls and ellipsoids)[C]. New Results and New Trends in Computer Science, LNCS 555, 1991: 359–370. DOI: 10.1007/BFb0038202.
- [4] Vander Hook J, Tokekar P, Isler V. Cautious greedy strategy for bearing-only active localization: analysis and field experiments[J]. Journal of Field Robotics, 2014, 31(2): 296–318. DOI: 10.1002/rob.21499.
- [5] Engin S, Isler V. Active localization of multiple targets using noisy relative measurements[EB/OL]. arXiv:2002.09850, 2020.
- [6] Song D, Kim C Y, Yi J. Simultaneous localization of multiple unknown and transient radio sources using a mobile robot[J]. IEEE Transactions on Robotics, 2012, 28(3): 668–680. DOI: 10.1109/TRO.2012.2183069.
- [7] 赵倩倩, 熊刚, 王李军, 等. 面向未知定向辐射源组合定位的无人机群优化部署 [J]. 信号处理, 2025, 41(4): 668–682.

十一、附录：问题求解程序

本附录列出四问正式求解程序及其依赖模块，源码直接读取自随文提交的“支撑材料”目录，以下路径均相对于该目录。完全相同的公共文件只展示一次，其余位置注明对应关系。程序中的注释、参数及计算逻辑均按提交版本保留，长行仅在排版时自动折行。

第一、二问分别以 `run_q1.py`、`run_q2.py` 为入口；第三问以 `q3/v6/run_robot.py --policy v6` 为入口；第四问以 `q4/M32/run_robot.py --policy joint` 为入口。第一、二问的样例输入、依赖安装说明，以及第三、四问连接题目模拟器所需的配置说明，见支撑材料中的 `README.md`。历史对照策略、绘图脚本及原始运行日志另见支撑材料。

11.1 支撑材料清单

表 8 列出随文提交的主要程序、数据和复核资料。表中路径均以“正式题目工作区/B 题/支撑材料”为根目录，程序文件的具体内容见本附录后续各节。

表 8 支撑材料清单

材料类别	文件或目录	内容及用途
说明与环境	<code>README.md</code> <code>requirements.txt</code> <code>q1-q2/requirements.txt</code>	目录说明、依赖及复现步骤。
第一问程序	<code>q1-q2/q1/</code> <code>q1-q2/run_q1.py</code>	角域求交、顶点与直径计算；入口为 <code>run_q1.py</code> 。
第二问程序	<code>q1-q2/q2/</code> <code>q1-q2/run_q2.py</code> <code>q1-q2/build.py</code>	反馈评价、权重搜索与 C++ 计算内核；入口为 <code>run_q2.py</code> 。
一二问输入与结果	<code>q1-q2/examples/</code> <code>q1-q2/reference_results/</code> <code>q1-q2/verification/</code>	算例输入、论文结果表及数值核验记录。
一二问复算	<code>q1-q2/reproduce_paper.py</code> <code>q1-q2/verify.py</code>	批量复算论文结果与独立数值检查。
第三问程序	<code>q3/v6/</code>	全程协同策略及依赖；入口为 <code>run_robot.py</code> ，参数 <code>-policy v6</code> 。
第四问程序	<code>q4/M32/</code>	联合区域规划策略、依赖及 <code>ring.json</code> 几何数据；入口为 <code>run_robot.py</code> ，参数 <code>-policy joint</code> 。
对照程序	<code>q3/baselines/</code> <code>q4/baselines/</code>	第三、四问的文献适配策略与历史对照。
统计与绘图	<code>q3/analysis/</code> <code>q4/analysis/</code>	逐场统计、配对比较、代表轨迹及图表重绘。
正式测试资料	<code>q3/formal/</code> <code>q4/formal/</code> <code>formal-results.json</code>	共六份原始 <code>jlog</code> 、指令响应记录及正式测试结果。
本地复核环境来源与校验	<code>local-simulator-snapshot/</code> 四问归集来源.json <code>SHA256SUMS.json</code> <code>verify_materials.py</code> 归集记录/ 归集核验/	本地模拟器审计快照，用于本地复核。 四问材料来源、文件哈希、原始归档记录及完整性核验。
补充说明	<code>AI 工具使用详情.pdf</code> <code>提交要求核对.md</code> <code>论文附录/</code>	三四问 AI 使用记录、提交条款核对及原始附录素材。

第三、四问各提交三份原始正式日志，具体文件名保留在相应的 `formal/logs/` 目录中；`formal/records/` 保存对应的动作与响应记录。四问完整材料以本表所列目录为准，三四问支撑材料.zip 为第三、四问原始归档。AI 使用详情和原始附录素材目前覆盖第三、四问。

11.2 问题一：定位区域与直径计算

文件 q1-q2/run_q1.py

```
1 """ 由 JSON 观测文件求解第一问。示向度以正东为零、逆时针为正，单位为度。 """
2 import argparse,json
3 from pathlib import Path
4 from q1.solve import solve
5
6
7 def main():
8     parser=argparse.ArgumentParser(description=__doc__)
9     parser.add_argument('input',type=Path)
10    parser.add_argument('--out',type=Path,help='新建 JSON 输出文件，不覆盖已有结果')
11    args=parser.parse_args()
12    try:
13        result=solve(json.loads(args.input.read_text(encoding='utf-8')))
14        text=json.dumps(result,ensure_ascii=False,indent=2,allow_nan=False)
15        if args.out:
16            args.out.parent.mkdir(parents=True,exist_ok=True)
17            with args.out.open('x',encoding='utf-8') as f:f.write(text+'\n')
18        print(text)
19    except (ValueError,RuntimeError,FileExistsError) as e:parser.exit(2,f'错误: {e}\n')
20
21 if __name__=='__main__':main()
```

文件 q1-q2/q1/__init__.py

本文件为空文件，用于标识 Python 包。

文件 q1-q2/q1/bearing_geometry.py

```
1 """Q1 M01 v01: directed bearing wedges, polygon diameter and enclosing disk.
2
3 All coordinates are metres, public angles degrees, inequalities A @ x <= b.
4 This module makes no stochastic noise assumption and no simulator requests.
5 """
6 from dataclasses import dataclass
7 from itertools import combinations
8 import numpy as np
9 from scipy.optimize import linprog
10
11
12 @dataclass
13 class Region:
14     status: str
15     vertices: np.ndarray
16     max_residual: float = 0.0
17
18
19 def unit(degrees):
20     a = np.deg2rad(degrees)
21     return np.array([np.cos(a), np.sin(a)])
22
23
24 def bearing(point, sensor):
25     d = np.asarray(point) - np.asarray(sensor)
26     return float(np.rad2deg(np.arctan2(d[1], d[0]))) % 360
27
28
29 def angle_difference(a, b):
30     return (np.asarray(a) - np.asarray(b) + 180) % 360 - 180
31
32
33 def wedge(sensor, angle_deg, epsilon_deg=1.0):
34     """Closed, forward wedge; valid for 0 < epsilon < 90 degrees."""
35     if not 0 < epsilon_deg < 90:
36         raise ValueError("epsilon must lie strictly between 0 and 90 degrees")
37     s = np.asarray(sensor, dtype=float)
38     lower, upper = unit(angle_deg - epsilon_deg), unit(angle_deg + epsilon_deg)
39     A = np.array([[lower[1], -lower[0]], [-upper[1], upper[0]]])
40     return A, A @ s
```

```

41
42
43 def observations_halfplanes(observations, epsilon_deg=1.0):
44     parts = [wedge(o['position'], o['bearing_deg'], epsilon_deg) for o in observations]
45     if not parts:
46         return np.empty((0, 2)), np.empty(0)
47     return np.vstack([p[0] for p in parts]), np.hstack([p[1] for p in parts])
48
49
50 def unique_points(points, tol=1e-7):
51     kept = []
52     for p in points:
53         if not any(np.linalg.norm(p - q) <= tol for q in kept):
54             kept.append(p)
55     return np.array(kept, dtype=float).reshape(-1, 2)
56
57
58 def halfplane_region(A, b, tol=1e-7):
59     """Classify without an artificial clipping box; enumerate bounded vertices.
60
61     Four LP support problems distinguish unbounded sets from bounded polygons.
62     Pairwise line intersections cost  $O(m^3)$ , appropriate for few observations.
63     The result is floating point, not interval-arithmetic certification.
64     """
65     A, b = np.asarray(A, float).reshape(-1, 2), np.asarray(b, float)
66     if not len(A):
67         return Region('unbounded', np.empty((0, 2)))
68     lengths = np.linalg.norm(A, axis=1)
69     zero = lengths < 1e-15
70     if np.any(b[zero] < -tol):
71         return Region('empty', np.empty((0, 2)))
72     A, b = A[~zero] / lengths[~zero, None], b[~zero] / lengths[~zero]
73     if not len(A):
74         return Region('unbounded', np.empty((0, 2)))
75     feasible = linprog([0, 0], A_ub=A, b_ub=b, bounds=[(None, None)] * 2,
76         ↪ method='highs')
77     if feasible.status == 2:
78         return Region('empty', np.empty((0, 2)))
79     if not feasible.success:
80         raise RuntimeError(feasible.message)
81     # Translation improves conditioning for noncentral observations.
82     origin = feasible.x
83     rhs = b - A @ origin
84     extrema = []
85     for c in [[1, 0], [-1, 0], [0, 1], [0, -1]]:
86         sol = linprog(c, A_ub=A, b_ub=rhs, bounds=[(None, None)] * 2, method='highs')
87         if sol.status == 3:
88             return Region('unbounded', np.empty((0, 2)))
89         if not sol.success:
90             raise RuntimeError(sol.message)
91         extrema.append(sol.x)
92     points = []
93     for i, j in combinations(range(len(A)), 2):
94         rows = A[[i, j]]
95         if abs(np.linalg.det(rows)) < 1e-12:
96             continue
97         x = np.linalg.solve(rows, rhs[[i, j]])
98         if np.max(A @ x - rhs) <= tol:
99             points.append(x)
100     if not points:
101         points = extrema # Degenerate point/segment fallback, never arbitrary box.
102     vertices = unique_points(points, tol) + origin
103     if len(vertices) > 2:
104         center = vertices.mean(axis=0)
105         order = np.argsort(np.arctan2(vertices[:, 1] - center[1], vertices[:, 0] -
106             ↪ center[0]))
107         vertices = vertices[order]
108     status = 'point' if len(vertices) == 1 else 'segment' if len(vertices) == 2 else
109     ↪ 'polygon'
110     return Region(status, vertices, float(max(0.0, np.max(A @ vertices.T - b[:,
111     ↪ None]))))
112
113 def diameter(vertices):
114     p = np.asarray(vertices, float).reshape(-1, 2)

```

```

112     if not len(p):
113         raise ValueError('diameter needs a nonempty bounded region')
114     d2 = np.sum((p[:, None] - p[None, :]) ** 2, axis=2)
115     i, j = np.unravel_index(np.argmax(d2), d2.shape)
116     return float(np.sqrt(d2[i, j])), (int(i), int(j))
117
118
119 def enclosing_circle(vertices, tol=1e-8):
120     """Enumerate support disks (2 or 3 points), with a diameter-disk shortcut.
121
122     Exact finite-support algorithm in real arithmetic;  $O(h^4)$  worst case.
123     Final radius is recomputed from all points, giving a conservative numerical
124     covering disk even when tolerance affects the support choice.
125     """
126     p = np.asarray(vertices, float).reshape(-1, 2)
127     D, (i, j) = diameter(p)
128     center = (p[i] + p[j]) / 2
129     if np.max(np.linalg.norm(p - center, axis=1)) <= D / 2 + tol:
130         return center, float(np.max(np.linalg.norm(p - center, axis=1))), [i, j]
131     best = (np.inf, None, None)
132     for i, j, k in combinations(range(len(p)), 3):
133         d = p[[j, k]] - p[i]
134         determinant = np.linalg.det(d)
135         if abs(determinant) <= 1e-13 * max(1., np.linalg.norm(d[0]) *
136             ↪ np.linalg.norm(d[1])):
137             continue
138         offset = np.linalg.solve(2 * d, np.sum(d * d, axis=1))
139         r = np.linalg.norm(offset)
140         if r >= best[0] + tol:
141             continue
142         c = p[i] + offset
143         distances = np.linalg.norm(p - c, axis=1)
144         if distances.max() <= r + tol:
145             best = (float(distances.max()), c, [i, j, k])
146     if best[1] is None:
147         raise ArithmeticError('Could not construct a covering disk; inspect
148             ↪ conditioning')
149     return best[1], best[0], best[2]
150
151 def summarize_region(region):
152     if region.status == 'empty':
153         return {'status': 'empty', 'diameter_m': None, 'cover_radius_m': None}
154     if region.status == 'unbounded':
155         return {'status': 'unbounded', 'diameter_m': 'infinity', 'cover_radius_m':
156             ↪ 'infinity'}
157     D, pair = diameter(region.vertices)
158     c, r, support = enclosing_circle(region.vertices)
159     return dict(status=region.status, vertices=region.vertices.tolist(), diameter_m=D,
160         diameter_pair=list(pair), cover_center=c.tolist(), cover_radius_m=r,
161         support_indices=support, diameter_disk_covers=bool(r <= D / 2 + 1e-7),
162         guaranteed_clear_possible=bool(r <= 20),
163         max_halfplane_residual_m=region.max_residual)
164
165 def clip_polygon(poly, normal, offset, tol=1e-9):
166     """Sutherland-Hodgman clipping; caller must supply a genuine prior polygon."""
167     p = np.asarray(poly, float).reshape(-1, 2)
168     if not len(p):
169         return p
170     values = p @ normal - offset
171     output = []
172     for i in range(len(p)):
173         j = (i + 1) % len(p)
174         inside_i, inside_j = values[i] <= tol, values[j] <= tol
175         if inside_i:
176             output.append(p[i])
177         if inside_i != inside_j:
178             t = values[i] / (values[i] - values[j])
179             output.append(p[i] + np.clip(t, 0, 1) * (p[j] - p[i]))
180     return np.asarray(output, float).reshape(-1, 2)
181
182 def clip_halfplanes(poly, A, b):
183     for a, rhs in zip(A, b):

```

```

184         poly = clip_polygon(poly, a, rhs)
185         if not len(poly):
186             break
187     return poly
188
189
190 def outer_disk(center, radius, sides=360):
191     """Circumscribed regular polygon, never an inscribed disk approximation."""
192     angles = (np.arange(sides) + .5) * 2 * np.pi / sides
193     return np.asarray(center) + radius / np.cos(np.pi / sides) *
        ↪ np.column_stack((np.cos(angles), np.sin(angles)))
194
195
196 def disk_halfplanes(center, radius, sides=360):
197     angles = np.arange(sides) * 2 * np.pi / sides
198     A = np.column_stack((np.cos(angles), np.sin(angles)))
199     return A, radius + A @ np.asarray(center)

```

文件 q1-q2/q1/clearance_geometry.py

```

1  """Euclidean projection onto the intersection of equal-radius clear disks."""
2  import math
3  import numpy as np
4
5
6  def nearest_clear_point(vertices, position, radius=19.5, feasible_point=None):
7      """Return the global nearest safe point, or None if no candidate is feasible.
8
9      An external projection lies on one circular arc, or a two-circle vertex.
10     Smooth-arc candidates are radial projections; vertices are enumerated.
11     A supplied feasible point is a numerical fallback, not an optimality claim.
12     """
13     v=np.asarray(vertices,float);p=np.asarray(position,float)
14     if not len(v):return None
15     def feasible(q):return np.linalg.norm(v-q,axis=1)<=radius+1e-8
16     if feasible(p):return p.copy()
17     candidates=[]
18     for center in v:
19         d=p-center;length=float(np.linalg.norm(d))
20         if length>1e-12:
21             q=center+radius*d/length
22             if feasible(q):candidates.append(q)
23     for i,a in enumerate(v):
24         for b in v[i:]:
25             d=b-a;length=float(np.linalg.norm(d))
26             if length<1e-10 or length>2*radius+1e-9:continue
27             offset=math.sqrt(max(0.,radius**2-length**2/4))*np.array([-d[1],d[0]])/len_
                ↪ gth
28             for q in [(a+b)/2+offset,(a+b)/2-offset]:
29                 if feasible(q):candidates.append(q)
30     if feasible_point is not None and feasible(np.asarray(feasible_point)):
31         candidates.append(np.asarray(feasible_point).copy())
32     if not candidates:return None
33     return min(candidates,key=lambda q:float(np.linalg.norm(q-p))).copy()

```

文件 q1-q2/q1/solve.py

```

1  """ 第一问完整流程：半平面交、区域直径、最小包围圆和最近保证清除位置。"""
2  import math
3  import numpy as np
4  from .bearing_geometry import observations_halfplanes,halfplane_region,summarize_region
5  from .clearance_geometry import nearest_clear_point
6
7
8  def solve(data):
9      observations=data.get('observations')
10     if not isinstance(observations,list) or not observations:raise
        ↪ ValueError('observations 必须是非空观测列表。')
11     for observation in observations:
12         p=np.asarray(observation.get('position'),float)

```

```

13         if p.shape!=(2,) or not np.all(np.isfinite(p)):raise ValueError('每个 position
    ↪ 必须是两个有限坐标。')
14         if not math.isfinite(float(observation.get('bearing_deg',float('nan')))):raise
    ↪ ValueError('每次观测需要有限的 bearing_deg。')
15     epsilon=float(data.get('epsilon_deg',1))
16     A,b=observations_halfplanes(observations,epsilon)
17     region=halfplane_region(A,b)
18     result=summarize_region(region)
19     result.update(observations=observations,epsilon_deg=epsilon,units={'length':'m','a
    ↪ ngle':'degree'},physical_clear_radius_m=20,
20                     halfplanes={'A':A.tolist(),'b':b.tolist()},
21                     model='directed_bearing_halfplanes_and_minimum_enclosing_circle')
22     result['clearance']=None
23     if region.status in ('empty','unbounded'):
24         result['message']='观测条件不相容。' if region.status=='empty' else '角域交集无界,
    ↪ 无法仅由这些测向约束给出有限包围圆。'
25     return result
26     result['physical_threshold_near_boundary']=abs(result['cover_radius_m']-20)<=1e-7
27     # Numerical tolerance is reported, not subtracted from the physical threshold.
28     if 'current_position' in data:
29         p=np.asarray(data['current_position'],float)
30         if p.shape!=(2,) or not np.all(np.isfinite(p)):raise
    ↪ ValueError('current_position 必须是两个有限坐标。')
31         radius=float(data.get('engineering_radius_m',19.5))
32         if not 0<radius<=20:raise ValueError('engineering_radius_m 必须在 (0,20] 内。')
33         c=np.array(result['cover_center'])
34         q=nearest_clear_point(region.vertices,p,radius,feasible_point=c)
35         result['clearance']={'engineering_radius_m':radius,'feasibility_tolerance_m':1
    ↪ e-8,'current_position':p.tolist(),
36                             'engineering_feasible':q is not None,'position':None if q
    ↪ is None else q.tolist(),
37                             'move_distance_m':None if q is None else
    ↪ float(np.linalg.norm(q-p)),
38                             'max_source_distance_m':None if q is None else float(np.m
    ↪ ax(np.linalg.norm(region.vertices-q,axis=1)))}
39     return result

```

文件 q1-q2/q1/validate_projection.py

```

1  """ 复现论文 200 组最近保证清除位置检验，保留原随机序列。 """
2  from pathlib import Path
3  import json
4  import numpy as np
5  from scipy.spatial import ConvexHull
6  from scipy.optimize import minimize
7  from .bearing_geometry import enclosing_circle
8  from .clearance_geometry import nearest_clear_point
9
10
11  def segment_clear_point(v,c,p):
12      if np.max(np.linalg.norm(v-p,axis=1))<=19.5:return p.copy()
13      low,high=0.,1.
14      for _ in range(45):
15          t=(low+high)/2;q=c+t*(p-c)
16          if np.max(np.linalg.norm(v-q,axis=1))<=19.5:low=t
17          else:high=t
18      return c+low*(p-c)
19
20
21  def run(out):
22      out=Path(out);out.mkdir(parents=True,exist_ok=False)
23      rng=np.random.default_rng(712233)
24      rows=[]
25      for i in range(200):
26          v=rng.normal(size=(rng.integers(3,14),2));v=v[ConvexHull(v).vertices]
27          center,r,_=enclosing_circle(v);v=(v-center)*rng.uniform(5,19.4)/r+rng.uniform(
    ↪ -1000,1000,2)
28          center,r,_=enclosing_circle(v);p=center+rng.normal(size=2)*300
29          q=nearest_clear_point(v,p,19.5,center);assert q is not None
30          vv=(v-center)/20;pp=(p-center)/20
31          objective=lambda x:float(np.sum((x-pp)**2)/2)
32          solution=minimize(objective,np.zeros(2),jac=lambda x:x-pp,method='SLSQP',

```

```

33         constraints={'type':'ineq','fun':lambda
34             ↪ x:(19.5/20)**2-np.sum((vv-x)**2,axis=1),
35             'jac':lambda x:2*(vv-x)},options={'ftol':1e-11,'maxiter':300})
36     qp=solution.x*20+center
37     assert np.max(np.linalg.norm(v-q,axis=1))<=19.5+1e-7
38     distance_error=abs(np.linalg.norm(q-p)-np.linalg.norm(qp-p))
39     assert distance_error<1e-4,(i,solution.message,distance_error)
40     oldq=segment_clear_point(v,center,p)
41     saved=float(np.linalg.norm(oldq-p)-np.linalg.norm(q-p))
42     assert saved>=-1e-6
43     rows.append(dict(case=i,saved_m=saved,independent_objective_error_m=float(dist_
44         ↪ ance_error),solver_success=bool(solution.success),solver_message=solution.m_
45         ↪ essage))
46     report={'cases':200,'seed':712233,'max_distance_difference_m':max(r['independent_o_
47         ↪ bjective_error_m'] for r in rows),
48         'mean_saved_m':float(np.mean([r['saved_m'] for r in rows])),
49         'max_saved_m':max(r['saved_m'] for r in rows),
50         'slsqp_success_count':sum(r['solver_success'] for r in rows),
51         'note':'SLSQP status is reported alongside objective/feasibility checks;
52         ↪ all comparisons use synthetic geometry.'}
53     (out/'cases.json').write_text(json.dumps(rows,indent=2));(out/'report.json').write_
54     ↪ _text(json.dumps(report,indent=2))
55     return report

```

文件 q1-q2/q1/validation.py

```

1  """ 复现论文第一问的 60 组观测几何检验，随机种子及抽样顺序与原记录一致。"""
2  from pathlib import Path
3  import json,time
4  import numpy as np
5  from scipy.optimize import linprog,minimize
6  from .bearing_geometry import
7  ↪ (observations_halfplanes,halfplane_region,summarize_region,
8  diameter,enclosing_circle,bearing,angle_difference,unit,wedge)
9
10 def run(out):
11     out=Path(out);out.mkdir(parents=True,exist_ok=False)
12     start = time.perf_counter()
13     seed = 20260910
14     rng = np.random.default_rng(seed)
15     checks = []
16     def check(name, condition, details=None):
17         checks.append(dict(name=name, passed=bool(condition), details=details))
18         if not condition:
19             raise AssertionError(f'{name}: {details}')
20
21     # A triangle realizable by THREE directed 2-degree bearing wedges.
22     L = 38.
23     triangle = np.array([[0, 0], [L, 0], [L/2, L*np.sqrt(3)/2]])
24     observations = []
25     for i in range(3):
26         u = (triangle[(i+1)%3] - triangle[i]) / L
27         s = triangle[i] - 1000*u
28         observations.append(dict(position=s.tolist(),
29             ↪ bearing_deg=(np.rad2deg(np.arctan2(u[1],u[0]))+1)%360))
30     A, b = observations_halfplanes(observations)
31     reg = halfplane_region(A,b)
32     tri = summarize_region(reg)
33     check('realizable_triangle_diameter', abs(tri['diameter_m']-L)<1e-7, tri)
34     check('realizable_triangle_radius', abs(tri['cover_radius_m']-L/np.sqrt(3))<1e-7)
35     target = triangle.mean(axis=0)
36     for o in observations:
37         check('triangle_direct_angle_consistency', abs(angle_difference(bearing(target_
38             ↪ ,o['position']),o['bearing_deg']))<=1+1e-10)
39         check('triangle_physical_reception', 5 < np.linalg.norm(target-o['position']) <
40             ↪ 1500)
41     check('diameter_38_not_clearable', tri['diameter_m']<40 and
42         ↪ tri['cover_radius_m']>20)
43     tri['observations'] = observations
44     tri['true_target_for_consistency'] = target.tolist()
45     (out/'counterexample.json').write_text(json.dumps(tri,ensure_ascii=False,indent=2)
46         ↪ ,encoding='utf-8')

```

```

42
43 # Orthogonal central bearings: independent slope-line construction below.
44 symmetric = [dict(position=[-1000,0],bearing_deg=0),dict(position=[0,-1000],bearing_
    ↪ g_deg=90)]
45 A0,b0=observations_halfplanes(symmetric)
46 sy = summarize_region(halfplane_region(A0,b0))
47 # It is not exactly a diamond about the origin: solve all four lines by hand
48 # in slope form (independent of the normal/LP representation).
49 t=np.tan(np.deg2rad(1.0)); expected=[]
50 for f in [-1,1]:
51     for g in [-1,1]:
52         expected.append(np.linalg.solve([[ -f*t,1],[1,-g*t]], [f*t*1000,g*t*1000]))
53 check('orthogonal_tangent_construction',
    ↪ abs(diameter(expected)[0]-sy['diameter_m'])<1e-7)
54 (out/'orthogonal_example.json').write_text(json.dumps(sy,indent=2),encoding='utf-8',
    ↪ ')
55
56 # Empty, unbounded, line and point must not be hidden by a finite box.
57 A1,b1=wedge([0,0],0)
58 check('single_wedge_unbounded',halfplane_region(A1,b1).status=='unbounded')
59 A2,b2=wedge([-10,0],180)
60 check('incompatible_wedges_empty',halfplane_region(np.vstack([A1,A2]),np.r_[b1,b2]
    ↪ ).status=='empty')
61 basic=np.array([[1,0],[-1,0],[0,1],[0,-1]])
62 check('segment_classification',halfplane_region(basic,[1,0,0,0]).status=='segment')
63 check('point_classification',halfplane_region(basic,[0,0,0,0]).status=='point')
64
65 # Direct atan2 membership independently validates angular signs and wrapping.
66 max_membership_mismatch=0
67 for theta in [0, .2, 89.8,90,179.9,270,359.8]:
68     s=rng.uniform(-2000,2000,2)
69     angles=theta+rng.uniform(-3,3,500)
70     points=s+rng.uniform(5,1500,(500,1))*np.column_stack((np.cos(np.deg2rad(angles)
    ↪ ),np.sin(np.deg2rad(angles))))
71     aa,bb=wedge(s,theta)
72     direct=np.abs(angle_difference(angles,theta))<=1
73     linear=np.all(points@aa.T<=bb+1e-9,axis=1)
74     max_membership_mismatch+=int(np.count_nonzero(direct!=linear))
75 check('atan2_vs_halfplanes_3500_points',max_membership_mismatch==0,max_membership_
    ↪ mismatch)
76
77 support_error=0.; circle_error=0.; containment_error=0.; equivariance_error=0.
78 for k in range(60):
79     x=rng.uniform(-700,700,2)
80     count=int(rng.integers(3,9))
81     angles=np.arange(count)*360/count+rng.uniform(-12,12,count)
82     sensors=x+rng.uniform(150,1450,(count,1))*np.array([unit(a) for a in angles])
83     obs=[dict(position=s.tolist(),bearing_deg=(bearing(x,s)+rng.uniform(-1,1))*360)
    ↪ for s in sensors]
84     aa,bb=observations_halfplanes(obs)
85     r=halfplane_region(aa,bb)
86     check('random_bounded_region',r.status=='polygon')
87     containment_error=max(containment_error,float(np.max(aa@x-bb)))
88     for a in rng.uniform(0,360,10):
89         u=unit(a)
90         lp=linprog(-u,A_ub=aa,B_ub=bb,bounds=[(None,None)]*2,method='highs')
91         support_error=max(support_error,abs(-lp.fun-np.max(r.vertices@u)))
92     c,radius,_=enclosing_circle(r.vertices)
93     # Independent convex norm epigraph minimization, scaled to unit geometry.
94     center=r.vertices.mean(axis=0);scale=max(1,np.max(np.linalg.norm(r.vertices-c)
    ↪ enter,axis=1)))
95     v=(r.vertices-center)/scale
96     sol=minimize(lambda z:z[2],np.r_[np.zeros(2),2.],method='SLSQP',
97         constraints={'type':'ineq','fun':lambda
    ↪ z:z[2]-np.linalg.norm(v-z[:2],axis=1)},
98         bounds=[(None,None),(None,None),(0,None)],options={'ftol':1e-10,'maxiter':50}
    ↪ 0})
99     check('independent_circle_solver_status',sol.success,sol.message)
100     circle_error=max(circle_error,abs(sol.x[2]*scale-radius))
101     check('circle_vertex_coverage',np.max(np.linalg.norm(r.vertices-c,axis=1))<=ra
    ↪ dius+1e-8)
102     check('diameter_circle_bounds',diameter(r.vertices)[0]/2<=radius+1e-7 and
    ↪ radius<=diameter(r.vertices)[0]/np.sqrt(3)+1e-7)

```

```

103     rot=unit(37);R=np.array([[rot[0],-rot[1]],[rot[1],rot[0]]]);shift=np.array([12
    ↪ 345.,-9876.])
104     obs2=[dict(position=(R*np.array(o['position'])+shift).tolist(),bearing_deg=(o[
    ↪ 'bearing_deg']+37)%360) for o in obs]
105     moved=halfplane_region(*observations_halfplanes(obs2))
106     equivariance_error=max(equivariance_error,abs(diameter(moved.vertices)[0]-diam
    ↪ eter(r.vertices)[0]))
107     check('LP_support_vs_vertex_enumeration',support_error<1e-6,support_error)
108     check('convex_optimization_vs_support_circle',circle_error<1e-5,circle_error)
109     check('true_target_containment',containment_error<1e-7,containment_error)
110     check('rotation_translation_invariance',equivariance_error<1e-6,equivariance_error)
111
112     # More observations cannot enlarge an intersection.
113     extra=dict(position=[800,400],bearing_deg=bearing([0,0],[800,400]))
114     aa,bb=observations_halfplanes(symmetrize+extra);more=summarize_region(halfplane_r
    ↪ egion(aa,bb))
115     check('information_monotonicity',more['diameter_m']<=sy['diameter_m']+1e-8 and
    ↪ more['cover_radius_m']<=sy['cover_radius_m']+1e-8)
116
117     report={'checks':checks,'passed':all(c['passed'] for c in
    ↪ checks),'random_cases':60,
118            'seed':seed,'support_max_error_m':support_error,'circle_max_error_m':circl
    ↪ e_error,
119            'equivariance_error_m':equivariance_error,'elapsed_s':time.perf_counter()-
    ↪ start}
120     (out/'report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2))
121     return report

```

11.3 问题二：第二检测点的数值优选

文件 q1-q2/run_q2.py

```

1  """ 问题二入口：默认 q 和正常反馈示向度均在题目原坐标系中输入。"""
2  import argparse,json,sys
3  from pathlib import Path
4  from q2.api import initial_geometry,evaluate,feedback_geometry
5  from q2.search import run,WEIGHTS
6
7
8  def main():
9      parser=argparse.ArgumentParser(description=__doc__)
10     parser.add_argument('mode',choices=['solve','info','evaluate','geometry'])
11     parser.add_argument('--a',type=float,default=0,help='首次检测点 (a,0), 单位 m')
12     parser.add_argument('--beta',type=float,default=0,help='首次示向度, 逆时针角度制')
13     parser.add_argument('--weights',nargs='+',type=float,default=WEIGHTS,help='solve
    ↪ 使用的权重列表')
14     parser.add_argument('--w',type=float,default=.5,help='evaluate 使用的单个权重')
15     parser.add_argument('--q',nargs=2,type=float,metavar=('X','Y'))
16     parser.add_argument('--frame',choices=['global','internal'],default='global')
17     parser.add_argument('--level',type=int,choices=range(7),default=6)
18     parser.add_argument('--scan',type=int,default=128)
19     parser.add_argument('--kind',choices=['H','N','normal'],default='normal')
20     parser.add_argument('--theta',type=float,default=0,help='第二次正常示向度, 角度制')
21     parser.add_argument('--out',type=Path,help='solve: 新建输出文件夹; 其余模式: JSON
    ↪ 文件路径')
22     args=parser.parse_args()
23     try:
24         if args.mode=='solve':
25             if args.out is None:parser.error('solve 需要 --out 指定一个尚不存在的文件夹')
26             result=run(args.a,args.beta,args.out,args.weights)
27             print('结果已保存到',args.out.resolve());return
28         if args.mode=='info':result=initial_geometry(args.a,args.beta)
29         else:
30             if args.q is None:parser.error('evaluate / geometry 需要 --q X Y')
31             if args.mode=='evaluate':result=evaluate(args.a,args.beta,args.q,args.w,ar
    ↪ gs.level,args.scan,args.frame)
32             else:result=feedback_geometry(args.a,args.beta,args.q,args.kind,args.theta
    ↪ ,args.frame)
33     text=json.dumps(result,ensure_ascii=False,indent=2,allow_nan=False)
34     if args.out:
35         args.out.parent.mkdir(parents=True,exist_ok=True)

```

```

36         with args.out.open('x') as f:f.write(text+'\n')
37         print(text)
38     except (ValueError,RuntimeError,FileExistsError) as e:parser.exit(2,f'错误: {e}\n')
39
40 if __name__=='__main__':main()

```

文件 q1-q2/build.py

```

1  """Build the portable C++17 numerical kernel using CXX, clang++ or g++."""
2  from pathlib import Path
3  import os,shlex,shutil,subprocess
4  ROOT=Path(__file__).resolve().parent
5
6  def build(force=False):
7      binary=ROOT/'build'/('q2_solver.exe' if os.name=='nt' else 'q2_solver')
8      sources=[ROOT/'q2/solver.cpp',ROOT/'q2/geometry.hpp']
9      if not force and binary.exists() and binary.stat().st_mtime>=max(p.stat().st_mtime
10         ↪ for p in sources):
11         return binary
12      compiler=shlex.split(os.environ.get('CXX',''))
13      if not compiler:
14         found=shutil.which('clang++') or shutil.which('g++')
15         if not found:raise RuntimeError('需要支持 C++17 的 clang++ 或 g++, 也可通过 CXX
16         ↪ 指定编译器。')
17         compiler=[found]
18      binary.parent.mkdir(exist_ok=True)
19      temporary=binary.with_suffix('.tmp.exe' if os.name=='nt' else '.tmp')
20      subprocess.run(compiler+['-std=c++17','-03',str(sources[0]),'-o',str(temporary)],c
21         ↪ heck=True)
22      temporary.replace(binary)
23      return binary
24
25 if __name__=='__main__':print(build(force=True))

```

文件 q1-q2/q2/__init__.py

本文件与 q1-q2/q1/__init__.py 内容完全相同, 使用前述代码。

文件 q1-q2/q2/api.py

```

1  """Python API: input/output lengths in m, angles in degrees unless stated otherwise."""
2  from pathlib import Path
3  import csv,json,math,subprocess
4  from build import build
5
6
7  def binary():
8      return build()
9
10
11 def validate_scenario(a,beta):
12     if not math.isfinite(a) or a<0 or not math.isfinite(beta):
13         raise ValueError('a 必须为有限非负数, beta 必须为有限角度。')
14
15
16 def native(a,beta,*args):
17     validate_scenario(a,beta)
18     run=subprocess.run([str(binary()),str(a),str(beta%360),*map(str,args)],text=True,c
19         ↪ apture_output=True)
20     if run.returncode:raise ValueError(run.stderr.strip() or '数值内核运行失败')
21     return run.stdout
22
23
24 def to_global(q,a,beta):
25     b=math.radians(beta);x,y=q
26     return [a+math.cos(b)*x-math.sin(b)*y,math.sin(b)*x+math.cos(b)*y]
27
28
29 def to_internal(q,a,beta):
30     b=math.radians(beta);x,y=q[0]-a,q[1]

```

```

30     return [math.cos(b)*x+math.sin(b)*y,-math.sin(b)*x+math.cos(b)*y]
31
32
33 def initial_geometry(a,beta):
34     result=json.loads(native(a,beta,'info'))
35     result['center_global']=to_global(result['center'],a,beta)
36     result['points_global']=[to_global(p,a,beta) for p in result['points']]
37     return result
38
39
40 def evaluate(a,beta,q,weight=.5,level=6,scan=128,frame='global'):
41     if not math.isfinite(weight) or not 0<=weight<=1:raise ValueError('w 必须在 [0,1]
42     ↪ 内。')
43     if frame not in ('global','internal'):raise ValueError('frame must be global or
44     ↪ internal')
45     p=to_internal(q,a,beta) if frame=='global' else q
46     result={k:float(v) for k,v in
47     ↪ next(csv.DictReader(native(a,beta,'eval'),*p,level,scan).splitlines()).items()}
48     result['admissible']=result['distance_K1']<=1500+1e-7
49     result['w']=weight
50     # Save raw probabilities; only floating-point excursions are clipped in F.
51     result['F']=(1-weight)*result['J']/20+weight*(1-min(1,max(0,result['P_finish'])))
52     return result
53
54 def feedback_geometry(a,beta,q,kind,theta_deg=0,frame='global'):
55     if frame not in ('global','internal'):raise ValueError('frame must be global or
56     ↪ internal')
57     p=to_internal(q,a,beta) if frame=='global' else q
58     theta=math.radians(theta_deg-beta if frame=='global' else theta_deg)
59     kinds={'H':0,'normal':1,'N':2}
60     if kind not in kinds:raise ValueError('kind must be H, N or normal')
61     result=json.loads(native(a,beta,'geometry',*p,kinds[kind],theta))
62     result['center_global']=to_global(result['center'],a,beta) if result['nonempty']
63     ↪ else None
64     return result

```

文件 q1-q2/q2/geometry.hpp

```

1  #pragma once
2  // General a,beta extension. Internal coordinates: S1 origin, first bearing +x.
3  // Model source remains 0-centered; TARGET_CENTER preserves its disk exactly.
4  #include <algorithm>
5  #include <array>
6  #include <chrono>
7  #include <cmath>
8  #include <fstream>
9  #include <iomanip>
10 #include <iostream>
11 #include <map>
12 #include <random>
13 #include <sstream>
14 #include <string>
15 #include <vector>
16 using namespace std;
17 constexpr double PI=3.1415926535897932384626433832795, A=PI/180, RMAX=1500, RMIN=1000,
18     ↪ INNER=5;
19 struct P {double x=0,y=0; P operator+(P b)const{return{x+b.x,y+b.y};} P operator-(P
20     ↪ b)const{return{x-b.x,y-b.y};} P operator*(double s)const{return{x*s,y*s};};};
21 double dot(P a,P b){return a.x*b.x+a.y*b.y;} double cross(P a,P b){return
22     ↪ a.x*b.y-a.y*b.x;} double norm2(P a){return dot(a,a);} double norm(P a){return
23     ↪ sqrt(norm2(a));} P unit(double t){return{cos(t),sin(t)};} double angle(P a){return
24     ↪ atan2(a.y,a.x);} double wrap(double t){return remainder(t,2*PI);} double pos(double
25     ↪ t){t=fmod(t,2*PI);return t<0?t+2*PI:t;}
26 // Uniform prior R in [1000,1500]. One shared radius for both observations.
27 double tail(double d){return 1-clamp((d-1000)/500,0.0,1.0);}
28 struct Prim {bool circle;P p;double r;int sign;}; // circle sign=+1 inside, -1 outside;
29     ↪ line p dot g<=r
30 struct Arc {P o;double r,l,h;};
31 struct Shape {vector<P> points;vector<Arc> arcs;};
32 using Region=vector<Prim>;
33 void line(Region&v,P n,double b){double
34     ↪ d=norm(n);if(d>1e-14)v.push_back({false,n*(1/d),b/d,1});}

```

```

27 void circle(Region&v,P o,double r,int sign=1){v.push_back({true,o,r,sign});}
28 void wedge(Region&v,P o,double t){P lo=unit(t-A),hi=unit(t+A);P
  ↳ n1={lo.y,-lo.x},n2={-hi.y,hi.x};line(v,n1,dot(n1,o));line(v,n2,dot(n2,o));}
29 double SCENARIO_A=0,SCENARIO_BETA=0,ZC=0; P TARGET_CENTER;
30 Region base(){Region v;circle(v,TARGET_CENTER,1800);circle(v,{0,0},1500);circle(v,{0,0}
  ↳ },5,-1);wedge(v,{0,0},0);return v;}
31 bool feasible(const Region&v,P g,double tol=2e-7){for(auto s:v){double
  ↳ f=s.circle?s.sign*(norm(g-s.p)-s.r):dot(s.p,g)-s.r;if(f>tol)return false;}return
  ↳ true;}
32 vector<P> intersect(Prim a,Prim b){vector<P>v;if(!a.circle&&!b.circle){double
  ↳ d=cross(a.p,b.p);if(abs(d)>1e-12)v.push_back({(a.r*b.p.y-a.p.y*b.r)/d,(a.p.x*b.r-a
  ↳ .r*b.p.x)/d});return v;}
33 if(!a.circle)swap(a,b);
34 if(!b.circle){double signed_d=b.r-dot(b.p,a.p);double
  ↳ z=a.r*a.r-signed_d*signed_d;if(z>=-1e-6){P c=a.p+b.p*signed_d,
  ↳ u={-b.p.y,b.p.x};double
  ↳ t=sqrt(max(0.0,z));v.push_back(c+u*t);if(t>1e-9)v.push_back(c-u*t);}return v;}
35 double d=norm(b.p-a.p);if(d<1e-10||d>a.r+b.r+1e-7||d<abs(a.r-b.r)-1e-7)return v;P
  ↳ u=(b.p-a.p)*(1/d);double t=(a.r*a.r-b.r*b.r+d*d)/(2*d),z=a.r*a.r-t*t;if(z<
  ↳ -1e-5)return v;P c=a.p+u*t,n={-u.y,u.x};double
  ↳ h=sqrt(max(0.0,z));v.push_back(c+n*h);if(h>1e-9)v.push_back(c-n*h);return v;}
36 void uniquev(vector<double>&v,double
  ↳ tol=1e-10){sort(v.begin(),v.end());v.erase(unique(v.begin(),v.end(), [&](double
  ↳ x,double y){return abs(x-y)<tol;}),v.end());}
37 Shape boundary(const Region&v){Shape s;for(size_t i=0;i<v.size();i++){auto
  ↳ p=v[i];vector<double>ts;P d={-p.p.y,p.p.x};for(size_t
  ↳ j=0;j<v.size();j++)if(i!=j)for(auto
  ↳ x:intersect(p,v[j]))ts.push_back(p.circle?pos(angle(x-p.p)):dot(d,x));uniquev(ts);
38 if(p.circle){if(ts.empty())ts={0};size_t n=ts.size();for(size_t k=0;k<n;k++){double
  ↳ l=ts[k],h=k+1<n?ts[k+1]:ts[0]+2*PI;if(h-l<1e-12)continue;P
  ↳ mid=p.p+unit((l+h)/2)*p.r;if(feasible(v,mid)){s.arcs.push_back({p.p,p.r,l,h});s.p
  ↳ oints.push_back(p.p+unit(l)*p.r);s.points.push_back(p.p+unit(h)*p.r);}}
39 else for(size_t k=0;k+1<ts.size();k++){P x=p.p+p.r*d*ts[k],y=p.p+p.r*d*ts[k+1];if(fea
  ↳ sible(v,(x+y)*.5)){s.points.push_back(x);s.points.push_back(y);}}
40 } vector<P>pts;for(auto x:s.points){bool exists=false;for(auto
  ↳ y:pts)if(norm(x-y)<1e-6){exists=true;break;}if(!exists&&feasible(v,x,1e-5))pts.pu
  ↳ sh_back(x);}s.points=pts;return s;}
41 struct Disk {P c;double r=-1;};
42 bool covers(Disk d,P p){return d.r>=0&&norm(p-d.c)<=d.r+2e-8;}
43 Disk two(P a,P b){return{(a+b)*.5,norm(a-b)*.5};}
44 Disk three(P a,P b,P c){Disk best;best.r=1e99;for(auto d:{two(a,b),two(a,c),two(b,c)})
  ↳ if(covers(d,a)&&covers(d,b)&&covers(d,c)&&d.r<best.r)best=d;
45 if(best.r<1e98)return best;P u=b-a,v=c-a;double
  ↳ den=2*cross(u,v);if(abs(den)<1e-14)return best;P z={(norm2(u)*v.y-u.y*norm2(v))/d
  ↳ en,(u.x*norm2(v)-norm2(u)*v.x)/den};return{a+z,norm(z)};}
46 Disk finite_mec(vector<P>ps){if(ps.empty())return{{0,0},0};std::mt19937
  ↳ gen(1729);shuffle(ps.begin(),ps.end(),gen);Disk d;for(size_t
  ↳ i=0;i<ps.size();i++)if(!covers(d,ps[i])){d={ps[i],0};for(size_t
  ↳ j=0;j<i;j++)if(!covers(d,ps[j])){d=two(ps[i],ps[j]);for(size_t
  ↳ k=0;k<j;k++)if(!covers(d,ps[k]))d=three(ps[i],ps[j],ps[k]);}}return d;}
47 P farthest(const Shape&s,P c){P best=s.points.empty()?P{}:s.points[0];double
  ↳ b=norm2(best-c);auto check=[&](P p){double
  ↳ v=norm2(p-c);if(v>b){b=v;best=p;}};for(auto p:s.points)check(p);for(auto
  ↳ a:s.arcs){double t=pos(angle(a.o-c));for(int k=-1;k<=2;k++){double
  ↳ v=t+2*PI*k;if(v>=a.l-1e-12&&v<=a.h+1e-12)check(a.o+unit(v)*a.r);}}return best;}
48 struct MEC {P c;double lo=0,hi=0;int it=0;};
49 MEC continuous_mec(const Shape&s,double
  ↳ eps=1e-7){if(s.points.empty())return{};vector<P>v=s.points;MEC out;for(int
  ↳ it=0;it<80;it++){Disk d=finite_mec(v);P p=farthest(s,d.c);double
  ↳ h=norm(p-d.c);out={d.c,d.r,max(h,d.r),it};if(out.hi-out.lo<=eps)return
  ↳ out;v.push_back(p);}return out;}
50 // Gauss-Legendre nodes on [-1,1], generated independently of integration intervals.
51 map<int,vector<pair<double,double>>> gcache;
52 vector<pair<double,double>> gauss(int n){if(gcache.count(n))return
  ↳ gcache[n];vector<pair<double,double>>v;for(int i=0;i<n;i++){double
  ↳ z=cos(PI*(i+.75)/(n+.5)),dp=0;for(int j=0;j<30;j++){double p0=1,p1=z;for(int
  ↳ k=2;k<=n;k++){double
  ↳ p=((2*k-1)*z*p1-(k-1)*p0)/k;p0=p1;p1=p;}dp=n*(z*p1-p0)/(z*z-1);double
  ↳ dz=p1/dp;z-=dz;if(abs(dz)<1e-15)break;}v.push_back({z,2/((1-z*z)*dp*dp)});}return
  ↳ gcache[n]=v;}

```

```

53 void addray(vector<double>&cuts,Prim p,P u){if(p.circle){double
    ↪ b=dot(u,p.p),disc=b*b+p.r*p.r-norm2(p.p);if(disc>=0){double h=sqrt(disc);for(double
    ↪ r:{b-h,b+h})if(r>5&&r<1500)cuts.push_back(r);}}else{double
    ↪ den=dot(p.p,u);if(abs(den)>1e-14){double
    ↪ r=p.r/den;if(r>5&&r<1500)cuts.push_back(r);}}
54 vector<double> spatial_breaks(const Region&v,const Shape&s,P
    ↪ q){vector<double>cuts={-A,A};auto add=[&](P p){double
    ↪ t=angle(p);if(t>-A&&t<A)cuts.push_back(t);};for(auto p:s.points)add(p);
55 vector<Prim>all=v;circle(all,q,1000); // kink in the integrated weight
56 line(all,q*2,norm2(q));circle(all,{0,0},1000);for(size_t
    ↪ i=0;i<all.size();i++)for(size_t j=i+1;j<all.size();j++)for(auto
    ↪ p:intersect(all[i],all[j]))add(p);
57 for(auto p:all)if(p.circle){double d=norm(p.p);if(d>p.r+1e-10){double
    ↪ t=angle(p.p),a=asin(p.r/d);for(double
    ↪ f:{wrap(t-a),wrap(t+a)})if(f>-A&&f<A)cuts.push_back(f);}}
58 uniquev(cuts);return cuts;}
59 double mass(const Region&v,const Shape&s,P q,int kind,int na,int
    ↪ nr){if(s.points.empty())return 0;auto ts=spatial_breaks(v,s,q);auto
    ↪ gn=gauss(na),gr=gauss(nr);double sum=0;for(size_t ti=0;ti+1<ts.size();ti++){double
    ↪ mid=(ts[ti]+ts[ti+1])/2,half=(ts[ti+1]-ts[ti])/2;for(auto [z,w]:gn){P
    ↪ u=unit(mid+half*z);vector<double>rs={5,1000,1500};for(auto
    ↪ p:v)addray(rs,p,u);addray(rs,{true,q,1000,1},u);double
    ↪ den=2*dot(q,u);if(abs(den)>1e-12){double
    ↪ r=norm2(q)/den;if(r>5&&r<1500)rs.push_back(r);}uniquev(rs,1e-8);double
    ↪ radial=0;for(size_t j=0;j+1<rs.size();j++){double rm=(rs[j]+rs[j+1])/2,rh=(rs[j+1]
    ↪ -rs[j])/2;if(!feasible(v,u*rm,1e-8))continue;for(auto [zz,ww]:gr){double
    ↪ r=rm+rh*zz,d=norm(u*r-q),val=0;if(kind==0)val=tail(r);if(kind==1)val=tail(max(r,d)
    ↪ );if(kind==2)val=tail(r)-tail(max(r,d));radial+=ww*rh*r*val;}}sum+=w*half*radial;}}
60 return max(0.0,sum);}
61 Region feedback(P q,int kind,double theta=0){Region
    ↪ v=base();if(kind==0)circle(v,q,5);else if(kind==1){circle(v,q,1500);circle(v,q,5,-
    ↪ 1);wedge(v,q,theta);}else{circle(v,q,1000,-1);line(v,q*2,norm2(q));}return v;}
62 vector<double> feedback_breaks(P q){Region
    ↪ v=base();circle(v,q,1500);circle(v,q,5,-1);Shape
    ↪ s=boundary(v);vector<double>ts={0,2*PI};vector<double>phis;for(auto
    ↪ p:s.points)phis.push_back(pos(angle(p-q)));for(auto ar:s.arcs){double
    ↪ d=norm(ar.o-q);if(d>ar.r+1e-10){double t=angle(q-ar.o),z=acos(ar.r/d);for(double
    ↪ b:{pos(t-z),pos(t+z)})for(int k=0;k<2;k++){double p=b+2*PI*k;if(p>=ar.l-1e-10&&p<=
    ↪ ar.h+1e-10)phis.push_back(pos(angle(ar.o+unit(p)*ar.r-q))};}else
    ↪ if(d<1e-9&&ar.h-ar.l>2*PI-1e-6)for(int j=0;j<8;j++)phis.push_back(j*PI/4);}
63 Region beyond=base();circle(beyond,q,1500);circle(beyond,q,20,-1);Shape
    ↪ bs=boundary(beyond);
64 for(auto p:bs.points)phis.push_back(pos(angle(p-q)));
65 for(auto ar:bs.arcs){double d=norm(ar.o-q);if(d>ar.r+1e-10){double
    ↪ t=angle(q-ar.o),z=acos(ar.r/d);for(double b:{pos(t-z),pos(t+z)})for(int
    ↪ k=-1;k<=2;k++){double p=b+2*PI*k;if(p>=ar.l-1e-10&&p<=ar.h+1e-10)phis.push_back(p
    ↪ );}os(angle(ar.o+unit(p)*ar.r-q));}}
66 for(double
    ↪ t:phis){ts.push_back(pos(t-A));ts.push_back(pos(t+A));}uniquev(ts,1e-11);return
    ↪ ts;}
67
68 struct Settings {int ns=4,nr=6,nt=4,depth=2;double rel=2e-4;};
69
70 double normalizer(){auto v=base();auto s=boundary(v);return mass(v,s,{0,0},0,64,16);}
71
72 Settings settings(int
    ↪ level){if(level>=6)return{28,36,14,11,2e-13};if(level==0)return{2,4,2,0,.001};if(le
    ↪ vel==1)return{4,6,3,1,.001};if(level==2)return{6,10,4,3,2e-5};if(level==3)return{1
    ↪ 0,14,6,5,2e-7};if(level==4)return{14,20,8,7,2e-9};return{20,28,10,9,2e-11};}

```

文件 q1-q2/q2/independent_geometry.py

```

1 """ 独立极坐标区间采样及有限支撑圆枚举，仅供几何核验。 """
2 import itertools,math
3 import numpy as np
4 A=math.pi/180
5
6 def independent_points(a,b,q,kind,t,n=8001):
7     phi=np.linspace(-A,A,n);u=np.c_[np.cos(phi),np.sin(phi)];q=np.array(q);B=math.radians
8     ↪ (b);center=np.array([-a*math.cos(B),a*math.sin(B)])
9     cuts=[np.full(n,5.),np.full(n,1500.)]

```

```

9 def circlecuts(c,r):
10     proj=u@c;disc=proj*proj+r*r-c@c;h=np.sqrt(np.maximum(0,disc));good=disc>=0
11     cuts.extend([np.where(good,proj-h,np.nan),np.where(good,proj+h,np.nan)])
12     circlecuts(center,1800)
13     for r in ([5] if kind==0 else [1000] if kind==2 else [5,1500]):circlecuts(q,r)
14     if kind==2:
15         v=u@q;cuts.append(np.divide(q@q,2*v,out=np.full(n,np.nan),where=abs(v)>1e-12))
16     if kind==1:
17         for ang in [t-A,t+A]:
18             v=np.array([math.cos(ang),math.sin(ang)]);den=u[:,0]*v[1]-u[:,1]*v[0];num=q[0]*v[1]-
19                 ↪ -q[1]*v[0]
20             cuts.append(np.divide(num,den,out=np.full(n,np.nan),where=abs(den)>1e-12))
21             rr=np.sort(np.stack(cuts,axis=1),axis=1);lo=rr[:,:-1];hi=rr[:,1:];mid=(lo+hi)/2
22             g=u[:,None,:]*mid[:,None];d=np.linalg.norm(g-q,axis=2)
23             ok=np.isfinite(mid)&(lo>=5-1e-8)&(hi<=1500+1e-8)&(hi-lo>1e-9)&(np.linalg.norm(g-cente
24                 ↪ r,axis=2)<=1800+1e-8)
25             if kind==0:ok&=d<=5+1e-8
26             elif kind==2:ok&=(d>=1000-1e-8)&(d>=mid-1e-8)
27             else:
28                 ang=np.arctan2(g[:,1]-q[1],g[:,0]-q[0]);delta=np.arctan2(np.sin(ang-t),np.cos(an
29                 ↪ g-t));ok&=(d>=5-1e-8)&(d<=1500+1e-8)&(abs(delta)<=A+1e-10)
30             return np.concatenate([(u[:,None,:]*lo[:,None])[ok],(u[:,None,:]*hi[:,None])[ok]])
31
32 def exact_finite_radius(v):
33     if len(v)<2:return 0.
34     v=v-v.mean(axis=0);n=len(v);pairs=np.array(list(itertools.combinations(range(n),2)))
35     cc=(v[pairs[:,0]]+v[pairs[:,1]])*.5;rr=np.sum((v[pairs[:,0]]-cc)**2,axis=1)
36     if n>=3:
37         tri=np.array(list(itertools.combinations(range(n),3)));a=v[tri[:,0]];b=v[tri[:,1]]-a
38         ↪ ;c=v[tri[:,2]]-a
39         det=2*(b[:,0]*c[:,1]-b[:,1]*c[:,0]);ok=abs(det)>1e-12;a=a[ok];b=b[ok];c=c[ok];det=de
40         ↪ t[ok]
41         b2=(b*b).sum(1);c2=(c*c).sum(1);z=np.c_[(b2*c[:,1]-b[:,1]*c2)/det,(b[:,0]*c2-b2*c[:,
42         ↪ 0])/det]
43         cc=np.r_[cc,a+z];rr=np.r_[rr,(z*z).sum(1)]
44     for i in np.argsort(rr):
45         if np.max(((cc[i]-v)**2).sum(1)-rr[i])<=1e-8:return math.sqrt(float(rr[i]))
46     raise RuntimeError('no enclosing circle')

```

文件 q1-q2/q2/nelder_mead.py

```

1 """Two-dimensional Nelder-Mead with explicit convergence diagnostics."""
2 import numpy as np
3
4 def minimize(fun,x0,step=20,xatol=.05,fatol=1e-6,maxiter=140):
5     x=np.array(x0,dtype=float);s=np.array([x,x+[step,0],x+[0,step]]);v=np.array([fun(x)
6     ↪ for x in s]);success=False
7     for it in range(maxiter):
8         ids=np.argsort(v);s=s[ids];v=v[ids]
9         if np.max(np.abs(s[1:]-s[0]))<=xatol and np.ptp(v)<=fatol:success=True;break
10         c=s[1:2].mean(0);xr=2*c-s[2];fr=fun(xr)
11         if fr<v[0]:
12             xe=c+2*(xr-c);fe=fun(xe);s[2],v[2]=(xe,fe) if fe<fr else (xr,fr)
13         elif fr<v[1]:s[2],v[2]=(xr,fr)
14         else:
15             outside=fr<v[2];xc=c+.5*((xr if outside else s[2])-c);fc=fun(xc)
16             if fc<(fr if outside else v[2]):s[2],v[2]=(xc,fc)
17             else:s[1:]=s[0]+.5*(s[1:]-s[0]);v[1:]=[fun(z) for z in s[1:]]
18     j=int(np.argmin(v));return
19     ↪ s[j],{'success':success,'iterations':it+1,'simplex_diameter_m':float(np.max(np.li
20     ↪ nalg.norm(s-s[j],axis=1))), 'simplex_value_spread':float(np.ptp(v))}

```

文件 q1-q2/q2/search.py

```

1  """ 覆盖网格、多起点 Nelder-Mead、精度递增与按权重输出候选位置。"""
2  from pathlib import Path
3  import csv,json,math,subprocess,time
4  import numpy as np
5
6  ROOT=Path(__file__).resolve().parents[1]
7  from .nelder_mead import minimize
8  from .api import binary, initial_geometry, to_global
9  WEIGHTS=[0,.05,.1,.2,.35,.5,.65,.8,.9,.95,1]
10
11 class Evaluator:
12     def __init__(self,a,b,out):
13         self.a=a;self.b=b;self.out=out;self.cache={};self.rows=[];self.start=time.monotonic()
14         self.p=subprocess.Popen([str(binary()),str(a),str(b),'stream'],stdin=subprocess.PIPE,
15             ↪ stdout=subprocess.PIPE,text=True,bufsize=1)
16         self.fields=self.p.stdout.readline().strip().split(',');self.file=(out/'evaluations.'
17             ↪ csv').open('w');self.writer=csv.DictWriter(self.file,fieldnames=self.fields+['level',
18             ↪ 'scan']);self.writer.writeheader()
19     def ev(self,q,level=3,scan=12):
20         q=np.asarray(q,dtype=float);x,y=q
21         if self.a==0 or self.b in (0,180):y=abs(y)
22         if math.hypot(x,y)<1e-8:return None
23         B=math.radians(self.b);gx=self.a+math.cos(B)*x-math.sin(B)*y;gy=math.sin(B)*x+math.c
24             ↪ os(B)*y
25         if math.hypot(gx,gy)>3300+1e-6:return None
26         key=(round(float(x),10),round(float(y),10),level,scan)
27         if key not in self.cache:
28             self.p.stdin.write(f'{x:.16g} {y:.16g} {level}
29             ↪ {scan}\n');self.p.stdin.flush();line=self.p.stdout.readline()
30             if not line:raise RuntimeError('evaluator closed unexpectedly')
31             r=dict(zip(self.fields,map(float,line.strip().split(','))))r.update(level=level,sc
32             ↪ an=scan)
33             if not all(math.isfinite(v) for v in r.values()):raise RuntimeError(r)
34             self.cache[key]=r;self.rows.append(r);self.writer.writerow(r)
35             if len(self.rows)%100==0:self.file.flush()
36             r=self.cache[key]
37             return r if r['distance_K1']<=1500+1e-7 else None
38     def score(self,q,w,level=3,scan=12):
39         r=self.ev(q,level,scan)
40         return score(r,w) if r else 1e10
41     def __enter__(self):return self
42     def __exit__(self,*args):self.close()
43     def close(self):
44         self.file.close()
45         try:
46             self.p.stdin.close();self.p.wait(timeout=5)
47         except (BrokenPipeError,subprocess.TimeoutExpired):
48             self.p.kill();self.p.wait()
49
50 def score(r,w):
51     # Only roundoff outside [0,1] is clipped for optimization; raw probability is saved.
52     p=min(1.,max(0.,r['P_finish']));return (1-w)*r['J']/20+w*(1-p)
53
54 def best_spread(rows,w,n,distance):
55     selected=[]
56     for r in sorted(rows,key=lambda r:(score(r,w),r['J'])):
57         q=np.array([r['x'],r['y']])
58         if all(np.linalg.norm(q-np.array([z['x'],z['y']]))>distance for z in
59             ↪ selected):selected.append(r)
60         if len(selected)>=n:break
61     return selected
62
63 def run(a,b,out,weights=None):
64     b=float(b)%360
65     weights=WEIGHTS if weights is None else list(weights)
66     if not weights or any(not math.isfinite(w) or not 0<=w<=1 for w in weights):raise
67         ↪ ValueError("weights must be in [0,1]")
68     start=time.monotonic();name=f'a{a:g}_b{b:g}';info=initial_geometry(a,b);out=Path(out)
69         ↪ ;out.mkdir(parents=True,exist_ok=False)
70     (out/'initial_geometry.json').write_text(json.dumps(info,indent=2))

```

```

63 qbase=info['center'];points=np.array(info['points']);span=float(np.ptp(points[:,0]));
    ↪ optima=[]
64 if info['rho1']<=20:
65     summary={'case':name,'a':a,'beta_deg':b,'status':'no_second_detection_needed','rho1'
    ↪ :info['rho1'],'clear_center_global':info['center_global'],'weights':weights,'can'
    ↪ didates':[]}]
66     (out/'summary.json').write_text(json.dumps(summary,indent=2));print(json.dumps(summa
    ↪ ry),flush=True);return summary
67 else:
68     with Evaluator(a,b,out) as ev:
69         basevalue=ev.ev(qbase,6,128)
70         lo=points.min(0)-1501;hi=points.max(0)+1501
71         gx=np.arange(math.floor(lo[0]/100)*100,hi[0]+100,100);gy=np.arange(math.floor(lo[1]
    ↪ /100)*100,hi[1]+100,100)
72         if a==0 or b in (0,180):gy=gy[gy>=0]
73         grid={(float(x),float(y)) for x in gx for y in gy}
74         xmin=max(5.,points[:,0].min()-1);xmax=min(1500.,points[:,0].max()+1)
75         step=max(2.,min(35.,span/20))
76         yy=np.arange(-.8*span,.8*span+step,step)
77         if a==0 or b in (0,180):yy=yy[yy>=0]
78         grid|={(float(x),float(y)) for x in np.arange(xmin-.25*span,xmax+.25*span+step,step)
    ↪ for y in yy}
79         grid|={(float(x),float(y)) for x in np.linspace(xmin,xmax,65) for y in
    ↪ [-30,-10,0,10,30]}
80         for i,q in enumerate(sorted(grid)):
81             ev.ev(q,2,6)
82             if i%1000==0:print(name,'grid',i,'/',len(grid),'elapsed',round(time.monotonic()-st
    ↪ art,1),flush=True)
83         pool=[r for r in ev.rows if r['distance_K1']<=1500+1e-7]
84         scan_weights=weights
85         starts=[]
86         for w in scan_weights:
87             seeds=best_spread(pool,w,5,max(4.,span*.06))
88             if basevalue is not None:seeds.append(basevalue)
89             for r in seeds:
90                 q=[r['x'],r['y']]
91                 z,diag=minimize(lambda q:ev.score(q,w),q,step=min(25.,max(1.,span*.025)),xatol=.0
    ↪ 7,fatol=2e-7,maxiter=150)
92                 value=ev.ev(z)
93                 if value is not
    ↪ None:starts.append(dict(value,w=w,optimizer=diag));pool.append(value)
94         print(name,'weight',w,'coarse best',min(score(r,w) for r in
    ↪ pool),'elapsed',round(time.monotonic()-start,1),flush=True)
95         (out/'multistart.json').write_text(json.dumps(starts,indent=2))
96         for w in scan_weights:
97             finer=[]
98             for r in best_spread(pool,w,2,max(1.,span*.002)):
99                 z,diag=minimize(lambda q:ev.score(q,w,4,24),[r['x'],r['y']],step=min(3.,max(.2,sp
    ↪ an*.003)),xatol=.015,fatol=2e-9,maxiter=130)
100                 finer.append(dict(ev.ev(z,6,128),w=w,optimizer=diag))
101             best=min(finer,key=lambda
    ↪ r:(score(r,w),r['J']));optima.append(dict(best,F=score(best,w)))
102             print(name,'refined',w,'q',round(best['x'],3),round(best['y'],3),'J',round(best['J'
    ↪ ],6),'P',round(best['P_finish'],6),flush=True)
103         # Choose the best among all high-precision candidates at each weight, including the
    ↪ initial enclosing-circle center.
104         eligible=optima+([basevalue] if basevalue is not None else [])
105         optima=[dict(min(eligible,key=lambda r:(score(r,w),r['J'])),w=w,F=min(score(r,w) for
    ↪ r in eligible)) for w in weights]
106         # High precision checks around the final points at scales independent of the final
    ↪ simplex.
107         neighbors=[]
108         for w in scan_weights:
109             r=next(r for r in optima if r['w']==w)
110             for h in [.1,1.,5.]:
111                 for angle in np.arange(8)*math.pi/4:
112                     q=np.array([r['x'],r['y']])+h*np.array([math.cos(angle),math.sin(angle)])
113                     z=ev.ev(q,4,24)
114                     if z:neighbors.append(dict(z,w=w,step=h,F=score(z,w),improvement=score(r,w)-scor
    ↪ e(z,w)))
115         (out/'neighbors.json').write_text(json.dumps(neighbors,indent=2))

```

```

116 summary={'case':name,'a':a,'beta_deg':b,'initial_center_evaluation':basevalue,'weight'
↪ s':weights,'candidates':optima,'evaluation_count':len(ev.rows),'elapsed_seconds':
↪ time.monotonic()-start,'global_optimality_certified':False,'mirror_equivalence':a
↪ ==0 or b in (0,180),'note':'Candidates are numerical optima from covering search
↪ and multistart refinement; continuous global optimality is not certified.'}
117 summary['diagnostics']={'max_candidate_probability_residual':max(abs(r['probability_r'
↪ esidual']) for r in optima),
118                             'max_candidate_mec_gap_m':max(r['mec_gap'] for r in optima),
119                             'candidate_depth_capped_count':sum(r['depth_capped'] for r in
↪ optima),
120                             'max_neighbor_score_improvement':max([0.]+[r['improvement']
↪ for r in neighbors])}
121 candidate_points=[]
122 for r in optima:
123     candidate_points.append({'w':r['w'],'x_global':r['x_global'],'y_global':r['y_global']
↪ },{'J':r['J'],'P_finish':r['P_finish'],'F':r['F'],'symmetry_copy':False})
124 if summary['mirror_equivalence'] and abs(r['y'])>1e-7:
125     x,y=to_global([r['x'],-r['y']],a,b)
126     candidate_points.append({'w':r['w'],'x_global':x,'y_global':y,'J':r['J'],'P_finish'
↪ :r['P_finish'],'F':r['F'],'symmetry_copy':True})
127 with (out/'candidate_points.csv').open('w') as f:
128     writer=csv.DictWriter(f,fieldnames=list(candidate_points[0]));writer.writeheader();w
↪ riter.writerows(candidate_points)
129 (out/'summary.json').write_text(json.dumps(summary,indent=2))
130 with (out/'candidates.csv').open('w') as f:
131     fields=['w','x_global','y_global','J','P_finish','P_onsite','F','probability_residua
↪ l','distance_K1'];writer=csv.DictWriter(f,fieldnames=fields,extrasaction='ignore'
↪ );writer.writeheader();writer.writerows(optima)
132 print('DONE',name,summary['evaluation_count'],'evaluations',round(summary['elapsed_se
↪ conds'],1),'seconds',flush=True)
133 return summary
134

```

文件 q1-q2/q2/solver.cpp

```

1  #include "geometry.hpp"
2  struct RadiusValue {
3      double p=0,j=0,finish=0,onsite=0;
4      RadiusValue operator+(RadiusValue
↪ b)const{return{p+b.p,j+b.j,finish+b.finish,onsite+b.onsite};}
5      RadiusValue operator*(double k)const{return{p*k,j*k,finish*k,onsite*k};}
6  };
7  struct RadiusEvaluator {
8      P q;Settings st;int scan=8,calls=0,capped=0,roots_count=0;double gap=0;
9      double threshold(double theta){auto s=boundary(feedback(q,1,theta));return
↪ s.points.empty()?1e6:continuous_mec(s).hi-20.;}
10     vector<double> breaks(){
11         auto t=feedback_breaks(q);vector<double> extra;
12         for(size_t i=0;i<t.size();i++){
13             double l=t[i],h=t[i+1];if(h-l<1e-11)continue;
14             double edge=min((h-l)*1e-7,1e-9),prev=l+edge,fp=threshold(prev);
15             for(int k=1;k<=scan;k++){
16                 double now=k==scan?h-edge:l+(h-l)*k/scan,fn=threshold(now);
17                 if(fp*fn<0){double a=prev,b=now,fa=fp;
18                     for(int z=0;z<40;z++){double
↪ m=(a+b)/2,fm=threshold(m);if(fa*fm<=0)b=m;else{a=m;fa=fm;}}
19                     extra.push_back((a+b)/2);
20                 }
21                 prev=now;fp=fn;
22             }
23         }
24         roots_count=extra.size();t.insert(t.end(),extra.begin(),extra.end());uniquev(t,1e-12
↪ );return t;
25     }
26     RadiusValue at(double theta){
27         calls++;auto v=feedback(q,1,theta);auto s=boundary(v);
28         double m=mass(v,s,q,1,st.ns,st.nr)/(2*A*ZC);if(m<=0)return{};
29         auto c=continuous_mec(s);gap=max(gap,c.hi-c.lo);
30         bool optical=norm(farthest(s,q)-q)<=20.;
31         return{m,m*c.hi,m*(c.hi<=20.),m*optical};
32     }

```

```

33 RadiusValue quad(double l,double h,int n){RadiusValue v;for(auto
↪ [z,w]:gauss(n))v=v+at((l+h)/2+(h-l)*z/2)*(w*(h-l)/2);return v;}
34 RadiusValue adaptive(double l,double h,int depth){
35 auto low=quad(l,h,st.nt),high=quad(l,h,st.nt*2);
36 bool ok=abs(low.j-high.j)<=st.rel*max(1.,abs(high.j))&&abs(low.p-high.p)<=st.rel*1&
↪ &abs(low.finish-high.finish)<=st.rel*1&&abs(low.onsite-high.onsite)<=st.rel*1;
37 if(ok)return high;if(depth>=st.depth){capped++;return high;}
38 double m=(l+h)/2;return adaptive(l,m,depth+1)+adaptive(m,h,depth+1);
39 }
40 };
41 double distance_to_initial(P q){
42 auto v=base();if(feasible(v,q,1e-9))return 0.;auto s=boundary(v);double best=1e99;
43 for(P x:s.points)best=min(best,norm(q-x));
44 for(auto ar:s.arcs){double theta=pos(angle(q-ar.o));for(int k=-1;k<=2;k++)if(theta+2*
↪ PI*k>=ar.l&&theta+2*PI*k<=ar.h)best=min(best,abs(norm(q-ar.o)-ar.r));}
45 for(auto p:v)if(!p.circle){P
↪ proj=q-p.p*(dot(p.p,q)-p.r);if(feasible(v,proj,1e-7))best=min(best,norm(proj-q));}
46 return best;
47 }
48 void radius_header(){cout<<"x,y,J,P_finish,P_onsite,P_H,P_N,P_normal,P_finish_N,P_fini
↪ sh_normal,probability_residual,mec_gap,angle_calls,root_count,depth_capped,distanc
↪ e_K1,x_global,y_global\n";}
49 void radius_evaluate(P q,int level,int scan){
50 auto st=settings(level);RadiusEvaluator e{q,st,scan};RadiusValue total;double
↪ ph=0,pn=0,fn=0;
51 for(int kind:{0,2}){auto v=feedback(q,kind);auto s=boundary(v);double
↪ p=mass(v,s,q,kind,st.ns*2,st.nr*2)/ZC;if(kind==0)ph=p;else pn=p;
52 if(p<=0)continue;auto c=continuous_mec(s);e.gap=max(e.gap,c.hi-c.lo);bool
↪ optical=norm(farthest(s,q)-q)<=20.;
53 total=total+RadiusValue{p,p*c.hi,p*(c.hi<=20.),p*optical};if(kind==2)fn=p*(c.hi<=20.
↪ );}
54 }
55 auto t=e.breaks();RadiusValue normal;for(size_t i=0;i<t.size();i++)if(t[i+1]-t[i]>1
↪ e-12)normal=normal+e.adaptive(t[i],t[i+1],0);total=total+normal;
56 cout<<setprecision(15)<<q.x<<","<<q.y<<","<<total.j<<","<<total.finish<<","<<total.on
↪ site<<","<<ph<<","<<pn<<","<<normal.p<<","<<fn<<","<<normal.finish<<","<<total.p-
↪ l<<","<<e.gap<<","<<e.calls<<","<<e.roots_count<<","<<e.capped<<","<<distance_to_
↪ initial(q)<<","<<(SCENARIO_A*cos(SCENARIO_BETA)*q.x-sin(SCENARIO_BETA)*q.y)<<","<<
↪ (sin(SCENARIO_BETA)*q.x+cos(SCENARIO_BETA)*q.y)<<\n";
57 }
58 void radius_mc(P q,int n,int seed){
59 mt19937_64 gen(seed);uniform_real_distribution<double> u(0,1);double
↪ sumj=0,sumj2=0;int done=0,onsite=0,nh=0,nn=0,invalid=0;double maxres=-1e99;
60 auto shN=boundary(feedback(q,2)),shH=boundary(feedback(q,0));auto
↪ cn=continuous_mec(shN),ch=continuous_mec(shH);
61 for(int i=0;i<n;i++){
62 double r,phi,R;do{r=sqrt(25+(2250000-25)*u(gen));phi=(2*u(gen)-1)*A;R=1000+500*u(gen)
↪ );}while(r>R||norm(unit(phi)*r-TARGET_CENTER)>1800);
63 P g=unit(phi)*r;double d=norm(g-q);MEC c;Shape s;Region v;
64 if(d<=5){nh++;c=ch;s=shH;v=feedback(q,0);}else
↪ if(d>R){nn++;c=cn;s=shN;v=feedback(q,2);}else{double theta=angle(g-q)+(2*u(gen)-
↪ 1)*A;v=feedback(q,1,theta);s=boundary(v);c=continuous_mec(s);}
65 if(s.points.empty()||!feasible(v,g,1e-6)){invalid++;continue;}
66 bool optical=norm(farthest(s,q)-q)<=20.;double j=c.hi;sumj+=j;sumj2+=j*j;done+=c.hi<
↪ =20.;onsite+=optical;maxres=max(maxres,norm(g-c.c)-c.hi);
67 }
68 double mean=sumj/n,p=double(done)/n;
69 cout<<setprecision(15)<<{"\n":"<<n<<","\seed":"<<seed<<","\J_mean":"<<mean<<","\J
↪ _se":"<<sqrt(max(0.,sumj2/n-mean*mean)/(n-1))<<","\P_finish":"<<p<<","\P_finish
↪ _se":"<<sqrt(p*(1-p)/n)<<","\P_onsite":"<<double(onsite)/n<<","\P_H":"<<double
↪ (nh)/n<<","\P_N":"<<double(nn)/n<<","\invalid":"<<invalid<<","\max_truth_outsid
↪ e":"<<maxres<<}\n";
70 }
71
72 // Native interface. Public Python commands convert degrees and global coordinates.
73 int main(int argc,char**argv){
74 try {
75 if(argc<4)throw runtime_error("usage: q2_solver a beta_deg
↪ info|eval|stream|mc|geometry ...");
76 SCENARIO_A=stod(argv[1]);SCENARIO_BETA=stod(argv[2])*PI/180;
77 if(!isfinite(SCENARIO_A)||SCENARIO_A<0||!isfinite(SCENARIO_BETA))throw
↪ runtime_error("a must be finite and nonnegative; beta must be finite");
78 TARGET_CENTER={-SCENARIO_A*cos(SCENARIO_BETA),SCENARIO_A*sin(SCENARIO_BETA)};

```

```

79 ZC=normalizer();if(!(ZC>0))throw runtime_error("First normal-bearing observation has
    ↪ zero probability under the model");
80 string mode=argv[3];
81 if(mode=="info"){
82     auto sh=boundary(base());auto c=continuous_mec(sh);
83     cout<<setprecision(15)<<"{"C1\":"<<ZC<<","rho1\":"<<c.hi<<,"mec_gap\":"<<c.hi-c
    ↪ .lo<<,"center\":"<<c.c.x<<,"<<c.c.y<<","points\":"<<[";
84     for(size_t i=0;i<sh.points.size();i++)cout<<(i?","":"")<<["<<sh.points[i].x<<,"<<s
    ↪ h.points[i].y<<"]";
85     cout<<"}\n";return 0;
86 }
87 auto validate=[](P q,int level,int scan){
88     if(!isfinite(q.x)||!isfinite(q.y)||norm(q)<1e-8)throw runtime_error("Second detector
    ↪ must be finite and distinct from the first detector");
89     if(level<0||level>6||scan<1||scan>4096)throw runtime_error("level must be 0..6 and
    ↪ scan 1..4096");
90 };
91 if(mode=="stream"){
92     radius_header();cout.flush();double x,y;int level,scan;
93     while(cin>>x>>y>>level>>scan){validate({x,y},level,scan);radius_evaluate({x,y},leve
    ↪ l,scan);cout.flush();}return 0;
94 }
95 if(argc<6)throw runtime_error("Missing detector coordinates");
96 P q{stod(argv[4]),stod(argv[5])};validate(q,0,1);
97 if(mode=="eval"){
98     if(argc<7)throw runtime_error("Missing integration level");int
    ↪ level=stoi(argv[6]),scan=argc>7?stoi(argv[7]):128;
99     validate(q,level,scan);radius_header();radius_evaluate(q,level,scan);
100 }else if(mode=="mc"){
101     if(argc<8||stoi(argv[6])<2)throw runtime_error("mc requires sample count >= 2 and
    ↪ seed");radius_mc(q,stoi(argv[6]),stoi(argv[7]));
102 }else if(mode=="geometry"){
103     if(argc<8)throw runtime_error("geometry requires kind (0=H,1=normal,2=N) and
    ↪ internal theta radians");
104     int kind=stoi(argv[6]);double
    ↪ theta=stod(argv[7]);if(kind<0||kind>2||!isfinite(theta))throw
    ↪ runtime_error("Invalid feedback");
105     auto v=feedback(q,kind,theta);auto s=boundary(v);auto c=continuous_mec(s);
106     cout<<setprecision(15)<<"{"nonempty\":"<<(s.points.empty()?"false":"true")<<,"ra
    ↪ dius_lower\":"<<c.lo<<,"radius_upper\":"<<c.hi<<,"center\":"<<c.c.x<<,"<<
    ↪ c.c.y<<","dmax\":"<<(s.points.empty()?0:norm(farthest(s,q)-q)<<,"points\":"
    ↪ [";
107     for(size_t i=0;i<s.points.size();i++)cout<<(i?","":"")<<["<<s.points[i].x<<,"<<s.p
    ↪ oints[i].y<<"]";
108     cout<<"},"arcs\":"<<[";for(size_t i=0;i<s.arcs.size();i++){auto
    ↪ ar=s.arcs[i];cout<<(i?","":"")<<["<<ar.o.x<<,"<<ar.o.y<<,"<<ar.r<<,"<<ar.l<<
    ↪ ","<<ar.h<<"]";}cout<<"}\n";
109 }else throw runtime_error("Unknown mode");
110 }catch(const exception&e){cerr<<e.what()<<"\n";return 2;}
111 }

```

11.4 第一、二问的批量复算与核验

文件 q1-q2/reproduce_paper.py

```

1 """ 从观测输入重新生成论文两问的数值结果，不读取已保存候选点作为起点。"""
2 from pathlib import Path
3 import argparse, csv, json, platform, hashlib
4 from q1.solve import solve
5 from q2.search import run as solve_q2
6 ROOT=Path(__file__).resolve().parent
7
8
9 def main():
10     parser=argparse.ArgumentParser(description=__doc__)
11     parser.add_argument('--out', type=Path, required=True, help='新建结果目录')
12     parser.add_argument('--cases', type=Path, default=ROOT/'examples/q2_cases.json')
13     args=parser.parse_args();config=json.loads(args.cases.read_text())
14     args.out.mkdir(parents=True, exist_ok=False)
15     q1=args.out/'q1';q1.mkdir()
16     for source in sorted((ROOT/'examples').glob('q1_*.json')):
17         result=solve(json.loads(source.read_text()))

```

```

18         (q1/source.name).write_text(json.dumps(result,ensure_ascii=False,indent=2))
19     summaries=[];rows=[]
20     for case in config['cases']:
21         a,b=case['a'],case['beta_deg']
22         summary=solve_q2(a,b,args.out/'q2'/f'a{a:g}_b{b:g}',config['weights'])
23         summaries.append(summary)
24         rows.extend(dict(row,a_m=a,beta_deg=b) for row in summary['candidates'])
25         (args.out/'case_summaries.json').write_text(json.dumps(summaries,ensure_ascii=
        ↪ False,indent=2))
26     fields=['a_m','beta_deg','w','x_global','y_global','J','P_finish','F','probability'
        ↪ '_residual','mec_gap']
27     tables={'all_candidates.csv':rows,
28             'origin_weights.csv':[r for r in rows if r['a_m']==0 and r['beta_deg']==0],
29             'typical_cases.csv':[r for r in rows if r['w']==.5]}
30     for name,data in tables.items():
31         with (args.out/name).open('w') as f:
32             writer=csv.DictWriter(f,fieldnames=fields,extrasaction='ignore');writer.wr
        ↪ iteheader();writer.writerows(data)
33     manifest={'input_config':config,'python':platform.python_version(),
34              'source_sha256':{str(p.relative_to(R00T)):hashlib.sha256(p.read_bytes())
        ↪ .hexdigest() for p in sorted(R00T.rglob('*')) if p.suffix in
        ↪ ('.py','.cpp','.hpp') and '__pycache__' not in p.parts},
35              'no_second_detection_cases':[r['case'] for r in summaries if
        ↪ r.get('status')=='no_second_detection_needed'],
36              'note':'重新运行搜索的坐标可能有小幅数值差异; 参照表保存在 reference_results,
        ↪ 未用于搜索。'}
37     (args.out/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,inden
        ↪ t=2))
38     print('论文算例复算完成: ',args.out.resolve())
39
40 if __name__=='__main__':main()

```

文件 q1-q2/verify.py

```

1  """ 运行第一、二问的数值验证, 并保存可核查报告。"""
2  from pathlib import Path
3  import argparse, csv, hashlib, json, math, platform, time
4  import numpy as np
5  import scipy
6  from q1.validation import run as validate_q1
7  from q1.validate_projection import run as validate_projection
8  from q1.bearing_geometry import Region, summarize_region
9  from q2.api import evaluate, initial_geometry, feedback_geometry, native
10 from q2.independent_geometry import independent_points, exact_finite_radius
11 R00T=Path(__file__).resolve().parent
12
13
14 def run(out):
15     out=Path(out);out.mkdir(parents=True,exist_ok=False);start=time.monotonic()
16     report={'q1_geometry':validate_q1(out/'q1_geometry'),'q1_projection':validate_proj
        ↪ ection(out/'q1_projection')}
17     edge=summarize_region(Region('segment',np.array([[−20.,0],[20.,0]])))
18     assert edge['guaranteed_clear_possible'],'rho=20 must pass physical criterion'
19     report['q1_radius_20_boundary']=edge
20     convergence=[]
21     references=[]
22     for name in ['origin_weights.csv','typical_cases.csv']:
23         references.extend(list(csv.DictReader((R00T/'reference_results'/name).open()))
24     for r in references:
25         a,b=float(r['a_m']),float(r['beta_deg']);q=[float(r['x']),float(r['y'])]
26         lo=evaluate(a,b,q,level=5,scan=64,frame='internal')
27         hi=evaluate(a,b,q,frame='internal')
28         record={'a':a,'beta_deg':b,'q_internal':q,'w':float(r['w']),
29                'J_reference_difference_m':hi['J']−float(r['J']),
30                'P_reference_difference':hi['P_finish']−float(r['P_finish']),
31                'J_precision_difference_m':hi['J']−lo['J'],
32                'P_precision_difference':hi['P_finish']−lo['P_finish'],
33                'evaluation':hi}
34         assert abs(record['J_reference_difference_m'])<1e−5,record
35         assert abs(record['P_reference_difference'])<2e−6,record
36         assert abs(record['J_precision_difference_m'])<1e−5,record
37         assert abs(record['P_precision_difference'])<2e−6,record

```

```

38     assert abs(hi['probability_residual'])<1e-7,record
39     assert -1e-7<=hi['P_finish']<=1e-7,record
40     assert hi['P_finish']>=max(hi['P_onsite'],hi['P_H'])-1e-7,record
41     convergence.append(record)
42     report['q2_reference_and_convergence']=convergence
43     q=[813.824371479152,521.018510869701];north=evaluate(0,0,q);south=evaluate(0,0,[q[
44     ↪ 0],-q[1]])
45     assert abs(north['J']-south['J'])<1e-5 and
46     ↪ abs(north['P_finish']-south['P_finish'])<2e-6
47     report['q2_symmetry']={'J_difference_m':north['J']-south['J'],'P_difference':north[
48     ↪ 'P_finish']-south['P_finish']}
49     strong=evaluate(0,0,[700,0]);geo=feedback_geometry(0,0,[700,0],'H')
50     assert strong['P_H']>0 and geo['radius_upper']>0 and strong['J']>0
51     report['q2_strong_signal']={'evaluation':strong,'strong_radius_m':geo['radius_uppe
52     ↪ r'],'strong_J_contribution_m':strong['P_H']*geo['radius_upper']}
53     no_signal=evaluate(1700,0,[-930,0],frame='internal');geo=feedback_geometry(1700,0,
54     ↪ [-930,0],'N',frame='internal')
55     assert no_signal['P_N']>0 and abs(no_signal['P_finish_N']-no_signal['P_N'])<1e-10
56     ↪ and geo['radius_upper']<20 and geo['dmax']>1000
57     report['q2_no_signal_completion']={'evaluation':no_signal,'geometry':geo}
58     assert initial_geometry(1780,0)['rho1']<20
59     try:initial_geometry(4000,0)
60     except ValueError:pass
61     else:raise AssertionError('impossible first observation accepted')
62     try:evaluate(0,0,[0,0])
63     except ValueError:pass
64     else:raise AssertionError('same-location independent-error query accepted')
65     # MC shares the C++ MEC algorithm, but independently samples the conditional joint
66     ↪ prior.
67     sim=json.loads(native(0,0,'mc',*q,20000,20260913))
68     sim['J_zscore']=(sim['J_mean']-north['J'])/sim['J_se']
69     sim['P_zscore']=(sim['P_finish']-north['P_finish'])/sim['P_finish_se']
70     assert sim['invalid']==0 and sim['max_truth_outside']<1e-5
71     assert abs(sim['J_zscore'])<4 and abs(sim['P_zscore'])<4,sim
72     report['q2_joint_prior_mc']=sim
73     # Independent construction uses radial intervals and finite support circles.
74     geometries=[]
75     cases=[(0,0,[700,0],0,0),(1700,0,[-930,0],2,0),
76     ↪ (0,0,q,1,math.atan2(-q[1],700-q[0])),
77     ↪ (1200,30,[458.35,275.45],1,math.atan2(-275.45,400-458.35))]
78     for a,b,p,kind,theta in cases:
79         g=json.loads(native(a,b,'geometry',*p,kind,theta))
80         points=independent_points(a,b,p,kind,theta,n=16001)
81         assert len(points)>0
82         dirs=np.c_[np.cos(np.arange(64)*math.pi/32),np.sin(np.arange(64)*math.pi/32)]
83         v=points[np.unique([int(np.argmax(points@u)) for u in dirs])]
84         lower=exact_finite_radius(v);upper=g['radius_upper']
85         outside=float(np.max(np.linalg.norm(points-np.array(g['center']),axis=1))-uppe
86         ↪ r)
87         assert outside<2e-5 and upper>=lower-2e-5
88         geometries.append({'a':a,'beta_deg':b,'kind':kind,'q_internal':p,'sample_count'
89         ↪ ':len(points),'lower_radius_m':lower,'upper_radius_m':upper,'gap_m':upper-
90         ↪ lower,'max_sample_outside_m':outside})
91     report['q2_independent_geometry']=geometries
92     report.update(status='PASS',elapsed_s=time.monotonic()-start,
93     ↪ versions={'python':platform.python_version(),'numpy':np.__version__,
94     ↪ 'scipy':scipy.__version__},
95     ↪ source_sha256={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes(
96     ↪ )).hexdigest() for p in sorted(ROOT.rglob('*')) if p.suffix in
97     ↪ ('.py','.cpp','.hpp') and '__pycache__' not in p.parts},
98     ↪ limits='有限几何采样、积分精度递增和独立抽样是数值核验：
99     ↪ 不构成连续全局最优或物理实验验证。')
100     (out/'report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2))
101     print(json.dumps({'status':'PASS','q1_random_observations':60,'q1_projection_cases'
102     ↪ ':200','q2_reference_points':len(convergence),'q2_mc_samples':sim['n'],'elapsed'
103     ↪ '_s':report['elapsed_s']},ensure_ascii=False,indent=2))
104     return report
105
106 if __name__=='__main__':
107     parser=argparse.ArgumentParser(description=__doc__);parser.add_argument('--out',ty
108     ↪ pe=Path,required=True)
109     run(parser.parse_args().out)

```

11.5 问题三：多源协同定位与清除

文件 q3/v6/action_value.py

```
1 """Observation-only action values, with persistent nonconvex optical exclusions.
2
3 Scenario averages are planning heuristics, never completion certificates.
4 Actual safety continues to use an outer region and paid API actions.
5 """
6 import copy
7 import math
8 import numpy as np
9 from shapely.geometry import Polygon, Point
10 from shapely.geometry.polygon import orient
11 from bearing_geometry import enclosing_circle, diameter
12 from legacy_strategy import Track, CLEAR_RADIUS
13 from service_routing import ServicePlanner
14 from cost_aware import reception_safe
15
16
17 class OpticalTrack(Track):
18     def sync_region(self):
19         polygon=Polygon(self.poly)
20         self.feasible=polygon if not hasattr(self,'feasible') else
21         ↪ self.feasible.intersection(polygon)
22         self.refresh()
23
24     def refresh(self):
25         if self.feasible.is_empty or self.feasible.area<1e-14:
26             raise ArithmeticError('Optical feasible region empty or numerically
27             ↪ degenerate')
28         self.poly=np.asarray(orient(self.feasible.convex_hull,sign=1.).exterior.coords_
29         ↪[:-1])
30         self.center,self.radius,_=enclosing_circle(self.poly)
31
32     def update(self,point,observation):
33         super().update(point,observation)
34         self.sync_region()
35
36     def exclude_clear(self,point):
37         # Inscribed polygon is strictly inside the physical failed-clear disk.
38         # Its removal keeps a conservative OUTER set, including boundary margin.
39         self.feasible=self.feasible.difference(Point(point).buffer(19.999,quad_segs=16)
40         ↪)
41         self.refresh()
42
43
44 def clone_track(track):
45     t=copy.copy(track);t.poly=track.poly.copy();t.history=list(track.history)
46     return t # Shapely geometries are immutable.
47
48
49 def scenarios(track,count=3):
50     region=getattr(track,'feasible',Polygon(track.poly))
51     _,(i,j)=diameter(track.poly);a,b=track.poly[i],track.poly[j]
52     points=[]
53     for f in np.linspace(.15,.85,count):
54         q=(1-f)*a+f*b
55         q=.95*q+.05*np.asarray(region.representative_point().coords[0])
56         if region.covers(Point(q)):points.append(q)
57     if len(points)<count:
58         pieces=list(region.geoms) if hasattr(region,'geoms') else [region]
59         for piece in sorted(pieces,key=lambda g:g.area,reverse=True):
60             points.append(np.asarray(piece.representative_point().coords[0]))
61             if len(points)>=count:break
62     return points
63
64
65 def synthetic_observation(q,truth,error):
66     if np.linalg.norm(truth-q)<=5:return dict(measure_result='near')
67     angle=math.degrees(math.atan2(truth[1]-q[1],truth[0]-q[0]))
68     return dict(measure_result='direction',svd_deg=round((angle+error)%360,2)%360)
```

```

67 def finish_cost(track,start,truth,error,first=None,exit_point=None):
68     """Bounded rollout of the unchanged geometric completion module in seconds."""
69     t=clone_track(track);p=np.asarray(start);cost=0.
70     for k in range(5):
71         q=t.safe_clear_point(p)
72         if q is not None:
73             cost+=np.linalg.norm(q-p)/5+5
74             if exit_point is not None:cost+=np.linalg.norm(q-exit_point)/5
75             return float(cost)
76         q=first if k==0 and first is not None else t.next_point(p)
77         cost+=np.linalg.norm(q-p)/5+6 # Conservative switch allowance in planning.
78         t.update(q,synthetic_observation(q,truth,error));p=q
79     return float(cost+1000)
80
81
82 class ActionValuePlanner(ServicePlanner):
83     def __init__(self,robot,*,optical=True,optical_radius=65.,optical_margin=1.,
84                 optical_update=True,optical_risk=.2,optical_max_trials=2,
85                 value_probes=False,value_margin=3.,**kwargs):
86         super().__init__(robot,**kwargs)
87         self.optical=optical;self.optical_radius=optical_radius
88         self.optical_margin=optical_margin;self.optical_update=optical_update
89         self.optical_risk=optical_risk;self.optical_max_trials=optical_max_trials
90         self.value_probes=value_probes;self.value_margin=value_margin
91         self.optical_attempts={}
92
93     def observe(self,point,channel):
94         result=super().observe(point,channel)
95         if channel in self.tracks:
96             t=self.tracks[channel]
97             if not isinstance(t,OpticalTrack):t.__class__=OpticalTrack
98             t.sync_region()
99             self.robot.record(dict(kind='optical_region',channel=channel,
100                                   wkt=t.feasible.wkt,vertices=t.poly.tolist(),radius_m=t.radius))
101         return result
102
103     def aggregate(self,values):
104         return float((1-self.optical_risk)*np.mean(values)+self.optical_risk*np.max(va
105 ↪ lues))
106
107     def optical_action(self,t):
108         if not self.optical or not CLEAR_RADIUS<t.radius<=self.optical_radius:return
109 ↪ None
110         if len(self.optical_attempts.get(t.channel,[]))>=self.optical_max_trials:return
111 ↪ None
112         start=np.asarray(self.robot.position);samples=scenarios(t)
113         states=[(x,e) for x in samples for e in [-1.,0.,1.]]
114         baseline=self.aggregate([finish_cost(t,start,x,e) for x,e in states])
115         candidates=[start,t.center]
116         d=start-t.center;length=np.linalg.norm(d)
117         if length>1e-8:candidates.append(t.center+d/length*min(15.,t.radius*.35))
118         _,(i,j)=diameter(t.poly)
119         if t.radius>35:
120             candidates.extend([.65*t.center+.35*t.poly[i],.65*t.center+.35*t.poly[j]])
121         best=None
122         for q in candidates:
123             if any(math.dist(q,p)<2. for p in
124 ↪ self.optical_attempts.get(t.channel,[])):continue
125             covered=t.feasible.intersection(Point(q).buffer(19.999,quad_segs=16)).area
126             if covered<.12*t.feasible.area:continue
127             failed=clone_track(t)
128             try:
129                 if self.optical_update:failed.exclude_clear(q)
130             except ArithmeticError:continue
131             values=[]
132             for x,e in states:
133                 value=np.linalg.norm(q-start)/5
134                 if np.linalg.norm(x-q)<=20:value+=5
135                 else:value+=3+finish_cost(failed,q,x,e)
136             values.append(value)
137             score=self.aggregate(values)
138             if score<baseline-self.optical_margin and (best is None or score<best[0]):
139                 best=(score,q,baseline,covered/t.feasible.area)
140         return best

```

```

138 def solve_track(self, track):
139     for _ in range(20):
140         self.robot.check_budget(); track=self.tracks[track.channel]
141         if track.radius<=CLEAR_RADIUS: return super().solve_track(track)
142         proposal=self.optical_action(track)
143         if proposal is not None:
144             score,q,baseline,fraction=proposal
145             self.robot.record(dict(kind='optical_decision', channel=track.channel, p
146                 ↪ oint=q.tolist(),
147                 ↪ predicted_s=score, radio_predicted_s=baseline, area_fraction=fraction
148                 ↪ n))
149             self.optical_attempts.setdefault(track.channel, []).append(q.tolist())
150             if self.robot.clear(tuple(q), track.channel): return
151             if self.optical_update: track.exclude_clear(q)
152             self.robot.record(dict(kind='optical_failure_update', channel=track.cha
153                 ↪ nnel,
154                 ↪ point=q.tolist(), wkt=track.feasible.wkt, vertices=track.poly.tolist
155                 ↪ (), radius_m=track.radius))
156             continue
157             q=track.next_point(np.asarray(self.robot.position))
158             if self.observe(q, track.channel)['measure_result']=='no_signal':
159                 raise ArithmeticError('Certified radio point returned no signal')
160             raise RuntimeError('Action-value localization budget exhausted')
161
162 def probe_on_leg(self, node):
163     if not self.value_probes: return super().probe_on_leg(node)
164     start=np.asarray(self.robot.position); end=np.asarray(node[2])
165     if node[0]=='target':
166         tr=self.tracks[node[1]]; safe=tr.safe_clear_point(start)
167         end=safe if safe is not None else tr.next_point(start)
168     if np.linalg.norm(end-start)<250: return None
169     tracks=[t for c,t in self.tracks.items() if c not in self.robot.cleared and
170         ↪ t.radius>40]
171     tracks=sorted(tracks, key=lambda t: np.linalg.norm(t.center-end))[:3]
172     baselines={}
173     for t in tracks:
174         states=[(x,e) for x in scenarios(t) for e in [-1.,0.,1.]]
175         baselines[t.channel]=(states, self.aggregate([finish_cost(t,end,x,e) for x,e
176             ↪ in states]))
177     best=None
178     for fraction in [.25,.5,.75]:
179         q=start+fraction*(end-start); channels=[]; net=0.
180         for t in tracks:
181             if not reception_safe(t,q): continue
182             if min(math.dist(q,h['position'])) for h in t.history)<150: continue
183             if math.dist(q, self.attempts.get(t.channel, (1e9,1e9)))<150: continue
184             states,before=baselines[t.channel]; values=[]
185             for x,e in states:
186                 after=clone_track(t); after.update(q, synthetic_observation(q,x,e))
187                 values.append(finish_cost(after,end,x,e))
188             saving=before-self.aggregate(values)-6
189             if saving>self.value_margin: channels.append(t.channel); net+=saving
190             if channels and (best is None or net>best[0]): best=(net,q,channels)
191     if best:
192         self.robot.record(dict(kind='value_bundle', point=best[1].tolist(), channels
193             ↪ =best[2], predicted_saving_s=best[0]))
194     return None if best is None else best[1:]

```

文件 q3/v6/api_client.py

```

1 """Serial, idempotent simulator client. No network calls on import."""
2 import datetime as dt
3 from http.client import HTTPException
4 import json
5 import math
6 from pathlib import Path
7 import time
8 import uuid
9 from urllib.error import HTTPError, URLError
10 from urllib.request import Request, build_opener, ProxyHandler
11
12

```

```

13 class BudgetReached(Exception):
14     pass
15
16
17 class Robot:
18     def __init__(self, team, url, directory):
19         self.team, self.url = team, url.rstrip('/')
20         self.directory = Path(directory)
21         self.directory.mkdir(parents=True, exist_ok=False)
22         self.log = (self.directory / 'actions.jsonl').open('w', encoding='utf-8')
23         self.opener = build_opener(ProxyHandler({}))
24         self.prefix, self.sequence = uuid.uuid4().hex[:12], 0
25         self.virtual, self.deadline = 0., float('inf')
26         self.started = None
27         self.exit_confirmed = self.transport_uncertain = False
28         self.cleared, self.discovered = set(), set()
29         self.measure_count = self.clear_failures = self.scanned_sites = 0
30         self.position, self.channel = (0., 0.), 1
31         self.distance_m = 0.
32         self.switches = 0
33
34     def record(self, value):
35         self.log.write(json.dumps(value, ensure_ascii=False, allow_nan=False) + '\n')
36         self.log.flush()
37
38     def check_budget(self):
39         if time.monotonic() > self.deadline - 20 or self.virtual > 349000:
40             raise BudgetReached('接近运行时间上限, 保留退出余量')
41         if self.transport_uncertain:
42             raise RuntimeError('上一动作是否执行尚不确定, 停止发送新动作')
43
44     def call(self, path, **fields):
45         if self.transport_uncertain:
46             raise RuntimeError('上一动作执行状态不确定, 不能发送新请求')
47         if path != '/exit':
48             self.check_budget()
49         self.sequence += 1
50         payload = dict(arena_id='default', robot_id=self.team,
51                       request_id=f'{self.prefix}-{self.sequence}', **fields)
52         body = json.dumps(payload, allow_nan=False).encode('utf-8')
53         for attempt in range(1, 4):
54             self.record(dict(kind='request', time=dt.datetime.now().isoformat(),
55                             ↪ path=path,
56                             attempt=attempt, payload=payload))
57         try:
58             req = Request(self.url + path, data=body,
59                           headers={'Content-Type': 'application/json'},
60                           ↪ method='POST')
61         except:
62             pass
63         try:
64             with self.opener.open(req, timeout=4) as response:
65                 status, raw = response.status, response.read()
66         except HTTPError as error:
67             status, raw = error.code, error.read()
68         if status >= 500:
69             raise ConnectionError(f'HTTP {status}: action state uncertain')
70         data = json.loads(raw.decode('utf-8'))
71         self.record(dict(kind='response', request_id=payload['request_id'],
72                         http_status=status, response=data))
73         if not isinstance(data, dict) or 'accepted' not in data:
74             raise ValueError('接口响应缺少 accepted 字段')
75         if status != 200 or data['accepted'] is not True:
76             # Explicit rejection is definite; its zero clock is not a reset.
77             raise RuntimeError(f'{path} 未接受: HTTP {status}, {data}')
78         virtual = float(data['virtual_time_s'])
79         if not math.isfinite(virtual) or virtual < self.virtual - 1e-5:
80             raise ValueError('接口时钟无效或倒退')
81         if path == '/measure':
82             if data.get('measure_result') not in ('no_signal', 'near',
83                                                 ↪ 'direction'):
84                 raise ValueError('无效测量结果')
85             if data['measure_result'] == 'direction' and not
86                 ↪ math.isfinite(float(data['svd_deg'])):
87                 raise ValueError('无效示向度')
88         if path == '/clear' and data.get('clear_result') not in ('success',
89                                                                 ↪ 'no_target_in_range'):

```

```

83         raise ValueError('无效清除结果')
84     if path == '/enter':
85         remaining = float(data['remaining_real_duration_s'])
86         if not math.isfinite(remaining) or remaining < 0:
87             raise ValueError('无效剩余时间')
88         self.deadline = self.started + min(remaining, 1200.)
89     self.virtual = virtual
90     if path == '/exit':
91         self.exit_confirmed = True
92     return data
93 except (URLError, TimeoutError, ConnectionError, OSError, HTTPException,
94         ValueError, KeyError, TypeError) as error:
95     self.record(dict(kind='uncertain_response', attempt=attempt,
96                     ↪ error=str(error)))
97     if attempt == 3:
98         self.transport_uncertain = True
99         raise RuntimeError('同一请求重试仍未确认; 已停止新动作, 请查看模拟器和日志')
100         ↪ from error
101     time.sleep(.25)
102
103 def _move(self, point):
104     self.distance_m += math.dist(self.position, point)
105     self.position = tuple(point)
106
107 def measure(self, point, channel):
108     data = self.call('/measure', position={'x': float(point[0]), 'y':
109     ↪ float(point[1])}, channel=channel)
110     self._move(point)
111     self.switches += int(channel != self.channel)
112     self.channel = channel
113     self.measure_count += 1
114     if data['measure_result'] != 'no_signal':
115         self.discovered.add(channel)
116     return data
117
118 def clear(self, point, channel):
119     data = self.call('/clear', position={'x': float(point[0]), 'y':
120     ↪ float(point[1])}, channel=channel)
121     self._move(point)
122     success = data['clear_result'] == 'success'
123     if success:
124         self.cleared.add(channel)
125         print(f' 清除频道 {channel:2d}; 累计 {len(self.cleared)} 个; 虚拟时间
126         ↪ {self.virtual:.1f} 秒', flush=True)
127     else:
128         self.clear_failures += 1
129     # /clear does not change the measurement channel.
130     return success

```

文件 q3/v6/bearing_geometry.py

```

1  """Q1 M01 v01: directed bearing wedges, polygon diameter and enclosing disk.
2
3  All coordinates are metres, public angles degrees, inequalities A @ x <= b.
4  This module makes no stochastic noise assumption and no simulator requests.
5  """
6  from dataclasses import dataclass
7  from itertools import combinations
8  import numpy as np
9  from scipy.optimize import linprog
10
11
12  @dataclass
13  class Region:
14      status: str
15      vertices: np.ndarray
16      max_residual: float = 0.0
17
18
19  def unit(degrees):
20      a = np.deg2rad(degrees)
21      return np.array([np.cos(a), np.sin(a)])
22

```

```

23
24 def bearing(point, sensor):
25     d = np.asarray(point) - np.asarray(sensor)
26     return float(np.rad2deg(np.arctan2(d[1], d[0]))) % 360
27
28
29 def angle_difference(a, b):
30     return (np.asarray(a) - np.asarray(b) + 180) % 360 - 180
31
32
33 def wedge(sensor, angle_deg, epsilon_deg=1.0):
34     """Closed, forward wedge; valid for 0 < epsilon < 90 degrees."""
35     if not 0 < epsilon_deg < 90:
36         raise ValueError("epsilon must lie strictly between 0 and 90 degrees")
37     s = np.asarray(sensor, dtype=float)
38     lower, upper = unit(angle_deg - epsilon_deg), unit(angle_deg + epsilon_deg)
39     A = np.array([[lower[1], -lower[0]], [-upper[1], upper[0]]])
40     return A, A @ s
41
42
43 def observations_halfplanes(observations, epsilon_deg=1.0):
44     parts = [wedge(o['position'], o['bearing_deg'], epsilon_deg) for o in observations]
45     if not parts:
46         return np.empty((0, 2)), np.empty(0)
47     return np.vstack([p[0] for p in parts]), np.hstack([p[1] for p in parts])
48
49
50 def unique_points(points, tol=1e-7):
51     kept = []
52     for p in points:
53         if not any(np.linalg.norm(p - q) <= tol for q in kept):
54             kept.append(p)
55     return np.array(kept, dtype=float).reshape(-1, 2)
56
57
58 def halfplane_region(A, b, tol=1e-7):
59     """Classify without an artificial clipping box; enumerate bounded vertices.
60
61     Four LP support problems distinguish unbounded sets from bounded polygons.
62     Pairwise line intersections cost  $O(m^3)$ , appropriate for few observations.
63     The result is floating point, not interval-arithmetic certification.
64     """
65     A, b = np.asarray(A, float).reshape(-1, 2), np.asarray(b, float)
66     if not len(A):
67         return Region('unbounded', np.empty((0, 2)))
68     lengths = np.linalg.norm(A, axis=1)
69     zero = lengths < 1e-15
70     if np.any(b[zero] < -tol):
71         return Region('empty', np.empty((0, 2)))
72     A, b = A[~zero] / lengths[~zero], b[~zero] / lengths[~zero]
73     if not len(A):
74         return Region('unbounded', np.empty((0, 2)))
75     feasible = linprog([0, 0], A_ub=A, b_ub=b, bounds=[(None, None)] * 2,
76         ↪ method='highs')
77     if feasible.status == 2:
78         return Region('empty', np.empty((0, 2)))
79     if not feasible.success:
80         raise RuntimeError(feasible.message)
81     # Translation improves conditioning for noncentral observations.
82     origin = feasible.x
83     rhs = b - A @ origin
84     extrema = []
85     for c in [[1, 0], [-1, 0], [0, 1], [0, -1]]:
86         sol = linprog(c, A_ub=A, b_ub=rhs, bounds=[(None, None)] * 2, method='highs')
87         if sol.status == 3:
88             return Region('unbounded', np.empty((0, 2)))
89         if not sol.success:
90             raise RuntimeError(sol.message)
91         extrema.append(sol.x)
92     points = []
93     for i, j in combinations(range(len(A)), 2):
94         rows = A[[i, j]]
95         if abs(np.linalg.det(rows)) < 1e-12:
96             continue
97         x = np.linalg.solve(rows, rhs[[i, j]])

```

```

97         if np.max(A @ x - rhs) <= tol:
98             points.append(x)
99     if not points:
100         points = extrema # Degenerate point/segment fallback, never arbitrary box.
101     vertices = unique_points(points, tol) + origin
102     if len(vertices) > 2:
103         center = vertices.mean(axis=0)
104         order = np.argsort(np.arctan2(vertices[:, 1] - center[1], vertices[:, 0] -
105             ↪ center[0]))
106         vertices = vertices[order]
107     status = 'point' if len(vertices) == 1 else 'segment' if len(vertices) == 2 else
108     ↪ 'polygon'
109     return Region(status, vertices, float(max(0.0, np.max(A @ vertices.T - b[:,
110     ↪ None]])))
111
112 def diameter(vertices):
113     p = np.asarray(vertices, float).reshape(-1, 2)
114     if not len(p):
115         raise ValueError('diameter needs a nonempty bounded region')
116     d2 = np.sum((p[:, None] - p[None, :]) ** 2, axis=2)
117     i, j = np.unravel_index(np.argmax(d2), d2.shape)
118     return float(np.sqrt(d2[i, j])), (int(i), int(j))
119
120 def enclosing_circle(vertices, tol=1e-8):
121     """Enumerate support disks (2 or 3 points), with a diameter-disk shortcut.
122
123     Exact finite-support algorithm in real arithmetic;  $O(h^4)$  worst case.
124     Final radius is recomputed from all points, giving a conservative numerical
125     covering disk even when tolerance affects the support choice.
126     """
127     p = np.asarray(vertices, float).reshape(-1, 2)
128     D, (i, j) = diameter(p)
129     center = (p[i] + p[j]) / 2
130     if np.max(np.linalg.norm(p - center, axis=1)) <= D / 2 + tol:
131         return center, float(np.max(np.linalg.norm(p - center, axis=1))), [i, j]
132     best = (np.inf, None, None)
133     for i, j, k in combinations(range(len(p)), 3):
134         d = p[[j, k]] - p[i]
135         determinant = np.linalg.det(d)
136         if abs(determinant) <= 1e-13 * max(1., np.linalg.norm(d[0]) *
137             ↪ np.linalg.norm(d[1])):
138             continue
139         offset = np.linalg.solve(2 * d, np.sum(d * d, axis=1))
140         r = np.linalg.norm(offset)
141         if r >= best[0] + tol:
142             continue
143         c = p[i] + offset
144         distances = np.linalg.norm(p - c, axis=1)
145         if distances.max() <= r + tol:
146             best = (float(distances.max()), c, [i, j, k])
147     if best[1] is None:
148         raise ArithmeticError('Could not construct a covering disk; inspect
149             ↪ conditioning')
150     return best[1], best[0], best[2]
151
152 def summarize_region(region):
153     if region.status == 'empty':
154         return {'status': 'empty', 'diameter_m': None, 'cover_radius_m': None}
155     if region.status == 'unbounded':
156         return {'status': 'unbounded', 'diameter_m': 'infinity', 'cover_radius_m':
157             ↪ 'infinity'}
158     D, pair = diameter(region.vertices)
159     c, r, support = enclosing_circle(region.vertices)
160     return dict(status=region.status, vertices=region.vertices.tolist(), diameter_m=D,
161         ↪ diameter_pair=list(pair), cover_center=c.tolist(), cover_radius_m=r,
162         ↪ support_indices=support, diameter_disk_covers=bool(r <= D / 2 + 1e-7),
163         ↪ guaranteed_clear_possible=bool(r <= 20 - 1e-6),
164         ↪ max_halfplane_residual_m=region.max_residual)
165
166 def clip_polygon(poly, normal, offset, tol=1e-9):
167     """Sutherland-Hodgman clipping; caller must supply a genuine prior polygon."""

```

```

166     p = np.asarray(poly, float).reshape(-1, 2)
167     if not len(p):
168         return p
169     values = p @ normal - offset
170     output = []
171     for i in range(len(p)):
172         j = (i + 1) % len(p)
173         inside_i, inside_j = values[i] <= tol, values[j] <= tol
174         if inside_i:
175             output.append(p[i])
176         if inside_i != inside_j:
177             t = values[i] / (values[i] - values[j])
178             output.append(p[i] + np.clip(t, 0, 1) * (p[j] - p[i]))
179     return np.asarray(output, float).reshape(-1, 2)
180
181
182 def clip_halfplanes(poly, A, b):
183     for a, rhs in zip(A, b):
184         poly = clip_polygon(poly, a, rhs)
185         if not len(poly):
186             break
187     return poly
188
189
190 def outer_disk(center, radius, sides=360):
191     """Circumscribed regular polygon, never an inscribed disk approximation."""
192     angles = (np.arange(sides) + .5) * 2 * np.pi / sides
193     return np.asarray(center) + radius / np.cos(np.pi / sides) *
194         ↪ np.column_stack((np.cos(angles), np.sin(angles)))
195
196
197 def disk_halfplanes(center, radius, sides=360):
198     angles = np.arange(sides) * 2 * np.pi / sides
199     A = np.column_stack((np.cos(angles), np.sin(angles)))
200     return A, radius + A @ np.asarray(center)

```

文件 q3/v6/belief_components.py

```

1  """Belief-sampled one-step policy improvement. No access to real world truth.
2
3  Uniform position/radius priors are efficiency assumptions, never certificates.
4  All physical actions retain bounded-region clearing and whole-cell completion.
5  """
6  import copy
7  import hashlib
8  import math
9  import numpy as np
10 from joint_strategies import JointPlanner, open_route
11
12
13 class BeliefWorld:
14     def __init__(self, targets, robot, salt):
15         self.targets=targets
16         self.position=tuple(robot.position);self.channel=robot.channel
17         self.cleared=set(robot.cleared);self.discovered=set(robot.discovered)
18         self.scanned_sites=0;self.virtual=0.;self.salt=salt
19     def check_budget(self):
20         if self.virtual>50000:raise RuntimeError('rollout cap')
21     def record(self,event):pass
22     def move(self,p):
23         self.virtual+=math.dist(p,self.position)/5;self.position=tuple(p)
24     def measure(self,p,c):
25         self.move(p);self.virtual+=5+int(c!=self.channel);self.channel=c
26         target=self.targets.get(c) if c not in self.cleared else None
27         d=math.dist(p,target[:2]) if target is not None else math.inf
28         if target is None or d>target[2]:return {'measure_result':'no_signal'}
29         self.discovered.add(c)
30         if d<=5:return {'measure_result':'near'}
31         error=math.sin(.039*p[0]+.021*p[1]+c*2.173+self.salt)
32         a=math.degrees(math.atan2(target[1]-p[1],target[0]-p[0]))
33         return {'measure_result':'direction','svd_deg':round((a+error)%360,2)%360}
34     def clear(self,p,c):
35         self.move(p);t=self.targets.get(c)

```

```

36         success=t is not None and c not in self.cleared and math.dist(p,t[:2])<=20
37         self.virtual+=5 if success else 3
38         if success:self.cleared.add(c)
39         return success
40
41
42     def execute(planner,node):
43         kind,c,p=node
44         if kind=='scan':planner.at_site_scan(p,force=True)
45         else:planner.solve_track(planner.tracks[c])
46
47
48     def finish(planner):
49         for _ in range(100):
50             planner.robot.check_budget()
51             pending=[c for c in planner.tracks if c not in planner.robot.cleared]
52             if not pending and not planner.unknown():return planner.robot.virtual
53             planner.at_site_scan(planner.robot.position)
54             nodes=planner.candidates()
55             if not nodes:continue
56             order=open_route(planner.robot.position,[n[2] for n in nodes])
57             execute(planner,nodes[order[0]])
58             raise RuntimeError('rollout exhausted')
59
60
61     class RolloutPlanner(JointPlanner):
62         def __init__(self,robot,*,rollout_samples=3,rollout_actions=5,rollout_margin=0.,**
        ↪ kwargs):
63             super().__init__(robot,**kwargs)
64             self.rollout_samples=rollout_samples;self.rollout_actions=rollout_actions
65             self.rollout_margin=rollout_margin;self.rollout_calls=0
66
67         def sample_worlds(self):
68             # Seed depends only on the observed history, never the benchmark seed.
69             digest=hashlib.sha256(repr((self.robot.position,[(c,len(t.history)) for c,t in
        ↪ self.tracks.items()]),self.rollout_calls)).encode()).digest()
70             rng=np.random.default_rng(int.from_bytes(digest[:8],'little'))
71             unknown=[c for c in range(1,21) if c not in self.robot.discovered]
72             radius=1800*np.sqrt(rng.random(12000));angle=rng.uniform(0,2*math.pi,12000)
73             particles=np.column_stack([radius*np.cos(angle),radius*np.sin(angle)])
74             if unknown and self.coverage.samples[unknown[0]]:
75                 positions=np.asarray(self.coverage.samples[unknown[0]])
76                 upper=np.minimum(1500.,np.min(np.linalg.norm(particles[:,None,:]-positions
        ↪ [None,[:,:],axis=2),axis=1))
77             else:upper=np.full(len(particles),1500.)
78             weights=np.maximum(upper-1000.,0.)
79             probability=float(np.mean(weights)/500)
80             m=len(self.robot.discovered)
81             counts=np.arange(max(10,m),17)
82             probs=np.array([math.comb(int(n),m)*probability**int(n-m) for n in
        ↪ counts],float)
83             if probs.sum()==0:counts=np.array([m]);probs=np.ones(1)
84             probs/=probs.sum()
85             worlds=[]
86             for sample in range(self.rollout_samples):
87                 targets={}
88                 for c,t in self.tracks.items():
89                     if c in self.robot.cleared:continue
90                     poly=t.poly;tri=poly[1:-1];tri2=poly[2:]
91                     d1=tri-poly[0];d2=tri2-poly[0]
92                     area=np.abs(d1[:,0]*d2[:,1]-d1[:,1]*d2[:,0])
93                     positive=np.asarray([h['position'] for h in t.history])
94                     negative=np.asarray(self.coverage.samples[c])
95                     for _ in range(100):
96                         ids=rng.choice(len(area),size=128,p=area/area.sum())
97                         uv=rng.random((128,2));uv[uv.sum(axis=1)>1]=1-uv[uv.sum(axis=1)>1]
98                         pts=poly[0]+uv[:,0,None]*d1[ids]+uv[:,1,None]*d2[ids]
99                         lower=np.maximum(1000.,np.max(np.linalg.norm(pts[:,None,:]-positiv
        ↪ e[None,[:,:],axis=2),axis=1))
100                         upper_r=np.full(128,1500.) if not len(negative) else
        ↪ np.minimum(1500.,np.min(np.linalg.norm(pts[:,None,:]-negative[
        ↪ None,[:,:],axis=2),axis=1))
101                         ok=np.flatnonzero((upper_r>lower)&(np.linalg.norm(pts,axis=1)<=180
        ↪ 0.))

```

```

102         if len(ok):
103             mass=upper_r[ok]-lower[ok];j=int(rng.choice(ok,p=mass/mass.sum()
                ↳ ());targets[c]=[*pts[j],rng.uniform(lower[j],upper_r[j])]
                ↳ ;break
104         else:raise RuntimeError('belief rejection exhausted')
105     count=int(rng.choice(counts,p=probs))-m
106     if count:
107         channels=rng.choice(unknown,size=count,replace=False)
108         ids=rng.choice(len(particles),size=count,p=weights/weights.sum())
109         for c,j in zip(channels,ids):targets[int(c)]=[*particles[j],rng.unifor
                ↳ m(1000.,upper[j])]
110     worlds.append((targets,float(rng.uniform(0,1000))))
111     return worlds
112
113 def candidates(self):
114     nodes=super().candidates()
115     if len(nodes)<=1:return nodes
116     order=open_route(self.robot.position,[n[2] for n in nodes])
117     # Include baseline choice, nearest alternatives and a search action.
118     choices=[order[0]]
119     nearest=sorted(range(len(nodes)),key=lambda
                ↳ i:math.dist(self.robot.position,nodes[i][2]))
120     scans=[i for i in order if nodes[i][0]=='scan']
121     if scans and scans[0] not in choices:choices.append(scans[0])
122     for i in nearest:
123         if i not in choices:choices.append(i)
124         if len(choices)>=self.rollout_actions:break
125     self.rollout_calls+=1
126     try:worlds=self.sample_worlds()
127     except (RuntimeError,ValueError):return [nodes[order[0]]]
128     state=copy.deepcopy({k:v for k,v in self.__dict__.items() if k!='robot'})
129     scores=[]
130     for i in choices:
131         costs=[]
132         for targets,salt in worlds:
133             robot=BeliefWorld(targets,self.robot,salt)
134             planner=object.__new__(JointPlanner)
135             planner.__dict__=copy.deepcopy(state);planner.robot=robot
136             try:
137                 execute(planner,nodes[i]);costs.append(finish(planner))
138             except (RuntimeError,ArithmeticError,ValueError):costs.append(float('i
                ↳ nf'))
139         scores.append(float(np.mean(costs)))
140     best=int(np.argmax(scores))
141     if scores[0]-scores[best]<self.rollout_margin:best=0
142     self.robot.record(dict(kind='rollout_decision',alternatives=[dict(kind=nodes[i]
                ↳ ][0],channel=int(nodes[i][1]),predicted_remaining_s=s if math.isfinite(s)
                ↳ else None) for i,s in
                ↳ zip(choices,scores)],selected=best,samples=len(worlds)))
143     return [nodes[choices[best]]]

```

文件 q3/v6/clearance_geometry.py

本文件与 q1-q2/q1/clearance_geometry.py 内容完全相同，使用前述代码。

文件 q3/v6/combined_cost_cover.py

```

1  """Combine the independently developed localization and scan-cover candidates."""
2  from cost_aware import CostAwarePlanner
3  from merged_cover import MergedCoverPlanner
4
5
6  class CombinedCostCoverPlanner(CostAwarePlanner):
7      candidates = MergedCoverPlanner.candidates
8
9      def __init__(self, robot, *, merge_stations=True, merge_target_scans=True,
                ↳ **kwargs):
10         super().__init__(robot, **kwargs)
11         self.merge_stations = merge_stations
12         self.merge_target_scans = merge_target_scans

```

文件 q3/v6/convex_cover.py

```

1  """Continuous waypoint relocation with an explicit full-coverage constraint."""
2  import math
3  import numpy as np
4  from scipy.optimize import minimize
5  from scipy.spatial import ConvexHull
6  from joint_strategies import JointPlanner, open_route
7
8
9  class ConvexCoverPlanner(JointPlanner):
10     def __init__(self, robot, *, cover_sweeps=2, **kwargs):
11         super().__init__(robot, family='adaptive', **kwargs)
12         self.cover_sweeps=cover_sweeps
13
14     def candidates(self):
15         targets=[]
16         for c,t in self.tracks.items():
17             if c in self.robot.cleared:continue
18             q=t.safe_clear_point(self.robot.position)
19             q=t.center if q is None else q
20             targets.append(('target',c,np.asarray(q)))
21         unknown=self.unknown()
22         if not unknown:return targets
23         actual=self.coverage.covered[unknown[0]]
24         # Targets are mandatory route vertices but optional sensing sites.
25         points=[n[2].copy() for n in targets]+[np.array(p) for p in self.anchors]
26         n_target=len(targets)
27         active=np.ones(len(points),dtype=bool)
28         present=np.ones(len(points),dtype=bool)
29         masks=[self.coverage.mask(p) for p in points]
30         start=np.asarray(self.robot.position)
31         def route_order():
32             indices=[i for i in range(len(points)) if present[i]]
33             return [indices[j] for j in open_route(start,[points[i] for i in indices])]
34         def total_without(i):
35             total=actual.copy()
36             for j in range(len(points)):
37                 if j!=i and active[j]:total+=masks[j]
38             return total
39         def prune():
40             while True:
41                 order=route_order();gains=[]
42                 for k,i in enumerate(order):
43                     if not active[i]:continue
44                     gain=30*len(unknown)
45                     if i>=n_target:
46                         a=start if k==0 else points[order[k-1]]
47                         gain+=np.linalg.norm(points[i]-a)
48                     if k+1<len(order):
49                         b=points[order[k+1]]
50                         gain+=np.linalg.norm(points[i]-b)-np.linalg.norm(a-b)
51                     gains.append((gain,i))
52                 removed=False
53                 for gain,i in sorted(gains,reverse=True):
54                     if np.all(total_without(i)):
55                         active[i]=False
56                         if i>=n_target:present[i]=False
57                         removed=True;break
58                 if not removed:break
59         prune()
60         for _ in range(self.cover_sweeps):
61             order=route_order()
62             improvement=0.
63             for k,i in enumerate(order):
64                 if i<n_target or not active[i]:continue
65                 needed=self.coverage.centers[~total_without(i)]
66                 if not len(needed):continue
67                 h=self.coverage.half
68                 corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in
69                                     ↪ [(-1,-1),(-1,1),(1,-1),(1,1)])])
69                 witnesses=corners[ConvexHull(corners).vertices]/1000
70                 before=points[i]/1000
71                 previous=(start if k==0 else points[order[k-1]])/1000
72                 following=points[order[k+1]]/1000 if k+1<len(order) else None

```

```

73     def objective(q):
74         value=np.sqrt(np.dot(q-previous,q-previous)+1e-16)
75         if following is not
76             ↪ None:value+=np.sqrt(np.dot(q-following,q-following)+1e-16)
77         return float(value)
78     def gradient(q):
79         value=(q-previous)/max(np.linalg.norm(q-previous),1e-8)
80         if following is not
81             ↪ None:value+=(q-following)/max(np.linalg.norm(q-following),1e-8)
82         return value
83     cons={'type':'ineq','fun':lambda q:
84         ↪ .999989999**2-np.sum((q-witnesses)**2,axis=1),
85         'jac':lambda q:-2*(q-witnesses)}
86     sol=minimize(objective,before,jac=gradient,constraints=cons,method='SL_
87         ↪ SQP',
88         options={'ftol':1e-10,'maxiter':60})
89     if np.max(np.linalg.norm(witnesses-sol.x,axis=1))<=.99999 and
90         ↪ objective(sol.x)<=objective(before)+1e-8:
91         improvement+=objective(before)-objective(sol.x)
92         points[i]=sol.x*1000;masks[i]=self.coverage.mask(points[i])
93     prune()
94     if improvement<1e-5:break
95     union=actual.copy()
96     for i in range(len(points)):
97         if active[i]:union|=masks[i]
98     if not np.all(union):raise ArithmeticError('Relocated cover lost cells')
99     scans=[('scan',-100-i,points[i]) for i in range(len(points)) if active[i]]
100     nodes=targets+scans
101     self.cover_targets=set()
102     order=open_route(start,[n[2] for n in nodes])
103     if nodes[order[0]][0]=='target':
104         first=nodes[order[0]]
105     matches=[n for n in scans if math.dist(n[2],first[2])<1e-5]
106     if matches:return [matches[0]]
107     return nodes

```

文件 q3/v6/cost_aware.py

```

1  """Cost-aware localization and paid sharing of intermediate sensing sites.
2
3  Inspired by the cost objectives of Vander Hook (2014), multi-target planning
4  of Engin & Isler (2020), and approach-to-source routing in Gu et al. (2018).
5  No Gaussian guarantees, true target inputs, or free simultaneous sensing.
6  """
7  import copy
8  import math
9  import numpy as np
10 from convex_cover import ConvexCoverPlanner
11 from legacy_strategy import Track,CLEAR_RADIUS,EPS
12 from bearing_geometry import diameter
13 from clearance_geometry import nearest_clear_point
14 from reception_region import reception_violation
15
16
17 class RefinedTrack(Track):
18     def safe_clear_point(self,position):
19         if self.radius>CLEAR_RADIUS:return None
20         return nearest_clear_point(self.poly,position,CLEAR_RADIUS,self.center)
21
22
23 def reception_safe(track,q):
24     if np.max(np.linalg.norm(track.poly-q,axis=1))<=999.9:return True
25     first=track.history[0]
26     if first['measure_result']!='direction':return False
27     angle=math.radians(first['svd_deg']);u=np.array([math.cos(angle),math.sin(angle)])
28     v=np.array([-u[1],u[0]]);d=q-np.asarray(first['position'])
29     a,b=np.dot(d,u),np.dot(d,v);l2=np.dot(d,d);ep=math.radians(EPS)
30     return l2<=1000**2-1e-3 and l2<=2000*(a*math.cos(ep)-abs(b)*math.sin(ep))-1e-3
31
32
33 def forecast(track,start,first_point,truth,error,exit_point=None):
34     t=copy.copy(track);t.poly=track.poly.copy();t.history=list(track.history)

```

```

35 p=np.asarray(start);q=first_point;cost=0.
36 for k in range(5):
37     cost+=float(np.linalg.norm(q-p))/5+5;p=q
38     if np.linalg.norm(truth-q)<=5:obs={'measure_result':'near'}
39     else:
40         angle=math.degrees(math.atan2(truth[1]-q[1],truth[0]-q[0]))
41         obs={'measure_result':'direction','svd_deg':round((angle+error)%360,2)%360}
42     t.update(q,obs)
43     clear=t.safe_clear_point(q)
44     if clear is not None:
45         cost+=float(np.linalg.norm(clear-q))/5+5
46         if exit_point is not None:cost+=float(np.linalg.norm(clear-exit_point))/5
47         return cost
48     q=t.next_point(p)
49 return cost+1000.
50
51
52 class CostAwarePlanner(ConvexCoverPlanner):
53     def __init__(self,robot,*,projection=True,cost_action=False,cost_risk=.25,
54                 intermediate_known=False,intermediate_gain=2.,exit_cost=False,exact_r_
55                 ↪ eception=False,**kwargs):
56         super().__init__(robot,**kwargs)
57         self.projection=projection;self.cost_action=cost_action;self.cost_risk=cost_r_
58         ↪ sk
59         self.intermediate_known=intermediate_known;self.intermediate_gain=intermediate_
60         ↪ gain
61         self.exit_cost=exit_cost
62         self.exact_reception=exact_reception
63
64     def observe(self,point,channel):
65         result=super().observe(point,channel)
66         if self.projection and channel in self.tracks and not
67         ↪ isinstance(self.tracks[channel],RefinedTrack):
68             track=RefinedTrack(channel);track.__dict__=self.tracks[channel].__dict__
69             self.tracks[channel]=track
70         return result
71
72     def pick_point(self,track):
73         start=np.asarray(self.robot.position);center=track.center
74         default=track.next_point(start)
75         _,(i,j)=diameter(track.poly);a,b=track.poly[i],track.poly[j]
76         direction=b-a;normal=np.array([-direction[1],direction[0]])/max(np.linalg.norm_
77         ↪ (direction),1e-9)
78         candidates=[default,start,center]
79         for f in [.35,.7,1.]:
80             mid=start+f*(center-start)
81             for offset in [-60.,0.,60.]:candidates.append(mid+offset*normal)
82         unique=[]
83         for q in candidates:
84             if any(math.dist(q,h['position'])<.1 for h in track.history):continue
85             if any(np.linalg.norm(q-p)<.1 for p in unique):continue
86             safe=reception_safe(track,q)
87             if not safe and self.exact_reception:
88                 safe=reception_violation(track.poly,track.history[0]['position'],q)[0]_
89                 ↪ <=-.01
90             if safe:unique.append(np.asarray(q))
91         samples=[a*.9+center*.1,b*.9+center*.1,center]
92         others=[t.center for c,t in self.tracks.items() if c!=track.channel and c not
93         ↪ in self.robot.cleared]
94         exit_point=min(others,key=lambda p:np.linalg.norm(p-center)) if self.exit_cost
95         ↪ and others else None
96         scores=[]
97         for q in unique:
98             values=[forecast(track,start,q,x,e,exit_point) for x in samples for e in
99             ↪ [-1.,0.,1.]]
100             score=(1-self.cost_risk)*np.mean(values)+self.cost_risk*np.max(values)
101             scores.append(float(score))
102         if not scores:return default
103         best=int(np.argmin(scores))
104         self.robot.record(dict(kind='cost_action',channel=track.channel,
105         ↪ candidates=[dict(point=q.tolist(),score_s=s) for q,s in
106         ↪ zip(unique,scores)],selected=best,
107         ↪ scoring='representative geometric scenarios; not a continuous minimax
108         ↪ certificate'))

```

```

98         return unique[best]
99
100     def solve_track(self, track):
101         # The first observation may replace Track by RefinedTrack; always refresh.
102         channel=track.channel
103         if not (self.cost_action or self.intermediate_known or
104                 ↪ self.intermediate_gain<1.):
105             return super().solve_track(track)
106         for _ in range(16):
107             self.robot.check_budget(); track=self.tracks[channel]
108             if track.radius<=CLEAR_RADIUS: return super().solve_track(track)
109             q=self.pick_point(track) if self.cost_action else
110             ↪ track.next_point(np.asarray(self.robot.position))
111             if self.observe(q, channel)['measure_result']=='no_signal':
112                 raise ArithmeticError('Certified reception point lost signal')
113             if self.intermediate_known or self.intermediate_gain<1.:
114                 old_gain, old_info=self.scan_gain, self.info_maxobs
115                 self.scan_gain=self.intermediate_gain
116                 if not self.intermediate_known: self.info_maxobs=0
117                 try: self.at_site_scan(self.robot.position)
118                     finally: self.scan_gain, self.info_maxobs=old_gain, old_info
119             raise RuntimeError('Cost-aware localization iteration limit')

```

文件 q3/v6/cover_milp.py

```

1  """Joint optional sensing-stop selection and open routing via bounded MILP."""
2  import numpy as np
3  from scipy.optimize import milp, Bounds, LinearConstraint
4  from scipy.sparse import lil_matrix
5  from service_routing import ServicePlanner
6
7
8  def solve_cover_route(points, mandatory, masks, query_cost, time_limit=1.):
9      n=len(points); edges=[(i,j) for i in range(n) for j in range(n) if i!=j]
10     ne=len(edges); idx={edge:k for k, edge in enumerate(edges)}
11     # x directed edges; f single-commodity connectivity flow; y visits; z scans.
12     yi=2*ne; zi=yi+n; size=zi+n
13     c=np.zeros(size); integrality=np.zeros(size, int)
14     lower=np.zeros(size); upper=np.ones(size); upper[ne:2*ne]=n-1
15     integrality[ne]=1; integrality[yi]=1
16     D=np.linalg.norm(np.asarray(points)[: ,None,:]-np.asarray(points)[None,: ,:], axis=2) ↪
17     ↪ /5
18     for k, (i,j) in enumerate(edges): c[k]=0. if j==0 else D[i,j]
19     c[zi]=query_cost
20     lower[yi]=1
21     for j in mandatory: lower[yi+j]=1
22     constraints=[]; lo=[]; hi=[]
23     def add(items, l, u): constraints.append(items); lo.append(l); hi.append(u)
24     for j in range(n):
25         add([(idx[i,j], 1.) for i in range(n) if i!=j]+[(yi+j, -1.)], 0., 0.)
26         add([(idx[j,k], 1.) for k in range(n) if k!=j]+[(yi+j, -1.)], 0., 0.)
27         add([(zi+j, 1.), (yi+j, -1.)], -np.inf, 0.)
28     for j in range(1, n):
29         add([(ne+idx[i,j], 1.) for i in range(n) if i!=j]+[(ne+idx[j,k], -1.) for k in
30                 ↪ range(n) if k!=j]+[(yi+j, -1.)], 0., 0.)
31     for k, (i,j) in enumerate(edges):
32         add([(ne+k, 1.), (k, -(n-1.))], -np.inf, 0.)
33         if j==0: upper[ne+k]=0.
34     for mask in np.unique(np.asarray(masks, dtype=bool).T, axis=0):
35         if not np.any(mask): return None
36         add([(zi+j, 1.) for j in np.flatnonzero(mask)], 1., np.inf)
37     A=lil_matrix((len(constraints), size))
38     for i, items in enumerate(constraints):
39         for j, value in items: A[i,j]+=value
40     result=milp(c, integrality=integrality, bounds=Bounds(lower, upper),
41                constraints=LinearConstraint(A.tocsc(), np.asarray(lo), np.asarray(hi)),
42                options={'time_limit':time_limit, 'mip_rel_gap':.01})
43     if result.x is None: return None
44     selected={j for j in range(n) if result.x[yi+j]>.5}; active={j for j in range(n) if
45     ↪ result.x[zi+j]>.5}
46     successors={i:j for k, (i,j) in enumerate(edges) if result.x[k]>.5}
47     route=[0]

```

```

45     for _ in range(n):
46         j=successors.get(route[-1])
47         if j is None: return None
48         if j==0: break
49         if j in route: return None
50         route.append(j)
51     if set(route)!=selected or not set(mandatory).issubset(selected): return None
52     if len(np.asarray(masks).T) and not
53     ↪ np.all(np.any(np.asarray(masks)[active],axis=0)): return None
54     cost=sum(D[a,b] for a,b in zip(route,route[1:]))+query_cost*len(active)
55     return dict(route=route[1:],active=active,cost=float(cost),gap=float(result.mip_gap)
56     ↪ p),status=int(result.status))
57
58 class MilpPlanner(ServicePlanner):
59     def __init__(self,robot,*,milp_seconds=1,milp_budget=16,**kwargs):
60         super().__init__(robot,**kwargs); self.milp_seconds=milp_seconds
61         self.milp_budget=milp_budget; self.milp_calls=0
62
63     def candidates(self):
64         baseline=super().candidates(); unknown=self.unknown()
65         if not baseline or not unknown or self.pending_probe is not None or
66         ↪ self.milp_calls>=self.milp_budget: return baseline
67         _,nodes,_=self.last_plan
68         targets=[n for n in nodes if n[0]=='target']
69         if not targets: return baseline
70         points=[np.asarray(self.robot.position)]+[n[2] for n in targets]
71         for q in [n[2] for n in nodes if n[0]=='scan']+[np.asarray(p) for p in
72         ↪ self.anchors[1:]]:
73             if not any(np.linalg.norm(q-p)<.1 for p in points): points.append(q)
74             missing=~self.coverage.covered[unknown[0]]
75             masks=np.array([self.coverage.mask(p)[missing] for p in points])
76             self.milp_calls+=1
77             solution=solve_cover_route(points,range(1,len(targets)+1),masks,6*len(unknown)
78             ↪ ,self.milp_seconds)
79             if solution is None or solution['cost']>=self.last_plan[0]/5-1e-6: return
80             ↪ baseline
81             self.robot.record(dict(kind='joint_cover_milp',predicted_cost_s=solution['cost']
82             ↪ ',incumbent_cost_s=self.last_plan[0]/5,
83             ↪ relative_solver_gap=solution['gap'],solver_status=solution['status'],nodes
84             ↪ =len(points),mandatory_targets=len(targets)))
85             j=0 if 0 in solution['active'] else solution['route'][0]
86             if j in solution['active']: return [('scan',-20000-j,points[j])]
87             if 1<=j<=len(targets): return [targets[j-1]]
88             return baseline

```

文件 q3/v6/exact_coverage.py

```

1  """Sufficient circle-arrangement certificate; coarse samples only guide routing."""
2  import math
3  import numpy as np
4  from legacy_strategy import Coverage
5
6
7  def arrangement_certificate(points, radius=999.99, arena_radius=1800.):
8      pts=np.unique(np.asarray(points,float).reshape(-1,2),axis=0)
9      if not len(pts): return False,np.array([arena_radius,0.])
10     angles=[0.]
11     for p in pts:
12         d=float(np.linalg.norm(p))
13         if d+arena_radius<=radius: return True,None
14         if d<1e-10: continue
15         cosine=(d*d+arena_radius**2-radius**2)/(2*d*arena_radius)
16         if -1<=cosine<=1:
17             a=math.atan2(p[1],p[0]); delta=math.acos(cosine)
18             angles += [(a-delta)%(2*math.pi),(a+delta)%(2*math.pi)]
19     angles=sorted(set(angles))
20     for i,a in enumerate(angles):
21         b=angles[i+1] if i+1<len(angles) else angles[0]+2*math.pi
22         q=arena_radius*np.array([math.cos((a+b)/2),math.sin((a+b)/2)])
23         if np.min(np.linalg.norm(pts-q,axis=1))>radius+1e-7: return False,q
24     # Any bounded uncovered interior component must have a vertex made by

```

```

25     # circle boundaries. Requiring a third disk's strict interior at all such
26     # crossings excludes it. Degenerate coincidences may be rejected safely.
27     for i in range(len(pts)):
28         for j in range(i):
29             v=pts[i]-pts[j];d=float(np.linalg.norm(v))
30             if d<1e-10 or d>2*radius:continue
31             mid=(pts[i]+pts[j])/2
32             normal=np.array([-v[1],v[0]])/d
33             half=math.sqrt(max(0.,radius**2-d*d/4))
34             for q in [mid+half*normal,mid-half*normal]:
35                 if np.linalg.norm(q)>=arena_radius-1e-7:continue
36                 distances=np.linalg.norm(pts-q,axis=1)
37                 distances[[i,j]]=float('inf')
38                 if np.min(distances)>=radius-1e-7:return False,q
39     return True,None
40
41
42 class ExactCoverage(Coverage):
43     def __init__(self):
44         super().__init__(level=6)
45         self.centers=self.centers[np.linalg.norm(self.centers,axis=1)<=1800]
46         self.covered=np.zeros((21,len(self.centers)),dtype=bool)
47         self.cache={}
48
49     def mask(self,point):
50         return np.sum((self.centers-np.asarray(point))**2,axis=1)<=999.99**2
51
52     def status(self,channel):
53         key=tuple(tuple(p) for p in self.samples[channel])
54         if key not in self.cache:self.cache[key]=arrangement_certificate(key)
55         return self.cache[key]
56
57     def absent(self,channel):return self.status(channel)[0]

```

文件 q3/v6/flexible_cover.py

```

1  """Continuous waypoint relocation with an explicit full-coverage constraint."""
2  import math
3  import numpy as np
4  from scipy.optimize import minimize
5  from scipy.spatial import ConvexHull
6  from joint_strategies import JointPlanner,open_route
7  from bearing_geometry import enclosing_circle,diameter
8
9
10 class FlexibleCoverPlanner(JointPlanner):
11     def __init__(self,robot,*,cover_sweeps=2,merge_stations=True,merge_target_scans=Fa
12         ↪ lse,**kwargs):
13         super().__init__(robot,family='adaptive',**kwargs)
14         self.cover_sweeps=cover_sweeps
15         self.merge_stations=merge_stations
16         self.merge_target_scans=merge_target_scans
17
18     def candidates(self):
19         targets=[]
20         for c,t in self.tracks.items():
21             if c in self.robot.cleared:continue
22             q=t.safe_clear_point(self.robot.position)
23             if q is None:
24                 q=t.next_point(np.asarray(self.robot.position)) if
25                 ↪ getattr(self,'cover_target_entry',False) else
26                 ↪ self.target_position(t)
27             targets.append(('target',c,np.asarray(q)))
28         unknown=self.unknown()
29         if not unknown:return targets
30         actual=self.coverage.covered[unknown[0]]
31         # Targets are mandatory route vertices but optional sensing sites.
32         points=[n[2].copy() for n in targets]+[np.array(p) for p in self.anchors]
33         n_target=len(targets)
34         active=np.ones(len(points),dtype=bool)
35         present=np.ones(len(points),dtype=bool)
36         masks=[self.coverage.mask(p) for p in points]

```

```

34 start=np.asarray(self.robot.position)
35 def route_order():
36     indices=[i for i in range(len(points)) if present[i]]
37     return [indices[j] for j in open_route(start,[points[i] for i in indices])]
38 def total_without(i):
39     total=actual.copy()
40     for j in range(len(points)):
41         if j!=i and active[j]:total|=masks[j]
42     return total
43 def exact_union_without(i):
44     sites=list(self.coverage.samples[unknown[0]]+[points[j] for j in
45         ↪ range(len(points)) if j!=i and active[j]])
46     return self.coverage.exact_status(sites)[0]<=999.98
47 def prune():
48     while True:
49         order=route_order();gains=[]
50         for k,i in enumerate(order):
51             if not active[i]:continue
52             gain=30*len(unknown)
53             if i>=n_target:
54                 a=start if k==0 else points[order[k-1]]
55                 gain+=np.linalg.norm(points[i]-a)
56                 if k+1<len(order):
57                     b=points[order[k+1]]
58                     gain+=np.linalg.norm(points[i]-b)-np.linalg.norm(a-b)
59             gains.append((gain,i))
60         removed=False
61         for gain,i in sorted(gains,reverse=True):
62             if np.all(total_without(i)) or (getattr(self,'hybrid_cover',False)
63                 ↪ and exact_union_without(i)):
64                 active[i]=False
65                 if i>=n_target:present[i]=False
66                 removed=True;break
67         if not removed:break
68 prune()
69 if self.merge_stations:
70     def full_cost(candidate_points,candidate_present,candidate_active):
71         indices=np.flatnonzero(candidate_present)
72         order=open_route(start,[candidate_points[i] for i in indices])
73         p=start;value=30*len(unknown)*int(np.count_nonzero(candidate_active))
74         for j in order:
75             q=candidate_points[indices[j]];value+=np.linalg.norm(q-p);p=q
76         return float(value)
77     for merge_iteration in range(4):
78         old_cost=full_cost(points,present,active);best=None
79         selected=[i for i in range(0 if self.merge_target_scans else
80             ↪ n_target,len(points)) if active[i]]
81         for at,i in enumerate(selected):
82             for j in selected[at:]:
83                 remaining=actual.copy()
84                 for k in range(len(points)):
85                     if active[k] and k not in [i,j]:remaining|=masks[k]
86                 needed=self.coverage.centers[~remaining]
87                 if not len(needed):continue
88                 h=self.coverage.half
89                 corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in
90                     ↪ [(-1,-1),(-1,1),(1,-1),(1,1)])])
91                 hull=corners[ConvexHull(corners).vertices]
92                 if diameter(hull)[0]>1999.9799:continue
93                 center,radius,_=enclosing_circle(hull)
94                 if radius>999.98999:continue
95                 new_points=points+[center]
96                 new_present=np.append(present,True);new_active=np.append(activ
97                     ↪ e,True)
98                 for removed in [i,j]:
99                     if removed>=n_target:new_present[removed]=False
100                     new_active[removed]=False
101                     value=full_cost(new_points,new_present,new_active)
102                     if value<old_cost-1e-6 and (best is None or value<best[0]):
103                         best=(value,i,j,center,new_points,new_present,new_active)
104             if best is None:break
105     value,i,j,center,points,present,active=best
106     masks.append(self.coverage.mask(center))
107     self.robot.record(dict(kind='merge_scans',point=center.tolist(),remove
108         ↪ d=[int(i),int(j)],

```

```

103         predicted_saving_s=(old_cost-value)/5,proof='MEC of all
        ↪ responsibility-cell corners <= 999.99 m'))
104     prune()
105     for _ in range(self.cover_sweeps):
106         order=route_order()
107         improvement=0.
108         for k,i in enumerate(order):
109             if i<n_target or not active[i]:continue
110             needed=self.coverage.centers[~total_without(i)]
111             if not len(needed):continue
112             h=self.coverage.half
113             corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in
        ↪ [(-1,-1),(-1,1),(1,-1),(1,1)])]
114             witnesses=corners[ConvexHull(corners).vertices]/1000
115             before=points[i]/1000
116             previous=(start if k==0 else points[order[k-1]])/1000
117             following=points[order[k+1]]/1000 if k+1<len(order) else None
118             def objective(q):
119                 value=np.sqrt(np.dot(q-previous,q-previous)+1e-16)
120                 if following is not
        ↪ None:value+=np.sqrt(np.dot(q-following,q-following)+1e-16)
121             return float(value)
122             def gradient(q):
123                 value=(q-previous)/max(np.linalg.norm(q-previous),1e-8)
124                 if following is not
        ↪ None:value+=(q-following)/max(np.linalg.norm(q-following),1e-8)
125             return value
126             cons={'type':'ineq','fun':lambda q:
        ↪ .999989999**2-np.sum((q-witnesses)**2,axis=1),
127                 'jac':lambda q:-2*(q-witnesses)}
128             sol=minimize(objective,before,jac=gradient,constraints=cons,method='SL_
        ↪ SQP',
129                          options={'ftol':1e-10,'maxiter':60})
130             if np.max(np.linalg.norm(witnesses-sol.x,axis=1))<=.99999 and
        ↪ objective(sol.x)<=objective(before)+1e-8:
131                 improvement+=objective(before)-objective(sol.x)
132                 points[i]=sol.x*1000;masks[i]=self.coverage.mask(points[i])
133             prune()
134             if improvement<1e-5:break
135             union=actual.copy()
136             for i in range(len(points)):
137                 if active[i]:union|=masks[i]
138             if not np.all(union) and not (getattr(self,'hybrid_cover',False) and
        ↪ exact_union_without(-1)):
139                 raise ArithmeticError('Relocated cover lost coverage')
140             scans=[('scan',-100-i,points[i]) for i in range(len(points)) if active[i]]
141             nodes=targets+scans
142             self.cover_targets=set()
143             return nodes

```

文件 q3/v6/joint_strategies.py

```

1  """Independent candidate families for P3; all maintain the same clear certificate."""
2  import math
3  import numpy as np
4  from scipy.spatial import ConvexHull
5  from legacy_strategy import Planner as BasePlanner, CLEAR_RADIUS, Track
6  from bearing_geometry import enclosing_circle, wedge, clip_halfplanes
7  from exact_coverage import ExactCoverage
8
9
10 def exclude_disk_hull(poly, center, radius=999.9):
11     """Convex outer hull of P outside a disk (uses P-boundary candidates).
12
13     A maximum of a linear functional over P minus an open disk is attained
14     at an outside vertex or an edge/circle crossing; the inner arc is concave.
15     Shrink the excluded radius by .1 m to protect floating-point intersections.
16     """
17     center=np.asarray(center);points=[]
18     for i,p in enumerate(poly):
19         q=poly[(i+1)%len(poly)];v=q-p;offset=p-center
20         if np.dot(offset,offset)>=radius**2:points.append(p)

```

```

21     a=float(np.dot(v,v));b=2*float(np.dot(v,offset));c=float(np.dot(offset,offset))
    ↪ -radius**2)
22     disc=b*b-4*a*c
23     if a>1e-18 and disc>=0:
24         for t in [(-b-math.sqrt(disc))/(2*a),(-b+math.sqrt(disc))/(2*a)]:
25             if 0<=t<=1:points.append(p+t*v)
26     if not points:raise ArithmeticError('Negative measurement contradicts region')
27     points=np.unique(np.round(np.asarray(points),9),axis=0)
28     if len(points)<3:return points
29     try:return points[ConvexHull(points).vertices]
30     except Exception:
31         low,high=points.min(axis=0)-1e-7,points.max(axis=0)+1e-7
32         return np.array([low,[high[0],low[1]],high,[low[0],high[1]]])
33
34
35 def exclude_union_hull(poly, centers, radius=999.8):
36     """Joint negative evidence, without convexifying between exclusions."""
37     if not centers:return poly
38     cs=np.unique(np.asarray(centers,float),axis=0)
39     edges=np.roll(poly,-1,axis=0)-poly
40     candidates=[p for p in poly]
41     for p,v in zip(poly,edges):
42         a=float(np.dot(v,v))
43         if a<1e-18:continue
44         for center in cs:
45             offset=p-center;b=2*float(np.dot(v,offset));c=float(np.dot(offset,offset))-
    ↪ radius**2)
46             disc=b*b-4*a*c
47             if disc<0:continue
48             for t in [(-b-math.sqrt(disc))/(2*a),(-b+math.sqrt(disc))/(2*a)]:
49                 if 0<=t<=1:candidates.append(p+t*v)
50     for i,c in enumerate(cs):
51         for d in cs[i:]:
52             delta=c-d;length=float(np.linalg.norm(delta))
53             if length<1e-9 or length>2*radius:continue
54             normal=np.array([-delta[1],delta[0]])/length
55             offset=math.sqrt(max(0.,radius**2-length**2/4))*normal
56             candidates += [(c+d)/2+offset,(c+d)/2-offset]
57     points=np.asarray(candidates)
58     outside=np.all(np.sum((points[:,None,:]-cs[None,:,:])**2,axis=2)>=radius**2-1e-4,a
    ↪ xis=1)
59     delta=points[:,None,:]-poly[None,:,:]
60     inside=np.all(edges[None,:,0]*delta[:,:,:1]-edges[None,:,1]*delta[:,:,:0]>=-1e-6*np.
    ↪ maximum(1.,np.linalg.norm(edges,axis=1)),axis=1)
61     points=np.unique(np.round(points[outside&inside],9),axis=0)
62     if not len(points):raise ArithmeticError('Joint negative evidence yields empty
    ↪ region')
63     if len(points)<3:return points
64     return points[ConvexHull(points).vertices]
65
66
67 def open_route(start, points, end=None, restarts=1):
68     """Nearest-neighbour initialization + open-path 2-opt, fixed start, free end."""
69     if not len(points):
70         return []
71     pts = np.vstack([start, points])
72     D = np.linalg.norm(pts[:, None] - pts[None, :], axis=2)
73     end_cost=np.zeros(len(pts)) if end is None else np.linalg.norm(pts-end,axis=1)
74     def length(route):return float(sum(D[a,b] for a,b in
    ↪ zip(route,route[1:]))+end_cost[route[-1]])
75     rng=np.random.default_rng(42)
76     best_route,best_length=None,float('inf')
77     for restart in range(restarts):
78         left = set(range(1, len(pts)))
79         route = [0]
80         if restart:
81             first=sorted(left,key=lambda j:D[0,j])[(restart-1)%len(left)]
82             route.append(first);left.remove(first)
83         while left:
84             q = min(left, key=lambda j: D[route[-1], j])
85             route.append(q);left.remove(q)
86         for _ in range(50):
87             gain, best = 1e-8, None
88             for i in range(1, len(route) - 1):
89                 for j in range(i + 1, len(route)):

```

```

90         old = D[route[i-1], route[i]]
91         new = D[route[i-1], route[j]]
92         if j + 1 < len(route):
93             old += D[route[j], route[j+1]]
94             new += D[route[i], route[j+1]]
95         else:
96             old += end_cost[route[j]]
97             new += end_cost[route[i]]
98         if old - new > gain:
99             gain, best = old-new, (i, j)
100     if best is not None:
101         i,j=best;route[i:j+1]=reversed(route[i:j+1]);continue
102     if restarts>1:
103         old_length=length(route);replacement=None
104         for i in range(1,len(route)):
105             reduced=route[:i]+route[i+1:]
106             for j in range(1,len(route)):
107                 trial=reduced[:j]+[route[i]]+reduced[j:]
108                 value=length(trial)
109                 if value<old_length-1e-7:
110                     old_length,replacement=value,trial
111             if replacement is not None:route=replacement;continue
112     break
113     value=length(route)
114     if value<best_length:best_length,best_route=value,route
115     return [i-1 for i in best_route[1:]]
116
117 class JointPlanner(BasePlanner):
118     def __init__(self, robot, *, family='joint', scan_gain=.05, info_distance=150.,
119                 initial_baseline=0., route_on=True, info_radius=60., info_maxobs=2,
120                 anchor_radius=1350., initial_force=True, initial_rotation=0.,
121                 ↪ negative_regions=False,
122                 cover_move_weight=1., grid_step=450., one_step=False, probe_length=0.,
123                 ↪ route_bias=0.,route_restarts=1,
124                 trial_radius=0.,trial_estimator='mec',recon_radius=0.,recon_count=3,
125                 info_range=float('inf'),rare_cover=False,exact_cover=False,smart_info ↪
126                 ↪ =False,rotate_cover=False,
127                 joint_negatives=False,shared_waypoint=False,explore_bonus=0.,expected ↪
128                 ↪ _count=13.,coverage_value=False,coverage_value_threshold_s=10.,mo ↪
129                 ↪ tion='side',first_fraction=1.,radius_coupling=False):
130         super().__init__(robot)
131         self.family = family
132         self.scan_gain = scan_gain
133         self.info_distance = info_distance
134         self.initial_baseline = initial_baseline
135         self.route_on = route_on
136         self.info_radius = info_radius
137         self.info_maxobs = info_maxobs
138         self.attempts = {}
139         self.anchors = [(0.,0.)]+[(anchor_radius*math.cos(i*math.pi/3),anchor_radius*m ↪
140         ↪ ath.sin(i*math.pi/3)) for i in range(6)]
141         self.initial_force,self.initial_rotation=initial_force,initial_rotation
142         self.negative_regions=negative_regions
143         self.cover_move_weight=cover_move_weight
144         self.grid_step=grid_step
145         self.static_cover=None
146         self.cover_targets=set()
147         self.one_step=one_step
148         self.probe_length,self.route_bias=probe_length,route_bias
149         self.route_restarts=route_restarts
150         self.trial_radius,self.trial_estimator=trial_radius,trial_estimator
151         self.failed_trials={}
152         self.recon_radius,self.recon_count=recon_radius,recon_count
153         self.info_range,self.rare_cover=info_range,rare_cover
154         self.exact_cover=exact_cover
155         self.smart_info=smart_info
156         self.rotate_cover=rotate_cover
157         self.rotation_cache=None
158         self.joint_negatives=joint_negatives
159         if joint_negatives:self.negative_regions=True
160         self.shared_waypoint=shared_waypoint
161         self.explore_bonus,self.expected_count=explore_bonus,expected_count
162         self.coverage_value,self.coverage_value_threshold_s=coverage_value,coverage_v ↪
163         ↪ lue_threshold_s

```

```

158     self.motion,self.first_fraction=motion,first_fraction
159     self.radius_coupling=radius_coupling
160     if radius_coupling:self.negative_regions=True
161     if exact_cover:self.coverage=ExactCoverage()
162     self.iterations = 0
163
164     def unknown(self):
165         if len(self.robot.discovered)==16:return []
166         return super().unknown()
167
168     def solve_track(self,track):
169         if self.motion=='midpoint':
170             for _ in range(16):
171                 self.robot.check_budget()
172                 if track.radius<=CLEAR_RADIUS:return super().solve_track(track)
173                 q=track.center.copy()
174                 if len(track.history)==1:
175                     s=np.asarray(track.history[0]['position'])
176                     q=s+self.first_fraction*(q-s)
177                     if any(math.dist(q,h['position'])<.1 for h in track.history):
178                         q=track.next_point(np.asarray(self.robot.position))
179                     if self.observe(q,track.channel)['measure_result']=='no_signal':
180                         raise ArithmeticError('Midpoint reception certificate failed')
181                     if self.one_step:return
182                 raise RuntimeError('Midpoint localization exhausted budget')
183         if CLEAR_RADIUS<track.radius<=self.trial_radius:
184             q=track.center.copy()
185             if self.trial_estimator=='wls':
186                 A=[];b=[]
187                 for h in track.history:
188                     if h['measure_result']!='direction':continue
189                     a=math.radians(h['svd_deg']);s=np.asarray(h['position']);w=max(10.
190                     ↪ ,np.linalg.norm(q-s))
191                     normal=np.array([-math.sin(a),math.cos(a)])/w
192                     A.append(normal);b.append(np.dot(normal,s))
193                     if len(A)>=2:q=np.linalg.lstsq(np.asarray(A),np.asarray(b),rcond=None)
194                     ↪ [0]
195                 previous=self.failed_trials.get(track.channel,[])
196                 if not any(math.dist(q,p)<2 for p in previous):
197                     self.robot.record(dict(kind='speculative_clear',channel=track.channel,
198                     ↪ point=q.tolist(),
199                     region_radius_m=track.radius,estimator=self.tri
200                     ↪ al_estimator))
201                 if self.robot.clear(tuple(q),track.channel):return
202                 self.failed_trials.setdefault(track.channel,[]).append(tuple(q))
203                 if self.one_step:return
204             if self.probe_length and len(track.history)==1 and
205             ↪ track.history[0]['measure_result']=='direction':
206                 first=track.history[0];s=np.asarray(first['position']);a=math.radians(firs
207                 ↪ t['svd_deg'])
208                 u=np.array([math.cos(a),math.sin(a)]);v=np.array([-math.sin(a),math.cos(a)
209                 ↪ ])
210                 length=self.probe_length
211                 forward=max(.35*length,length*length/1900+length*.018)
212                 lateral=math.sqrt(length*length-forward*forward)
213                 choices=[s+forward*u+lateral*v,s+forward*u-lateral*v]
214                 q=min(choices,key=lambda q:math.dist(q,self.robot.position))
215                 if self.observe(q,track.channel)['measure_result']=='no_signal':
216                     raise ArithmeticError('Q2 short-baseline reception certificate failed')
217                 if self.one_step:return
218             if not self.one_step or track.radius <= CLEAR_RADIUS:
219                 return super().solve_track(track)
220             q=track.next_point(np.asarray(self.robot.position))
221             if self.observe(q,track.channel)['measure_result']=='no_signal':
222                 raise ArithmeticError('Guaranteed next observation lost reception')
223
224     def observe(self, point, channel):
225         if not self.negative_regions:
226             result = super().observe(point, channel)
227         else:
228             point=tuple(float(v) for v in point)
229             result=self.robot.measure(point,channel)
230             if result['measure_result']=='no_signal':self.coverage.add(channel,point)
231             else:

```

```

225         track=self.tracks.setdefault(channel,Track(channel))
226         track.update(point,result)
227     if channel in self.tracks:
228         track=self.tracks[channel]
229         if self.radius_coupling:
230             for positive in track.history:
231                 s=np.asarray(positive['position'])
232                 for negative in self.coverage.samples[channel]:
233                     n=np.asarray(negative)
234                     normal=2*(n-s)
235                     offset=float(np.dot(n,n)-np.dot(s,s))+1e-6*np.linalg.norm(
↪ normal)
236                     track.poly=clip_halfplanes(track.poly,[normal],[offset])
237                 if not len(track.poly):raise ArithmeticError('Fixed-radius
↪ positive/negative conflict')
238         if self.joint_negatives:
239             track.poly=exclude_union_hull(track.poly,self.coverage.samples[cha
↪ nnel])
240         else:
241             for p in self.coverage.samples[channel]:
242                 track.poly=exclude_disk_hull(track.poly,p)
243             track.center,track.radius,_=enclosing_circle(track.poly)
244             self.robot.record(dict(kind='region',channel=channel,position=list(poi
↪ nt),
245                                   radius_m=track.radius,center=track.center.tolist(),vertices=track.
↪ poly.tolist(),
246                                   observations=len(track.history),negative_positions=self.coverage.s
↪ amples[channel]))
247     self.attempts[channel] = tuple(point)
248     return result
249
250 def at_site_scan(self, point, force=False):
251     """Unknown scans buy coverage; known scans buy intersecting bearings."""
252     unknown = self.unknown()
253     scan_unknown=force or self.coverage.gain(point,unknown)>=self.scan_gain
254     if not force and self.coverage_value and unknown and
↪ math.dist(point,self.robot.position)<1e-6:
255         scan_unknown=self.coverage_time_value(point,unknown)>self.coverage_value_t
↪ hreshold_s
256     channels = set(unknown if scan_unknown else [])
257     for c, track in self.tracks.items():
258         if c in self.robot.cleared or track.radius <= self.info_radius or
↪ len(track.history) >= self.info_maxobs:
259             continue
260         if math.dist(point,track.center)>self.info_range:continue
261         if c in self.attempts and math.dist(point,self.attempts[c]) <
↪ self.info_distance:
262             continue
263         if self.smart_info and len(track.history)>=2 and
↪ math.dist(point,track.center)>100:
264             a=math.degrees(math.atan2(track.center[1]-point[1],track.center[0]-poi
↪ nt[0]))
265             radii=[]
266             for reading in [a-1.,a,a+1.]:
267                 A,b=wedge(point,reading,1.0051)
268                 posterior=clip_halfplanes(track.poly,A,b)
269                 if len(posterior):radii.append(enclosing_circle(posterior)[1])
270                 if not radii or not (max(radii)<=19.5 or
↪ np.mean(radii)<=.4*track.radius):continue
271         if min(math.dist(point, h['position']) for h in track.history) >=
↪ self.info_distance:
272             # No-signal is allowed for batch measurements; never corrupt P.
273             channels.add(c)
274     if channels:
275         self.robot.scanned_sites += 1
276     for c in sorted(channels, key=lambda c: (c != self.robot.channel, c)):
277         self.observe(point, c)
278
279 def coverage_time_value(self,point,unknown):
280     if self.coverage.gain(point,unknown)<.005:return -1e9
281     def estimated_cost():
282         nodes=self.candidates()
283         order=open_route(self.robot.position,[n[2] for n in nodes])
284         p=np.asarray(self.robot.position);distance=0.

```

```

285         for i in order: distance+=np.linalg.norm(nodes[i][2]-p);p=nodes[i][2]
286         return distance/5+6*len(unknown)*sum(n[0]=='scan' for n in nodes)
287     before=estimated_cost()
288     saved=self.coverage.covered[unknown].copy()
289     try:
290         self.coverage.covered[unknown]=self.coverage.mask(point)
291         after=estimated_cost()
292     finally: self.coverage.covered[unknown]=saved
293     return float(before-after-6*len(unknown))
294
295 def candidates(self):
296     pending = [t for c,t in self.tracks.items() if c not in self.robot.cleared]
297     nodes = []
298     for t in pending:
299         q = t.safe_clear_point(self.robot.position)
300         q = t.center if q is None else q
301         if self.route_bias and len(t.history)==1:
302             q=q+self.route_bias*(q-np.asarray(t.history[0]['position']))
303         nodes.append(('target', t.channel, np.asarray(q)))
304     unknown = self.unknown()
305     if not unknown:
306         return nodes
307     if not nodes and self.exact_cover and
308     ⇨ np.all(self.coverage.covered[unknown[0]]):
309         return [('scan',-2,self.coverage.status(unknown[0])[1])]
310     if self.family == 'adaptive':
311         return self.adaptive_cover(nodes,unknown)
312     if self.rotate_cover:
313         return self.rotated_cover(nodes,unknown)
314     # Keep a conservative fallback cover from original anchors. We may
315     # anticipate target visits for route scoring, but stopping uses real scans.
316     covered = self.coverage.covered[unknown[0]].copy()
317     if self.family != 'anchors':
318         for _,_, q in nodes:
319             covered |= self.coverage.mask(q)
320     anchors = [(i,p,self.coverage.mask(p)) for i,p in enumerate(self.anchors)]
321     needed = []
322     while not np.all(covered):
323         if not anchors:
324             raise RuntimeError('Search fallback cover exhausted')
325         # Greedy set cover only selects waypoints; it does not certify absence.
326         a = max(anchors, key=lambda a: int(np.count_nonzero(a[2] & ~covered)))
327         gain = np.count_nonzero(a[2] & ~covered)
328         if not gain:
329             raise RuntimeError('No progress in waypoint cover')
330         needed.append(('scan', a[0], np.asarray(a[1])))
331         covered |= a[2]
332         anchors.remove(a)
333     return nodes + needed
334
335 def rotated_cover(self,nodes,unknown):
336     if self.rotation_cache is None:
337         self.rotation_cache=[]
338         for radius in [1200.,1350.,1550.]:
339             for rotation in range(0,60,10):
340                 pts=[np.zeros(2)]+[radius*np.array([math.cos(math.radians(rotation)
341                 ⇨ +i*60)),math.sin(math.radians(rotation+i*60))]) for i in
342                 ⇨ range(6)]
343                 self.rotation_cache.append([(i,p,self.coverage.mask(p)) for i,p in
344                 ⇨ enumerate(pts)])
345     actual=self.coverage.covered[unknown[0]].copy()
346     for _,_,q in nodes:actual|=self.coverage.mask(q)
347     best=(float('inf'),None)
348     for choice in self.rotation_cache:
349         covered=actual.copy();remaining=choice.copy();needed=[]
350         while not np.all(covered):
351             a=max(remaining,key=lambda a:np.count_nonzero(a[2]&~covered))
352             if not np.any(a[2]&~covered):break
353             needed.append(('scan',a[0],a[1]));covered|=a[2];remaining.remove(a)
354         if not np.all(covered):continue
355         candidate=nodes+needed
356         order=open_route(self.robot.position,[n[2] for n in candidate])
357         p=np.asarray(self.robot.position);cost=0.
358         for i in order:cost+=np.linalg.norm(candidate[i][2]-p);p=candidate[i][2]
359         cost+=30*len(unknown)*len(needed)

```

```

356         if cost<best[0]:best=(cost,candidate)
357     if best[1] is None:raise RuntimeError('No rotated cover found')
358     return best[1]
359
360 def certificate(self):
361     result=super().certificate()
362     result['coverage_method']='circle_arrangement_sufficient' if self.exact_cover
363     ↪ else 'whole_square_cover'
364     if self.exact_cover:result['grid_role']='routing_heuristic_only_not_completion'
365     ↪ '_proof'
366     return result
367
368 def adaptive_cover(self,nodes,unknown):
369     def bits(mask):
370         return
371         ↪ int.from_bytes(np.packbits(mask,bitorder='little').tobytes(),'little')
372     if self.static_cover is None:
373         pts=[np.array(p) for p in self.anchors]
374         pts += [np.array([x,y]) for x in np.arange(-1350,1351,self.grid_step)
375                 for y in np.arange(-1350,1351,self.grid_step) if
376                 ↪ math.hypot(x,y)<=1750]
377         self.static_cover=[(q,bits(self.coverage.mask(q))) for q in pts]
378     covered=bits(self.coverage.covered[unknown[0]])
379     full=(1<len(self.coverage.centers))-1
380     p=np.asarray(self.robot.position)
381     route=open_route(p,[n[2] for n in nodes])
382     route_points=[p]+[nodes[i][2] for i in route]
383     # Marginal route insertion distance, not straight distance from dog.
384     def detour(q):
385         best=float(np.linalg.norm(route_points[-1]-q))
386         for a,b in zip(route_points,route_points[1:]):
387             best=min(best,float(np.linalg.norm(a-q)+np.linalg.norm(q-b)-np.linalg.
388             ↪ norm(a-b)))
389         return max(0.,best)
390     candidates=[('scan',-1,p,bits(self.coverage.mask(p)),0.))
391     candidates += [(kind,c,q,bits(self.coverage.mask(q)),0.) for kind,c,q in nodes]
392     candidates += [('scan',i,q,mask,detour(q)) for i,(q,mask) in
393     ↪ enumerate(self.static_cover)]
394     needed=[];self.cover_targets=set();selected=[];actual=covered
395     groups=[]
396     if self.rare_cover:
397         arrays=np.array([np.unpackbits(np.frombuffer(a[3].to_bytes((len(self.cover
398         ↪ age.centers)+7)//8,'little'),dtype=np.uint8,bitorder='little')[:len(s
399         ↪ elf.coverage.centers)] for a in candidates])
400         frequency=np.maximum(arrays.sum(axis=0),1)
401         weights=np.ceil(100/frequency).astype(int)
402         groups=[(w,bits(weights==w)) for w in np.unique(weights)]
403     while covered != full:
404         missing=full^covered
405         def score(a):
406             gain=sum(w*(a[3]&missing&mask).bit_count() for w,mask in groups) if
407             ↪ groups else (a[3]&missing).bit_count()
408             return gain/(6*len(unknown)+self.cover_move_weight*a[4]/5)
409         best=max(candidates,key=score)
410         if not best[3]&missing:raise RuntimeError('Adaptive cover cannot close
411         ↪ remaining cells')
412         covered |= best[3]
413         selected.append(best)
414         candidates.remove(best)
415     if self.rare_cover:
416         for a in list(reversed(selected)):
417             union=actual
418             for other in selected:
419                 if other is not a:union|=other[3]
420                 if union==full:selected.remove(a)
421     for kind,c,q,mask,d in selected:
422         if kind=='target':self.cover_targets.add(c)
423         else:needed.append((kind,c,q))
424     return nodes+needed
425
426 def run(self):
427     self.at_site_scan((0.,0.), force=True)
428     if self.initial_baseline:
429         pending = list(self.tracks.values())

```

```

420         angle = (math.atan2(pending[0].center[1], pending[0].center[0]) if pending
↪         else 0.)+math.radians(self.initial_rotation)
421         q = np.array([math.cos(angle),math.sin(angle)]) * self.initial_baseline
422         self.at_site_scan(q, force=self.initial_force)
423     if self.recon_radius:
424         for i in range(self.recon_count):
425             a=2*math.pi*i/self.recon_count
426             q=self.recon_radius*np.array([math.cos(a),math.sin(a)])
427             self.at_site_scan(q,force=True)
428     # Keep all anchors available; real coverage masks prevent redundant work.
429     while True:
430         self.iterations += 1
431         self.robot.check_budget()
432         if self.iterations > 180:
433             raise RuntimeError('Joint planner iteration cap')
434         pending = [t for c,t in self.tracks.items() if c not in self.robot.cleared]
435         if not pending and (len(self.robot.cleared) == 16 or not self.unknown()):
436             self.stop_reason = 'cleared_upper_bound_16' if
↪             len(self.robot.cleared)==16 else 'continuous_coverage_certificate'
437             self.robot.record(dict(kind='completion_certificate',
↪             **self.certificate()))
438             return self.stop_reason
439         if self.family=='adaptive':
440             gain=self.scan_gain;self.scan_gain=2.
441             self.at_site_scan(self.robot.position)
442             self.scan_gain=gain
443         else:self.at_site_scan(self.robot.position)
444         nodes = self.candidates()
445         if not nodes:
446             continue
447         order = open_route(self.robot.position, [n[2] for n in
↪         nodes],restarts=self.route_restarts) if self.route_on else
↪         sorted(range(len(nodes)),key=lambda i:
↪         math.dist(nodes[i][2],self.robot.position))
448         if self.explore_bonus and nodes[order[0]][0]=='target':
449             def tour_length(start,indices):
450                 total=0.;p=np.asarray(start)
451                 for i in indices:total+=np.linalg.norm(nodes[i][2]-p);p=nodes[i][2]
452                 return float(total)
453             normal=tour_length(self.robot.position,order)
454             best=(float('inf'),None)
455             for i,n in enumerate(nodes):
456                 if n[0]!='scan':continue
457                 others=[j for j in range(len(nodes)) if j!=i]
458                 rest=open_route(n[2],[nodes[j][2] for j in others])
459                 cost=math.dist(self.robot.position,n[2])+tour_length(n[2],[others[j]
↪                 j] for j in rest])
460                 if cost<best[0]:best=(cost,i)
461             allowance=self.explore_bonus*max(1.,self.expected_count-len(self.robot)
↪             .discovered))
462             if best[1] is not None and best[0]-normal<=allowance:
463                 order=[best[1]]+[i for i in order if i!=best[1]]
464             kind, c, point = nodes[order[0]]
465             if kind == 'scan':
466                 self.at_site_scan(point, force=True)
467             else:
468                 track = self.tracks[c]
469                 if self.shared_waypoint and self.family=='adaptive' and c in
↪                 self.cover_targets and track.radius>CLEAR_RADIUS:
470                     self.at_site_scan(point,force=True)
471                     if math.dist(self.attempts.get(c,(1e9,1e9)),point)>1e-5:
472                         self.observe(point,c)
473                     continue
474                 self.solve_track(track)
475                 if self.family=='adaptive' and c in self.cover_targets and
↪                 len(self.robot.cleared)<16:
476                     self.at_site_scan(self.robot.position,force=True)
477
478
479     class SweepPlanner(JointPlanner):
480         """Finish targets near the next coverage leg; defer distant returns."""
481         def __init__(self, robot, **kwargs):
482             super().__init__(robot, family='anchors', **kwargs)
483
484         def candidates(self):

```

```

485         nodes = super().candidates()
486         scans = [n for n in nodes if n[0]=='scan']
487         if not scans:
488             return nodes
489         route = open_route(self.robot.position,[n[2] for n in scans])
490         next_scan = scans[route[0]]
491         p,q = np.asarray(self.robot.position),next_scan[2]
492         targets=[]
493         for n in nodes:
494             if n[0]!='target':continue
495             detour = np.linalg.norm(n[2]-p)+np.linalg.norm(n[2]-q)-np.linalg.norm(q-p)
496             if detour <= 350:
497                 targets.append(n)
498         return targets + [next_scan]
499
500
501 class SectorPlanner(JointPlanner):
502     """One angular sweep, local open routes with the next station as endpoint."""
503     def __init__(self,robot,*,sector_rotation=0.,sector_width=60.,preclear=False,**kwargs):
504         ↪ rgs):
505         super().__init__(robot,**kwargs)
506         self.sector_rotation=math.radians(sector_rotation)
507         self.sector_width=sector_width
508         self.preclear=preclear
509
510     def run(self):
511         self.at_site_scan((0.,0.),force=True)
512         radius=math.hypot(*self.anchors[1])
513         sites=[radius*np.array([math.cos(self.sector_rotation+i*math.pi/3),
514                                ↪ math.sin(self.sector_rotation+i*math.pi/3)]) for i in
515                                ↪ range(6)]
516         for sector,site in enumerate(sites):
517             if not self.unknown():break
518             if self.preclear:
519                 for _ in range(16):
520                     p=np.asarray(self.robot.position)
521                     good=[t for c,t in self.tracks.items() if c not in
522                            ↪ self.robot.cleared and
523                            ↪ np.linalg.norm(t.center-p)+np.linalg.norm(t.center-site)-np.
524                            ↪ ↪ linalg.norm(site-p)<=100]
525                     if not good:break
526                     self.solve_track(min(good,key=lambda t:np.linalg.norm(t.center-p)))
527             self.at_site_scan(site,force=True)
528             while True:
529                 self.robot.check_budget()
530                 pending=[]
531                 for c,t in self.tracks.items():
532                     if c in self.robot.cleared:continue
533                     angle=(math.degrees(math.atan2(t.center[1],t.center[0])-self.secto
534                            ↪ ↪ r_rotation)+30)%360
535                     if angle <= (sector+1)*60 or sector==5:
536                         pending.append(t)
537                 if not pending:break
538                 end=sites[sector+1] if sector<5 and self.unknown() else None
539                 route=open_route(self.robot.position,[t.center for t in
540                            ↪ ↪ pending],end=end)
541                 self.solve_track(pending[route[0]])
542                 # Only measurements of uncertain known targets, no blind scans.
543                 gain=self.scan_gain;self.scan_gain=2.
544                 self.at_site_scan(self.robot.position)
545                 self.scan_gain=gain
546             while True:
547                 pending=[t for c,t in self.tracks.items() if c not in self.robot.cleared]
548                 if not pending:break
549                 route=open_route(self.robot.position,[t.center for t in pending])
550                 self.solve_track(pending[route[0]])
551             if self.unknown():
552                 raise RuntimeError('Sector sweep did not certify full coverage')
553             self.stop_reason='cleared_upper_bound_16' if len(self.robot.cleared)==16 else
554                 ↪ 'continuous_coverage_certificate'
555             self.robot.record(dict(kind='completion_certificate',**self.certificate()))
556             return self.stop_reason
557
558
559 class OrderedPlanner(JointPlanner):

```

```

553 """Insert target actions into a monotone station tour; no per-sector lock."""
554 def __init__(self, robot, *, clockwise=False, **kwargs):
555     super().__init__(robot, **kwargs)
556     self.remaining=list(self.anchors[1:])
557     if clockwise: self.remaining=list(reversed(self.remaining))
558
559 def candidates(self):
560     targets=[]
561     for c,t in self.tracks.items():
562         if c in self.robot.cleared:continue
563         q=t.safe_clear_point(self.robot.position)
564         q=t.center if q is None else q
565         targets.append(('target',c,np.asarray(q)))
566     unknown=self.unknown()
567     remaining=self.remaining.copy() if unknown else []
568     if unknown:
569         actual=self.coverage.covered[unknown[0]]
570         for p in list(reversed(remaining)):
571             total=actual.copy()
572             for q in remaining:
573                 if q is not p:total|=self.coverage.mask(q)
574                 if np.all(total):remaining.remove(p)
575     route=[('scan',i,np.asarray(p)) for i,p in enumerate(remaining)]
576     start=('start',-1,np.asarray(self.robot.position))
577     def distance(r):
578         p=start[2];value=0.
579         for n in r:value+=np.linalg.norm(n[2]-p);p=n[2]
580         return float(value)
581     while targets:
582         best=(float('inf'),None,None)
583         base=distance(route)
584         for i,t in enumerate(targets):
585             for j in range(len(route)+1):
586                 cost=distance(route[:j]+[t]+route[j:])-base
587                 if cost<best[0]:best=(cost,i,j)
588         _,i,j=best;route.insert(j,targets.pop(i))
589     for _ in range(8):
590         best=distance(route);replacement=None
591         for i,n in enumerate(route):
592             if n[0]!='target':continue
593             reduced=route[:i]+route[i+1:]
594             for j in range(len(route)):
595                 trial=reduced[:j]+[n]+reduced[j:];cost=distance(trial)
596                 if cost<best-1e-6:best,replacement=cost,trial
597             if replacement is None:break
598         route=replacement
599     # Returning just the first action prevents the base free-order solver
600     # from undoing the station precedence constraint.
601     return route[1:]
602
603 def at_site_scan(self,point,force=False):
604     super().at_site_scan(point,force=force)
605     if force:
606         self.remaining=[p for p in self.remaining if math.dist(p,point)>1e-6]
607
608
609 class StickyPlanner(JointPlanner):
610     """Commit to a shared coverage plan instead of repeatedly buying overlap."""
611     def __init__(self, robot, **kwargs):
612         super().__init__(robot, family='adaptive', **kwargs)
613         self.committed_points=None
614         self.committed_targets=set()
615
616     def candidates(self):
617         nodes=[]
618         for c,t in self.tracks.items():
619             if c in self.robot.cleared:continue
620             q=t.safe_clear_point(self.robot.position)
621             q=t.center if q is None else q
622             nodes.append(('target',c,np.asarray(q)))
623         unknown=self.unknown()
624         if not unknown:return nodes
625         if self.committed_points is None or (not self.committed_points and not
        ↪ self.committed_targets):
626             plan=self.adaptive_cover(nodes,unknown)

```

```

627         self.committed_points=[n for n in plan if n[0]=='scan']
628         self.committed_targets=self.cover_targets.copy()
629         self.cover_targets=self.committed_targets.copy()
630         return nodes+self.committed_points
631
632     def at_site_scan(self,point,force=False):
633         super().at_site_scan(point,force=force)
634         if force and self.committed_points is not None:
635             self.committed_points=[n for n in self.committed_points if
636                                     ↪ math.dist(n[2],point)>1e-5]
637             self.committed_targets-=self.robot.cleared
638
639 class TargetFirstPlanner(JointPlanner):
640     """Clear known targets as a routed group, ending toward the next scan."""
641     def candidates(self):
642         nodes=super().candidates()
643         targets=[n for n in nodes if n[0]=='target']
644         scans=[n for n in nodes if n[0]=='scan']
645         if not targets:return nodes
646         if not scans:
647             order=open_route(self.robot.position,[n[2] for n in
648                               ↪ targets],restarts=self.route_restarts)
649             return [targets[order[0]]]
650         best=(float('inf'),None)
651         for end in scans:
652             order=open_route(self.robot.position,[n[2] for n in
653                               ↪ targets],end=end[2],restarts=self.route_restarts)
654             p=np.asarray(self.robot.position);cost=0.
655             for i in order:cost+=np.linalg.norm(targets[i][2]-p);p=targets[i][2]
656             cost+=np.linalg.norm(end[2]-p)
657             remainder=[n for n in scans if n is not end]
658             tour=open_route(end[2],[n[2] for n in remainder]);p=end[2]
659             for j in tour:cost+=np.linalg.norm(remainder[j][2]-p);p=remainder[j][2]
660             if cost<best[0]:best=(cost,targets[order[0]])
661         return [best[1]]
662
663 class PathCoverPlanner(JointPlanner):
664     """Choose sensing stops on route segments; update insertion costs jointly."""
665     def __init__(self,robot,*,path_spacing=200.,**kwargs):
666         super().__init__(robot,family='adaptive',**kwargs)
667         self.path_spacing=path_spacing
668         self.path_static=None
669
670     def adaptive_cover(self,nodes,unknown):
671         def bits(mask):return
672             ↪ int.from_bytes(np.packbits(mask,bitorder='little').tobytes(),'little')
673         if self.path_static is None:
674             pts=[np.asarray(p) for p in self.anchors]
675             pts += [np.array([x,y]) for x in np.arange(-1500,1501,self.grid_step) for y
676                    ↪ in np.arange(-1500,1501,self.grid_step) if math.hypot(x,y)<1750]
677             self.path_static=[(q,bits(self.coverage.mask(q))) for q in pts]
678         covered=bits(self.coverage.covered[unknown[0]])
679         full=(1<<len(self.coverage.centers))-1
680         selected=[];self.cover_targets=set()
681         p=np.asarray(self.robot.position)
682         while covered!=full:
683             future=nodes+[( 'scan',-100-i,q) for i,(q,mask) in enumerate(selected)]
684             order=open_route(p,[n[2] for n in future])
685             route=[p]+[future[i][2] for i in order]
686             candidates=self.path_static.copy()
687             for a,b in zip(route,route[1:]):
688                 count=max(1,int(math.ceil(np.linalg.norm(b-a)/self.path_spacing)))
689                 for t in np.linspace(0.,1.,count+1):
690                     q=a+t*(b-a);candidates.append((q,bits(self.coverage.mask(q))))
691             if len(route)==1:candidates.append((p,bits(self.coverage.mask(p))))
692         def detour(q):
693             best=float(np.linalg.norm(q-route[-1]))
694             for a,b in zip(route,route[1:]):
695                 best=min(best,float(np.linalg.norm(a-q)+np.linalg.norm(b-q)-np.linalg
696                               ↪ alg.norm(a-b)))
697             return max(0.,best)
698         missing=full^covered

```

```

695         best=max(candidates,key=lambda a:(a[1]&missing).bit_count()/(6*len(unknown_
        ↪ )+self.cover_move_weight*detour(a[0])/5))
696         if not best[1]&missing:raise RuntimeError('Path cover cannot progress')
697         covered|=best[1];selected.append(best)
698     # Reverse-delete coverage redundancy before routing the chosen stops.
699     for a in list(reversed(selected)):
700         total=bits(self.coverage.covered[unknown[0]])
701         for other in selected:
702             if other is not a:total|=other[1]
703             if total==full:selected=[x for x in selected if x is not a]
704         return nodes+[('scan',-100-i,q) for i,(q,mask) in enumerate(selected)]
705
706     def candidates(self):
707         nodes=super().candidates()
708         if not nodes:return nodes
709         order=open_route(self.robot.position,[n[2] for n in nodes])
710         first=nodes[order[0]]
711         if first[0]=='target':
712             matching=[n for n in nodes if n[0]=='scan' and
713                       ↪ math.dist(n[2],first[2])<1e-5]
714             if matching:return [matching[0]]
715         return nodes

```

文件 q3/v6/legacy_strategy.py

```

1  """P3 M01/A01 v1: bounded-bearing regions, local routing, certified search.
2
3  No simulator connection at import; coordinates metres, angles degrees.
4  Heuristic efficiency; safety checks use outer regions, not point estimates.
5  """
6  import math
7  import numpy as np
8  from bearing_geometry import (outer_disk, disk_halfplanes, wedge,
9                               clip_halfplanes, enclosing_circle, diameter)
10
11  VERSION = 'p3-region-routing-v2'
12  EPS = 1.0051 # bounded error plus 0.005 degree rounding and numerical margin
13  CLEAR_RADIUS = 19.5
14  SIDES = 180
15
16
17  def scan_sites():
18      return [(0., 0.)] + [(1350 * math.cos(i * math.pi / 3),
19                          1350 * math.sin(i * math.pi / 3)) for i in range(6)]
20
21
22  class Coverage:
23      """Whole-square coverage certificate of the continuous 1800 m arena.
24
25      Keep EVERY square intersecting the arena, including boundary squares.
26      no_signal implies exclusion of the closed 1000 m disk for P3 only.
27      Cover a square only if its farthest corner is within 999.99 m.
28      """
29      def __init__(self, level=7):
30          self.level = level
31          self.half = 1800 / 2 ** level
32          axis = np.linspace(-1800 + self.half, 1800 - self.half, 2 ** level)
33          x, y = np.meshgrid(axis, axis)
34          centers = np.column_stack((x.ravel(), y.ravel()))
35          nearest = np.maximum(np.abs(centers) - self.half, 0)
36          self.centers = centers[np.sum(nearest ** 2, axis=1) <= 1800 ** 2 + 1e-6]
37          self.covered = np.zeros((21, len(self.centers)), dtype=bool)
38          self.samples = {c: [] for c in range(1, 21)}
39
40      def mask(self, point):
41          far = np.abs(self.centers - np.asarray(point)) + self.half
42          return np.sum(far ** 2, axis=1) <= 999.99 ** 2
43
44      def add(self, channel, point):
45          self.covered[channel] |= self.mask(point)
46          self.samples[channel].append([float(v) for v in point])
47

```

```

48     def absent(self, channel):
49         return bool(np.all(self.covered[channel]))
50
51     def gain(self, point, channels):
52         if not channels:
53             return 0.
54         return float(np.mean((~self.covered[channels]) & self.mask(point)))
55
56
57 class Track:
58     def __init__(self, channel):
59         self.channel = channel
60         self.poly = outer_disk((0, 0), 1800, SIDES)
61         self.history = []
62         self.center = None
63         self.radius = float('inf')
64
65     def update(self, point, observation):
66         kind = observation['measure_result']
67         if kind == 'no_signal':
68             return # Disk exclusion is nonconvex; do not cut it incorrectly.
69         if kind not in ('near', 'direction'):
70             raise ValueError(f'Unknown measure_result: {kind}')
71         poly = self.poly
72         if kind == 'direction':
73             A, b = wedge(point, float(observation['svd_deg']), EPS)
74             poly = clip_halfplanes(poly, A, b)
75         A, b = disk_halfplanes(point, 5.000001 if kind == 'near' else 1500.000001,
76             ↪ SIDES)
77         poly = clip_halfplanes(poly, A, b)
78         if not len(poly):
79             raise ArithmeticError(f'频道 {self.channel} 观测区域为空; 保留日志后退出,
80             ↪ 不能据此清除')
81         self.poly = poly
82         self.center, self.radius, _ = enclosing_circle(poly)
83         self.history.append(dict(position=[float(v) for v in point], **observation))
84
85     def safe_clear_point(self, position):
86         """Nearest safe point along center->robot segment (not global projection)."""
87         if self.radius > CLEAR_RADIUS:
88             return None
89         c, p = self.center, np.asarray(position, float)
90         if np.max(np.linalg.norm(self.poly - p, axis=1)) <= CLEAR_RADIUS:
91             return p.copy()
92         low, high = 0., 1.
93         for _ in range(45):
94             t = (low + high) / 2
95             q = c + t * (p - c)
96             if np.max(np.linalg.norm(self.poly - q, axis=1)) <= CLEAR_RADIUS:
97                 low = t
98             else:
99                 high = t
100         return c + low * (p - c)
101
102     def next_point(self, position):
103         """Move toward the region with a bounded cross-bearing offset.
104
105         If every possible source is <= 1000 m away, reception is guaranteed.
106         Also recognize Q2's first-observation safe lens, valid even if a
107         source's first distance was > 1000 m. The lens is a sufficient test.
108         """
109         c, r = self.center, self.radius
110         if r <= 120:
111             candidates = [c]
112         else:
113             _, (i, j) = diameter(self.poly)
114             d = self.poly[j] - self.poly[i]
115             normal = np.array([-d[1], d[0]]) / max(np.linalg.norm(d), 1e-12)
116             offset = min(150., r * .25)
117             candidates = [c + offset * normal, c - offset * normal]
118         first = self.history[0]
119         s = np.asarray(first['position'])
120         safe = []
121         for q in candidates:
122             all_close = np.max(np.linalg.norm(self.poly - q, axis=1)) <= 999.9

```

```

121         in_lens = False
122         if first['measure_result'] == 'direction':
123             a = math.radians(float(first['svd_deg']))
124             u, v = np.array([math.cos(a), math.sin(a)]), np.array([-math.sin(a),
125                             ↪ math.cos(a)])
126             ab = np.array([np.dot(q - s, u), np.dot(q - s, v)])
127             ep = math.radians(EPS)
128             length2 = float(np.dot(ab, ab))
129             in_lens = (length2 <= 1000 ** 2 - 1e-3 and
130                       ↪ length2 <= 2000 * (ab[0] * math.cos(ep) - abs(ab[1]) *
131                               ↪ math.sin(ep)) - 1e-3)
132         if all_close or in_lens:
133             safe.append(q)
134         # Region fits a disk of radius < 1000 after one direction report.
135         if not safe:
136             if r > 999.9:
137                 raise ArithmeticError('未找到可证明接收的下一观测点')
138             safe = [c]
139             q = min(safe, key=lambda p: float(np.linalg.norm(p - position)))
140             # Same-coordinate noise is fixed: never repeat an uninformative point.
141             if any(math.dist(q, h['position']) < .1 for h in self.history):
142                 _, (i, j) = diameter(self.poly)
143                 d = self.poly[j] - self.poly[i]
144                 normal = np.array([-d[1], d[0]]) / max(np.linalg.norm(d), 1e-12)
145                 alternates = [c + normal * 40, c - normal * 40]
146                 safe = [p for p in alternates if np.max(np.linalg.norm(self.poly - p,
147                             ↪ axis=1)) <= 999.9]
148             if not safe:
149                 raise ArithmeticError('重复观测点且没有安全侧移点')
150             q = min(safe, key=lambda p: float(np.linalg.norm(p - position)))
151         return np.asarray(q)
152
153 class Planner:
154     def __init__(self, robot, *, opportunistic=False, nearest_order=True):
155         self.robot = robot
156         self.coverage = Coverage()
157         self.tracks = {}
158         self.anchors = scan_sites()
159         self.opportunistic = opportunistic
160         self.nearest_order = nearest_order
161         self.stop_reason = None
162
163     def unknown(self):
164         return [c for c in range(1, 21) if c not in self.robot.discovered
165                 and not self.coverage.absent(c)]
166
167     def observe(self, point, channel):
168         p = tuple(float(v) for v in point)
169         observation = self.robot.measure(p, channel)
170         if observation['measure_result'] == 'no_signal':
171             self.coverage.add(channel, p)
172         else:
173             track = self.tracks.setdefault(channel, Track(channel))
174             track.update(p, observation)
175             self.robot.record(dict(kind='region', channel=channel, position=list(p),
176                                   ↪ radius_m=track.radius, center=track.center.tolist(),
177                                   ↪ vertices=track.poly.tolist(),
178                                   ↪ observations=len(track.history)))
179
180         return observation
181
182     def scan(self, point, full=False):
183         unknown = self.unknown()
184         scan_unknown = full or self.coverage.gain(point, unknown) >= .05
185         channels = set(unknown if scan_unknown else [])
186         for c, track in self.tracks.items():
187             if c in self.robot.cleared:
188                 continue
189             if full or (track.radius > CLEAR_RADIUS and
190                       ↪ math.dist(point, track.history[-1]['position']) >= 200 and
191                       ↪ np.max(np.linalg.norm(track.poly - point, axis=1)) <= 999.9):
192                 channels.add(c)
193         if channels:
194             self.robot.scanned_sites += 1
195         # Start with current measurement channel if possible. Clear does not switch it.

```

```

192         for c in sorted(channels, key=lambda c: (c != self.robot.channel, c)):
193             self.observe(point, c)
194
195     def solve_track(self, track):
196         for _ in range(16):
197             self.robot.check_budget()
198             q = track.safe_clear_point(self.robot.position)
199             if q is not None:
200                 bound = float(np.max(np.linalg.norm(track.poly - q, axis=1)))
201                 self.robot.record(dict(kind='clear_certificate', channel=track.channel,
202                                     point=q.tolist(), worst_distance_m=bound))
203                 if not self.robot.clear(tuple(float(v) for v in q), track.channel):
204                     raise ArithmeticError('有界误差清除证书与接口失败冲突; 请保留记录检查')
205                 return
206             q = track.next_point(np.asarray(self.robot.position))
207             observation = self.observe(q, track.channel)
208             if observation['measure_result'] == 'no_signal':
209                 raise ArithmeticError('保证接收点返回无信号; 检查问题模式、参数和角度约定')
210             raise RuntimeError(f'频道 {track.channel} 达到 16 次定位预算, 仍未达到可靠清除精度')
211
212     def run(self):
213         self.scan(self.anchors.pop(0), full=True)
214         while True:
215             self.robot.check_budget()
216             pending = [t for c, t in self.tracks.items() if c not in
217                       ↪ self.robot.cleared]
218             if pending:
219                 def score(t):
220                     q = t.safe_clear_point(self.robot.position)
221                     q = t.center if q is None else q
222                     return math.dist(q, self.robot.position) + .15 * t.radius
223                 track = min(pending, key=score if self.nearest_order else lambda t:
224                           ↪ t.channel)
225                 self.solve_track(track)
226                 if len(self.robot.cleared) < 16 and self.opportunistic:
227                     self.scan(self.robot.position)
228                     continue
229             if len(self.robot.cleared) == 16:
230                 self.stop_reason = 'cleared_upper_bound_16'
231                 break
232             unknown = self.unknown()
233             if not unknown:
234                 self.stop_reason = 'continuous_coverage_certificate'
235                 break
236             if not self.anchors:
237                 raise RuntimeError('七点扫描后仍有频道未被覆盖证书排除; 不能声称全部清除')
238             site = min(self.anchors, key=lambda p: (math.dist(p, self.robot.position) +
239             ↪ 100) /
240                       max(self.coverage.gain(p, unknown), 1e-6))
241             self.anchors.remove(site)
242             self.scan(site, full=True)
243             self.robot.record(dict(kind='completion_certificate', **self.certificate()))
244             return self.stop_reason
245
246     def certificate(self):
247         absent = [c for c in range(1, 21) if c not in self.robot.discovered and
248                 ↪ self.coverage.absent(c)]
249         return dict(stop_reason=self.stop_reason, grid_level=self.coverage.level,
250                   square_side_m=self.coverage.half * 2,
251                   ↪ square_count=len(self.coverage.centers),
252                   absent_channels=absent,
253                   ↪ cleared_channels=sorted(self.robot.cleared),
254                   no_signal_positions={str(c): self.coverage.samples[c] for c in
255                   ↪ absent})

```

文件 q3/v6/lookahead.py

```

1  """Sparse posterior-consistent policy rollouts, using observation history only."""
2  import copy, math
3  import numpy as np
4  from service_routing import ServicePlanner
5  from belief_components import BeliefWorld, RolloutPlanner, execute

```

```

6
7
8 def complete(planner):
9     for _ in range(120):
10         planner.robot.check_budget()
11         pending=[c for c in planner.tracks if c not in planner.robot.cleared]
12         if not pending and not planner.unknown():return planner.robot.virtual
13         gain=planner.scan_gain;planner.scan_gain=2.
14         try:planner.at_site_scan(planner.robot.position)
15         finally:planner.scan_gain=gain
16         nodes=planner.candidates()
17         if nodes:execute(planner,nodes[0])
18     raise RuntimeError('Lookahead completion cap')
19
20
21 class LookaheadPlanner(ServicePlanner):
22     sample_worlds=RolloutPlanner.sample_worlds
23
24     def __init__(self,robot,*,rollout_samples=2,rollout_actions=3,rollout_margin=20.,*
↪ *kwargs):
25         super().__init__(robot,**kwargs)
26         self.rollout_samples=rollout_samples;self.rollout_actions=rollout_actions
27         self.rollout_margin=rollout_margin;self.rollout_calls=0;self.rollout_stages=se
↪ t()
28
29     def candidates(self):
30         result=super().candidates();stage=len(self.robot.cleared)//4
31         if not result or self.pending_probe is not None or stage>2 or stage in
↪ self.rollout_stages:return result
32         if self.last_plan is None or len(self.last_plan[1])<2:return result
33         self.rollout_stages.add(stage);self.rollout_calls+=1
34         _,nodes,order=self.last_plan
35         choices=list(result)
36         ordered=[nodes[i] for i in order]
37         scans=[n for n in ordered if n[0]=='scan']
38         for node in (scans[:1]+ordered):
39             if any(node[0]==old[0] and node[1]==old[1] and
↪ np.linalg.norm(node[2]-old[2])<.1 for old in choices):continue
40             choices.append(node)
41             if len(choices)>=self.rollout_actions:break
42         try:worlds=self.sample_worlds()
43         except (RuntimeError,ValueError):return result
44         state=copy.deepcopy({k:v for k,v in self.__dict__.items() if k!='robot'})
45         scores=[]
46         for node in choices:
47             costs=[]
48             for targets,salt in worlds:
49                 robot=BeliefWorld(targets,self.robot,salt)
50                 planner=object.__new__(ServicePlanner);planner.__dict__=copy.deepcopy(
↪ state);planner.robot=robot
51                 planner.cover_trials=1;planner.cover_radii=None;planner.transit_probes
↪ =False
52                 planner.service_route=False;planner.pending_probe=None;planner.merge_t
↪ rials=False
53                 try:execute(planner,node);costs.append(complete(planner))
54                 except (RuntimeError,ArithmeticError,ValueError):costs.append(float('i
↪ nf'))
55             scores.append(float(np.mean(costs)))
56         best=int(np.argmin(scores))
57         if not math.isfinite(scores[best]) or
↪ scores[0]-scores[best]<self.rollout_margin:best=0
58         self.robot.record(dict(kind='posterior_lookahead',stage=stage,samples=len(worl
↪ ds),
59             alternatives=[dict(kind=n[0],channel=int(n[1]),predicted_remaining_s=s if
↪ math.isfinite(s) else None) for n,s in zip(choices,scores)],
60             selected=best,prior='uniform area/radius and source count 10..16; planning
↪ only'))
61         return [choices[best]]

```

文件 q3/v6/merged_cover.py

```

1  """Continuous waypoint relocation with an explicit full-coverage constraint."""
2  import math
3  import numpy as np
4  from scipy.optimize import minimize
5  from scipy.spatial import ConvexHull
6  from joint_strategies import JointPlanner, open_route
7  from bearing_geometry import enclosing_circle, diameter
8
9
10 class MergedCoverPlanner(JointPlanner):
11     def __init__(self, robot, *, cover_sweeps=2, merge_stations=True, merge_target_scans=False,
12     ↪ lse, **kwargs):
13         super().__init__(robot, family='adaptive', **kwargs)
14         self.cover_sweeps=cover_sweeps
15         self.merge_stations=merge_stations
16         self.merge_target_scans=merge_target_scans
17
18     def candidates(self):
19         targets=[]
20         for c,t in self.tracks.items():
21             if c in self.robot.cleared:continue
22             q=t.safe_clear_point(self.robot.position)
23             q=t.center if q is None else q
24             targets.append(('target',c,np.asarray(q)))
25         unknown=self.unknown()
26         if not unknown:return targets
27         actual=self.coverage.covered[unknown[0]]
28         # Targets are mandatory route vertices but optional sensing sites.
29         points=[n[2].copy() for n in targets]+[np.array(p) for p in self.anchors]
30         n_target=len(targets)
31         active=np.ones(len(points),dtype=bool)
32         present=np.ones(len(points),dtype=bool)
33         masks=[self.coverage.mask(p) for p in points]
34         start=np.asarray(self.robot.position)
35         def route_order():
36             indices=[i for i in range(len(points)) if present[i]]
37             return [indices[j] for j in open_route(start,[points[i] for i in indices])]
38         def total_without(i):
39             total=actual.copy()
40             for j in range(len(points)):
41                 if j!=i and active[j]:total+=masks[j]
42             return total
43         def prune():
44             while True:
45                 order=route_order();gains=[]
46                 for k,i in enumerate(order):
47                     if not active[i]:continue
48                     gain=30*len(unknown)
49                     if i>=n_target:
50                         a=start if k==0 else points[order[k-1]]
51                         gain+=np.linalg.norm(points[i]-a)
52                         if k+1<len(order):
53                             b=points[order[k+1]]
54                             gain+=np.linalg.norm(points[i]-b)-np.linalg.norm(a-b)
55                         gains.append((gain,i))
56                 removed=False
57                 for gain,i in sorted(gains,reverse=True):
58                     if np.all(total_without(i)):
59                         active[i]=False
60                         if i>=n_target:present[i]=False
61                         removed=True;break
62                 if not removed:break
63         prune()
64         if self.merge_stations:
65             def full_cost(candidate_points,candidate_present,candidate_active):
66                 indices=np.flatnonzero(candidate_present)
67                 order=open_route(start,[candidate_points[i] for i in indices])
68                 p=start;value=30*len(unknown)*int(np.count_nonzero(candidate_active))
69                 for j in order:
70                     q=candidate_points[indices[j]];value+=np.linalg.norm(q-p);p=q
71                 return float(value)
72             for merge_iteration in range(4):

```

```

72     old_cost=full_cost(points,present,active);best=None
73     selected=[i for i in range(0 if self.merge_target_scans else
74         ↪ n_target,len(points)) if active[i]]
75     for at,i in enumerate(selected):
76         for j in selected[at:]:
77             remaining=actual.copy()
78             for k in range(len(points)):
79                 if active[k] and k not in [i,j]:remaining|=masks[k]
80             needed=self.coverage.centers[~remaining]
81             if not len(needed):continue
82             h=self.coverage.half
83             corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in
84                 ↪ [(-1,-1),(-1,1),(1,-1),(1,1)])])
85             hull=convexHull(corners).vertices
86             if diameter(hull)[0]>1999.9799:continue
87             center,radius,_=enclosing_circle(hull)
88             if radius>999.98999:continue
89             new_points=points+[center]
90             new_present=np.append(present,True);new_active=np.append(activ
91                 ↪ e,True)
92             for removed in [i,j]:
93                 if removed>=n_target:new_present[removed]=False
94                 new_active[removed]=False
95             value=full_cost(new_points,new_present,new_active)
96             if value<old_cost-1e-6 and (best is None or value<best[0]):
97                 best=(value,i,j,center,new_points,new_present,new_active)
98             if best is None:break
99             value,i,j,center,points,present,active=best
100             masks.append(self.coverage.mask(center))
101             self.robot.record(dict(kind='merge_scans',point=center.tolist(),remove
102                 ↪ d=[int(i),int(j)],
103                 ↪ predicted_saving_s=(old_cost-value)/5,proof='MEC of all
104                 ↪ responsibility-cell corners <= 999.99 m'))
105             prune()
106 for _ in range(self.cover_sweeps):
107     order=route_order()
108     improvement=0.
109     for k,i in enumerate(order):
110         if i<n_target or not active[i]:continue
111         needed=self.coverage.centers[~total_without(i)]
112         if not len(needed):continue
113         h=self.coverage.half
114         corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in
115             ↪ [(-1,-1),(-1,1),(1,-1),(1,1)])])
116         witnesses=convexHull(corners).vertices/1000
117         before=points[i]/1000
118         previous=(start if k==0 else points[order[k-1]])/1000
119         following=points[order[k+1]]/1000 if k+1<len(order) else None
120         def objective(q):
121             value=np.sqrt(np.dot(q-previous,q-previous)+1e-16)
122             if following is not
123                 ↪ None:value+=np.sqrt(np.dot(q-following,q-following)+1e-16)
124             return float(value)
125         def gradient(q):
126             value=(q-previous)/max(np.linalg.norm(q-previous),1e-8)
127             if following is not
128                 ↪ None:value+=(q-following)/max(np.linalg.norm(q-following),1e-8)
129             return value
130         cons={'type':'ineq','fun':lambda q:
131             ↪ .99998999**2-np.sum((q-witnesses)**2,axis=1),
132             ↪ 'jac':lambda q:-2*(q-witnesses)}
133         sol=minimize(objective,before,jac=gradient,constraints=cons,method='SL
134             ↪ SQP',
135             ↪ options={'ftol':1e-10,'maxiter':60})
136         if np.max(np.linalg.norm(witnesses-sol.x,axis=1))<=.99999 and
137             ↪ objective(sol.x)<=objective(before)+1e-8:
138             improvement+=objective(before)-objective(sol.x)
139             points[i]=sol.x*1000;masks[i]=self.coverage.mask(points[i])
140         prune()
141         if improvement<1e-5:break
142     union=actual.copy()
143     for i in range(len(points)):
144         if active[i]:union|=masks[i]
145     if not np.all(union):raise ArithmeticError('Relocated cover lost cells')
146     scans=[('scan',-100-i,points[i]) for i in range(len(points)) if active[i]]

```

```

136         nodes=targets+scans
137         self.cover_targets=set()
138         order=open_route(start,[n[2] for n in nodes])
139         if nodes[order[0]][0]=='target':
140             first=nodes[order[0]]
141             matches=[n for n in scans if math.dist(n[2],first[2])<1e-5]
142             if matches:return [matches[0]]
143         return nodes

```

文件 q3/v6/reception_region.py

```

1  """Exact first-positive reception separator on a convex polygon outer region.
2
3  For each x, the least admissible reception radius is max(R0, |x-s|).
4  Evaluate the maximum violation over polygon vertices, edge/circle crossings,
5  and the antipodal stationary point on the R0-circle when inside the polygon.
6  """
7  import math
8  import numpy as np
9
10
11  def reception_violation(poly,first_sensor,query,r0=1000.):
12      poly=np.asarray(poly,float);s=np.asarray(first_sensor,float);q=np.asarray(query,float)
13      if not len(poly):raise ValueError('Empty source region')
14      points=list(poly)
15      edges=np.roll(poly,-1,axis=0)-poly
16      for p,d in zip(poly,edges):
17          a=float(np.dot(d,d))
18          if a<1e-18:continue
19          offset=p-s;b=2*float(np.dot(offset,d));c=float(np.dot(offset,offset)-r0*r0)
20          discr=b*b-4*a*c
21          if discr>=0:
22              root=math.sqrt(discr)
23              for t in [(-b-root)/(2*a),(-b+root)/(2*a)]:
24                  if 0<=t<=1:points.append(p+t*d)
25      d=q-s;length=float(np.linalg.norm(d))
26      if length>1e-12:
27          far=s-r0*d/length
28          delta=far-poly
29          cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
30          if np.all(cross>=-1e-8*np.maximum(1,np.linalg.norm(edges,axis=1))):points.append(far)
31      points=np.asarray(points)
32      values=np.sum((points-q)**2,axis=1)-np.maximum(r0*r0,np.sum((points-s)**2,axis=1))
33      k=int(np.argmax(values))
34      return float(values[k]),points[k].copy()

```

文件 q3/v6/run_robot.py

```

1  """ 问题三多策略研究版；尚未达到平均 200 秒/点目标。 """
2  import argparse
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import sys
9  import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):
17         sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['v5','v6','conservative','no_update'],
20                         default='v6', help='选择冻结策略，默认v6光学动作价值候选')

```

```

20 parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21 parser.add_argument('--port', type=int, help='默认复用旧配置, 缺省端口 2026')
22 parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23 parser.add_argument('--check', action='store_true', help='仅检查依赖和配置,
    ↪ 不连接模拟器')
24 args = parser.parse_args()
25 try:
26     import numpy
27     import scipy
28     import shapely
29     from api_client import Robot, BudgetReached
30     from strategy import Planner, VERSION, POLICIES
31 except ImportError as error:
32     print(f'缺少依赖: {error}\n 请安装 requirements.txt 中的依赖后重试。')
33     return 2
34 paths = [args.config] if args.config else [ROOT / 'config.json', ROOT.parent /
    ↪ 'simple-strategy' / 'config.json']
35 config_path = next((p for p in paths if p and p.is_file()), None)
36 config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path
    ↪ else {}
37 team, port = args.team or config.get('team_no'), args.port or config.get('port',
    ↪ 2026)
38 if not team:
39     print('找不到队号配置, 请使用 --config 指定旧 config.json, 或使用 --team 参数。')
40     return 2
41 if args.check:
42     print(f'策略 {args.policy}: 研究版, 200 秒/点目标尚未达成。')
43     print(f'自检通过: Python {sys.version.split()[0]} / numpy {numpy.__version__} /
    ↪ scipy {scipy.__version__}')
44     print(f'配置可读: 端口 {port}; 未连接模拟器。')
45     return 0
46 print(f'问题 3 研究策略 {VERSION} / {args.policy} | 端口 {port}')
47 print('这是可运行的研究候选, 尚未达到平均 200 秒/点目标。')
48 print('请在模拟器选择【问题 3 演练测试】, 开始后等待“接口已就绪/等待机器狗进入”。')
49 print('接口无法查询演练/正式模式, 程序按问题 3 全向干扰器规则工作。')
50 input('界面就绪后按 Enter 开始: ')
51 folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' +
    ↪ uuid.uuid4().hex[:6])
52 robot = Robot(team, f'http://127.0.0.1:{port}', folder)
53 planner = Planner(robot, policy=args.policy)
54 snapshot = folder / 'source_snapshot'
55 snapshot.mkdir()
56 hashes = {}
57 for p in list(ROOT.glob('*.py')) + [ROOT / 'policies.json']:
58     shutil.copy2(p, snapshot / p.name)
59     hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60 (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION,
    ↪ policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__,
    ↪ shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path),
    ↪ mode='user_selected_P3_practice_not_API_verified'),
63     ensure_ascii=False, indent=2), encoding='utf-8')
64 outcome, error_text, entered = 'completed', None, False
65 robot.started = time.monotonic()
66 try:
67     robot.call('/enter')
68     entered = True
69     planner.run()
70 except BudgetReached as error:
71     outcome, error_text = 'budget_stop', str(error)
72 except (Exception, KeyboardInterrupt) as error:
73     outcome, error_text = 'error', str(error) or '用户中断'
74     # Interruption during HTTP could leave execution state unknown.
75     if isinstance(error, KeyboardInterrupt):
76         robot.transport_uncertain = True
77 finally:
78     if entered and not robot.transport_uncertain:
79         try:
80             robot.call('/exit')
81         except Exception as error:
82             outcome = 'exit_unconfirmed'
83             error_text = (error_text or '') + '\n 退出未确认: ' + str(error)
84     count = len(robot.cleared)

```

```

85     summary = dict(strategy=VERSION, policy=args.policy,
86     ↪ policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
87     ↪ exit_confirmed=robot.exit_confirmed,
88     ↪ transport_uncertain=robot.transport_uncertain,
89     ↪ cleared_count=count, cleared_channels=sorted(robot.cleared),
90     ↪ observed_but_uncleared=sorted(robot.discovered - robot.cleared),
91     ↪ measure_count=robot.measure_count, clear_failures=robot.clear_failures,
92     ↪ scanned_sites=robot.scanned_sites, distance_m=robot.distance_m,
93     ↪ switches=robot.switches,
94     ↪ virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if
95     ↪ count else None,
96     ↪ local_elapsed_s=time.monotonic() - robot.started,
97     ↪ actual_jammer_count=None, completion=planner.certificate(),
98     ↪ note='实际目标总数和正式成绩以模拟器为准：完整性证书以问题 3 全向规则为前提。')
99     (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False,
100     ↪ indent=2), encoding='utf-8')
101     robot.log.close()
102     print(f'\n结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动
103     ↪ {robot.distance_m:.1f} 米。')
104     print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据
105     ↪ {planner.stop_reason}')
106     if error_text:
107         print(error_text)
108     print(f'运行记录: {folder}')
109     print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
110     return 0 if outcome == 'completed' and robot.exit_confirmed else 1
111
112 if __name__ == '__main__':
113     sys.exit(main())

```

文件 q3/v6/service_routing.py

```

1  """Whole-service routing, rotated cover plans and paid measurements in transit."""
2  import copy, math
3  import numpy as np
4  from combined_cost_cover import CombinedCostCoverPlanner
5  from flexible_cover import FlexibleCoverPlanner
6  from joint_strategies import open_route
7  from voronoi_coverage import HybridCoverage, covering_radius
8
9
10 def directed_route(start, nodes, tracks, restarts=3):
11     exits=[np.asarray(start)]+[np.asarray(tracks[n[1]].center if n[0]=='target' else
12     ↪ n[2]) for n in nodes]
13     size=len(nodes); D=np.zeros((size+1, size+1))
14     for i, p in enumerate(exits):
15         for j, (kind, c, q) in enumerate(nodes, 1):
16             if i==j: continue
17             entry=q
18             if kind=='target':
19                 tr=tracks[c]; safe=tr.safe_clear_point(p)
20                 entry=safe if safe is not None else tr.next_point(p)
21                 D[i, j]=np.linalg.norm(entry-p)+np.linalg.norm(exits[j]-entry)
22     def value(r): return float(sum(D[a, b] for a, b in zip(r, r[1:])))
23     best=None
24     for restart in range(min(restarts, size)):
25         left=set(range(1, size+1)); r=[0]
26         first=sorted(left, key=lambda
27         ↪ j: D[0, j])[restart]; r.append(first); left.remove(first)
28         while left:
29             j=min(left, key=lambda j: D[r[-1], j]); r.append(j); left.remove(j)
30         for _ in range(25):
31             old=value(r); chosen=None
32             for i in range(1, len(r)-1):
33                 for j in range(i+1, len(r)):
34                     trial=r[:i]+list(reversed(r[i:j+1]))+r[j+1:]; v=value(trial)
35                     if v<old-1e-7: old, chosen=v, trial=trial
36             if chosen is None: break
37             r=chosen
38     val=value(r)
39     if best is None or val<best[0]: best=(val, r)

```

```

38     return [j-1 for j in best[1][1:]],best[0]
39
40
41 class ServicePlanner(CombinedCostCoverPlanner):
42     def __init__(self,robot,*,service_route=False,cover_trials=1,transit_probes=False,
43                 probe_gain=100.,probe_min_radius=100.,cover_radii=None,initial_design_
44                 ↪ =False,cover_target_entry=False,target_estimator='mec',merge_trial
45                 ↪ ls=False,warm_cover=False,hybrid_cover=False,continuous_stop=False
46                 ↪ e,**kwargs):
47         super().__init__(robot,**kwargs)
48         self.service_route=service_route;self.cover_trials=cover_trials
49         self.transit_probes=transit_probes;self.probe_gain=probe_gain
50         self.probe_min_radius=probe_min_radius;self.pending_probe=None
51         self.cover_radii=cover_radii;self.initial_design=initial_design
52         self.initial_designed=False
53         self.cover_target_entry=cover_target_entry
54         self.target_estimator=target_estimator;self.merge_trials=merge_trials
55         self.warm_cover=warm_cover;self.warm_scans=[]
56         self.hybrid_cover=hybrid_cover
57         if hybrid_cover or continuous_stop:self.coverage=HybridCoverage()
58
59     def target_position(self,tr):
60         if self.target_estimator=='mec':return tr.center
61         v=tr.poly-tr.center;u=np.roll(v,-1,axis=0)
62         cross=v[:,0]*u[:,1]-u[:,0]*v[:,1]
63         center=tr.center+np.sum((v+u)*cross[:,None],axis=0)/(3*cross.sum()) if
64         ↪ abs(cross.sum())>1e-10 else tr.center
65         if self.target_estimator=='wls':
66             A=[];b=[]
67             for h in tr.history:
68                 if h['measure_result']!='direction':continue
69                 angle=math.radians(h['svd_deg']);p=np.asarray(h['position'])
70                 normal=np.array([-math.sin(angle),math.cos(angle)]) / max(10,np.linalg.n_
71                 ↪ orm(center-p))
72                 A.append(normal);b.append(np.dot(normal,p))
73             if len(A)>=2:
74                 estimate=np.linalg.lstsq(np.asarray(A),np.asarray(b),rcond=None)[0]
75                 edges=np.roll(tr.poly,-1,axis=0)-tr.poly;delta=estimate-tr.poly
76                 if np.all(edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]>=-1e-7):center=e
77                 ↪ stimate
78         return center
79
80     def design_initial_point(self):
81         original_radius=self.initial_baseline;best=None
82         for radius in [250.,450.,650.]:
83             for angle in np.linspace(0,2*math.pi,12,endpoint=False):
84                 q=radius*np.array([math.cos(angle),math.sin(angle)]);centers=[];uncert_
85                 ↪ ainty=0.
86                 for tr in self.tracks.values():
87                     if np.linalg.norm(q-tr.center)>1000:
88                         centers.append(tr.center);uncertainty+=tr.radius;continue
89                     bearing=math.degrees(math.atan2(tr.center[1]-q[1],tr.center[0]-q[0]
90                     ↪ ))
91                     radii=[];cs=[]
92                     for error in [-1.,0.,1.]:
93                         temporary=copy.copy(tr);temporary.history=list(tr.history);tem_
94                         ↪ porary.poly=tr.poly.copy()
95                         temporary.update(q,dict(measure_result='direction',svd_deg=rou_
96                         ↪ nd((bearing+error)%360,2)))
97                         radii.append(temporary.radius);cs.append(temporary.center)
98                         centers.append(np.mean(cs,axis=0));uncertainty+=np.mean(radii)
99                     points=centers+[np.asarray(p) for p in self.anchors[1:]]
100                     order=open_route(q,points);p=q;score=radius+uncertainty
101                     for i in order:score+=np.linalg.norm(points[i]-p);p=points[i]
102                     if best is None or score<best[0]:best=(score,q)
103         return best[1]
104
105     def route(self,nodes):
106         if self.service_route:return
107         ↪ directed_route(self.robot.position,nodes,self.tracks)
108         order=open_route(self.robot.position,[n[2] for n in
109         ↪ nodes],restarts=self.route_restarts)
110         p=np.asarray(self.robot.position);cost=0.
111         for i in order:cost+=np.linalg.norm(nodes[i][2]-p);p=nodes[i][2]

```

```

100         return order, float(cost)
101
102     def probe_on_leg(self, node):
103         start=np.asarray(self.robot.position);end=node[2]
104         if node[0]=='target':
105             tr=self.tracks[node[1]];safe=tr.safe_clear_point(start)
106             end=safe if safe is not None else tr.next_point(start)
107         if np.linalg.norm(end-start)<250: return None
108         best=None
109         for f in [.25,.5,.75]:
110             q=start+f*(end-start);channels=[];benefit=0.
111             for c,tr in self.tracks.items():
112                 if c in self.robot.cleared or tr.radius<self.probe_min_radius:continue
113                 if min(math.dist(q,h['position']) for h in tr.history)<150:continue
114                 if math.dist(q,self.attempts.get(c,(1e9,1e9)))<150:continue
115                 if np.linalg.norm(q-tr.center)>1000:continue
116                 radii=[]
117                 angle=math.degrees(math.atan2(tr.center[1]-q[1],tr.center[0]-q[0]))
118                 for error in [-1.,0.,1.]:
119                     temporary=copy.copy(tr);temporary.history=list(tr.history);tempora
120                     ↪ ry.poly=tr.poly.copy()
121                     ↪ temporary.update(q,dict(measure_result='direction',svd_deg=round((
122                     ↪ angle+error)%360,2)))
123                     radii.append(temporary.radius)
124                     gain=tr.radius-float(np.mean(radii))
125                     if gain>=self.probe_gain:channels.append(c);benefit+=gain-30
126                 if channels and (best is None or benefit>best[0]):best=(benefit,q,channels)
127             return None if best is None else best[1:]
128
129     def plan(self):
130         original=self.anchors;radius=math.hypot(*original[1]);best=None;original_merge
131         ↪ =self.merge_stations
132         try:
133             for trial in range(self.cover_trials*len(self.cover_radii or [radius])):
134                 radius=(self.cover_radii or [radius])[trial//self.cover_trials]
135                 rotation=trial*self.cover_trials
136                 angle=math.pi/3*rotation/self.cover_trials
137                 self.anchors=[(0.,0.)]+[(radius*math.cos(angle+i*math.pi/3),radius*mat
138                 ↪ h.sin(angle+i*math.pi/3)) for i in range(6)]
139                 cover=np.zeros(len(self.coverage.centers),dtype=bool)
140                 for p in self.anchors:cover|=self.coverage.mask(p)
141                 if not np.all(cover) and not (self.hybrid_cover and
142                 ↪ covering_radius(self.anchors)[0]<=999.98):continue
143                 for merged in ([False,True] if self.merge_trials else
144                 ↪ [original_merge]):
145                     self.merge_stations=merged
146                     nodes=FlexibleCoverPlanner.candidates(self)
147                     if not nodes:continue
148                     order,cost=self.route(nodes)
149                     score=cost+30*len(self.unknown())*sum(n[0]=='scan' for n in nodes)
150                     if best is None or score<best[0]:best=(score,nodes,order)
151                 if self.warm_cover and self.warm_scans:
152                     self.anchors=original+[tuple(p) for p in self.warm_scans]
153                     self.merge_stations=original_merge
154                     nodes=FlexibleCoverPlanner.candidates(self)
155                     if nodes:
156                         order,cost=self.route(nodes)
157                         score=cost+30*len(self.unknown())*sum(n[0]=='scan' for n in nodes)
158                         if best is None or score<best[0]:best=(score,nodes,order)
159                 finally:self.anchors=original;self.merge_stations=original_merge
160             return best
161
162     def candidates(self):
163         best=self.plan();self.last_plan=best
164         if best is None:return []
165         _,nodes,order=best;first=nodes[order[0]]
166         if self.warm_cover:self.warm_scans=[n[2].copy() for n in nodes if n[0]=='scan']
167         # A planned scan at the target point must happen before a moving solve.
168         if first[0]=='target':
169             match=[n for n in nodes if n[0]=='scan' and math.dist(n[2],first[2])<1e-5]
170             if match:first=match[0]
171         if self.transit_probes:
172             probe=self.probe_on_leg(first)
173             if probe is not None:
174                 self.pending_probe=probe

```

```

169         return [('scan', -9999, probe[0])]
170     return [first]
171
172     def at_site_scan(self, point, force=False):
173         if self.initial_design and not self.initial_designed and self.iterations==0 and
            ↪ np.linalg.norm(point)>.1:
174             point=self.design_initial_point();self.initial_designed=True
175             self.robot.record(dict(kind='initial_design', point=point.tolist()))
176         if force and self.pending_probe is not None and
            ↪ math.dist(point, self.pending_probe[0])<1e-6:
177             q, channels=self.pending_probe;self.pending_probe=None
178             self.robot.record(dict(kind='transit_probe', point=q.tolist(), channels=chan
            ↪ nels))
179             for c in sorted(channels, key=lambda
            ↪ c: (c!=self.robot.channel, c)):self.observe(q, c)
180             return
181         return super().at_site_scan(point, force=force)
182
183     def certificate(self):
184         result=super().certificate()
185         if isinstance(self.coverage, HybridCoverage):
186             result['exact_disk_cover']={str(c):dict(max_nearest_distance_m=self.covera
            ↪ ge.exact_status(self.coverage.samples[c])[0],
187                 witness=self.coverage.exact_status(self.coverage.samples[c])[1].tolist
            ↪ (), accepted_by_grid=bool(np.all(self.coverage.covered[c]))) for c
            ↪ in result['absent_channels']}
188         return result
189
190
191     class TransitSearchPlanner(ServicePlanner):
192         def __init__(self, robot, *, search_margin_s=10., search_bonus=0., **kwargs):
193             super().__init__(robot, **kwargs)
194             self.search_margin_s=search_margin_s;self.search_bonus=search_bonus
195
196         def candidates(self):
197             nodes=super().candidates();unknown=self.unknown()
198             if not nodes or not unknown or self.pending_probe is not None:return nodes
199             first=nodes[0];p=np.asarray(self.robot.position);q=first[2]
200             if first[0]=='target':
201                 tr=self.tracks[first[1]];safe=tr.safe_clear_point(p)
202                 q=safe if safe is not None else tr.next_point(p)
203             if np.linalg.norm(q-p)<350:return nodes
204             previous=self.coverage.covered[unknown].copy();old_position=self.robot.position
205             before=self.last_plan[0];best=None
206             try:
207                 for fraction in [.33,.67]:
208                     probe=p+fraction*(q-p);mask=self.coverage.mask(probe)
209                     gain=float(np.mean(mask&~previous[0]))
210                     if gain<.015:continue
211                     self.coverage.covered[unknown]=previous|mask
212                     self.robot.position=tuple(probe)
213                     after=self.plan()
214                     if after is None:continue
215                     cost=np.linalg.norm(probe-p)+30*len(unknown)+after[0]
216                     hidden=max(0, 13-len(self.robot.discovered))
217                     bonus=self.search_bonus*hidden*gain/max(.05, float(np.mean(~previous[0]
            ↪ )))
            ↪ saving=(before-cost+bonus)/5
218                     if saving>self.search_margin_s and (best is None or
            ↪ saving>best[0]):best=(saving, probe, gain)
219
220             finally:
221                 self.coverage.covered[unknown]=previous;self.robot.position=old_position
222             if best is not None:
223                 self.robot.record(dict(kind='transit_search_plan', point=best[1].tolist(), p
            ↪ redicted_saving_s=best[0], new_coverage_fraction=best[2], planning_only=
            ↪ True))
224                 return [('scan', -9998, best[1])]
225             return nodes

```

文件 q3/v6/strategy.py

```
1 """Frozen optical-action candidate and previous v5 control."""
2 import json
3 from pathlib import Path
4 from service_routing import ServicePlanner
5 from action_value import ActionValuePlanner
6 VERSION='p3-action-value-v6-research'
7 POLICIES=json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-
  ↪ 8'))
8 def Planner(robot,*,policy='v6'):
9     params=POLICIES[policy].copy();kind=params.pop('_class',None)
10    return (ActionValuePlanner if kind=='action' else ServicePlanner)(robot,**params)
```

文件 q3/v6/voronoi_coverage.py

```
1 """Exact finite candidates for maximum distance to nearest sensing site in a disk."""
2 import itertools
3 import numpy as np
4 from legacy_strategy import Coverage
5
6
7 def covering_radius(sites,arena_radius=1800.):
8     p=np.unique(np.asarray(sites,float).reshape(-1,2),axis=0)
9     if not len(p):return float('inf'),np.array([arena_radius,0.])
10    candidates=[np.array([arena_radius,0.])]
11    norms=np.linalg.norm(p,axis=1)
12    candidates.extend(-arena_radius*p[norms>1e-10]/norms[norms>1e-10,None])
13    for i,a in enumerate(p):
14        for b in p[i:]:
15            d=b-a;length=np.linalg.norm(d)
16            if length<1e-10:continue
17            normal=d/length;offset=np.dot((a+b)/2,normal)
18            if abs(offset)>arena_radius:continue
19            foot=offset*normal;side=np.sqrt(max(0.,arena_radius**2-offset**2))*np.arra
  ↪ y([-normal[1],normal[0]])
20            candidates.extend([foot+side,foot-side])
21    triples=np.array(list(itertools.combinations(range(len(p)),3)),dtype=int)
22    if len(triples):
23        a,b,c=p[triples[:,0]],p[triples[:,1]],p[triples[:,2]]
24        u=b-a;v=c-a;det=u[:,0]*v[:,1]-u[:,1]*v[:,0]
25        ok=abs(det)>1e-12*np.maximum(1.,np.linalg.norm(u,axis=1)*np.linalg.norm(v,axis
  ↪ =1))
26        a,u,v,det=a[ok],u[ok],v[ok],det[ok]
27        hu=np.sum(u*u,axis=1)/2;hv=np.sum(v*v,axis=1)/2
28        centers=a+np.column_stack([(hu*v[:,1]-hv*u[:,1])/det,(u[:,0]*hv-v[:,0]*hu)/det
  ↪ ])
29        candidates.extend(centers[np.linalg.norm(centers,axis=1)<=arena_radius])
30    x=np.asarray(candidates);values=np.min(np.linalg.norm(x[:,None,:]-p[None,:,:,axis
  ↪ =2),axis=1)
31    k=int(np.argmax(values));return float(values[k]),x[k].copy()
32
33
34 class HybridCoverage(Coverage):
35     def __init__(self):
36         super().__init__();self.exact_cache={}
37
38     def exact_status(self,sites):
39         key=tuple(sorted(set(tuple(float(x) for x in p) for p in sites)))
40         if key not in self.exact_cache:self.exact_cache[key]=covering_radius(key)
41         return self.exact_cache[key]
42
43     def absent(self,channel):
44         return super().absent(channel) or
  ↪ self.exact_status(self.samples[channel])[0]<=999.98
```

11.6 问题四：定向源联合区域规划

文件 q4/M32/api_client.py

本文件与 q3/v6/api_client.py 内容完全相同, 使用前述代码。

文件 q4/M32/base.py

```
1 """Fresh Q4 prototypes. No imports from any previous Q4 strategy.
2
3 Only Q1 geometry is reused. Physics hides truth behind the Robot facade.
4 All strategies share a proved discovery mesh and a finite optical fallback.
5 """
6 from pathlib import Path
7 import sys, math, json, argparse, time, hashlib, platform
8 import numpy as np
9 from shapely.geometry import Point, Polygon
10 from shapely.ops import unary_union
11 from scipy.spatial import Delaunay
12 MODEL = Path(__file__).resolve().parents[1]
13 ROOT = MODEL
14 from bearing_geometry import outer_disk, clip_halfplanes, wedge, enclosing_circle,
15     ↪ diameter, unit
16 EPS = 1.0051
17 SPACING = 980.0
18 NAMES = {'staged': 'A 先搜后清', 'interleave': 'B 边搜边清', 'heading': 'C 朝向情景选点',
19     ↪ 'optical': 'D 朝向选点加光学捷径', 'ring': 'E 环带围合加朝向选点'}
20
21 def save(path, data):
22     path.parent.mkdir(parents=True, exist_ok=True)
23     tmp = path.with_suffix('.tmp')
24     tmp.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False),
25         ↪ encoding='utf-8')
26     tmp.replace(path)
27
28 def mesh():
29     mapping = {}
30     faces = []
31     for i in range(-5, 5):
32         for j in range(-5, 5):
33             for ids in [(i, j), (i + 1, j), (i, j + 1)], [(i + 1, j), (i + 1, j + 1),
34                 ↪ (i, j + 1)]:
35                 vs = np.array([[SPACING * (x + 0.5 * y), SPACING * math.sqrt(3) / 2 *
36                     ↪ y] for (x, y) in ids])
37                 if Polygon(vs).distance(Point(0, 0)) <= 1800.0:
38                     faces.append(ids)
39                     for (key, v) in zip(ids, vs):
40                         mapping[key] = v
41     keys = sorted(mapping)
42     pts = np.array([mapping[k] for k in keys])
43     idx = {k: i for (i, k) in enumerate(keys)}
44     tri = [[idx[k] for k in face] for face in faces]
45     return (pts, tri)
46 (SITES, FACES) = mesh()
47
48 class Track:
49
50     def __init__(self, ch):
51         self.channel = ch
52         self.region = None
53         self.history = []
54         self.positive = []
55         self.center = None
56         self.radius = np.inf
57         self.poly = None
58
59     def refresh(self):
60         if self.region.is_empty:
61             raise ArithmeticError('empty outer position set')
62         self.poly = np.array(self.region.convex_hull.exterior.coords[:-1])
63         (self.center, self.radius, _) = enclosing_circle(self.poly)
64
65     def update(self, q, z):
66         self.history.append(dict(point=np.asarray(q).tolist(), **z))
67         if z['measure_result'] == 'no_signal':
68             return
69         self.positive.append(self.history[-1])
```

```

65         rad = 5.0 if z['measure_result'] == 'near' else 1500.0
66         new = outer_disk(q, rad, 144 if rad > 5 else 32)
67         if z['measure_result'] == 'direction':
68             new = clip_halfplanes(new, *wedge(q, z['svd_deg'], EPS))
69         g = Polygon(new).intersection(Polygon(outer_disk([0, 0], 1800, 144)))
70         self.region = g if self.region is None else self.region.intersection(g)
71         self.refresh()
72
73     def exclude(self, q):
74         self.region = self.region.difference(Point(*q).buffer(19.999, quad_segs=24))
75         self.refresh()
76
77     def samples(track):
78         p = track.poly
79         c = np.array(track.region.representative_point().coords[0])
80         (_, (i, j)) = diameter(p)
81         candidates = [c, *[0.9 * ((1 - f) * p[i] + f * p[j]) + 0.1 * c for f in
82             ↪ np.linspace(0, 1, 9)]]
83         candidates += [0.95 * x + 0.05 * c for x in p]
84         keep = []
85         for x in candidates:
86             if track.region.covers(Point(*x)) and all((np.linalg.norm(x - y) > 0.0001 for y
87                 ↪ in keep)):
88                 keep.append(x)
89         return np.array(keep[:16])
90
91     def heading_hypotheses(track):
92         g = samples(track)
93         positions = np.repeat(g, 3 * 25, axis=0)
94         radii = np.tile(np.repeat([1000.0, 1250.0, 1500.0], 25), len(g))
95         normals = np.tile(np.vstack([np.zeros(2), np.c_[np.cos(np.arange(24) * np.pi / 12),
96             ↪ np.sin(np.arange(24) * np.pi / 12)]]), (len(g) * 3, 1))
97         good = np.ones(len(positions), dtype=bool)
98         for h in track.history:
99             delta = np.asarray(h['point']) - positions
100             d = np.linalg.norm(delta, axis=1)
101             visible = (delta * normals).sum(axis=1) >= -1e-08
102             heard = (d <= radii + 1e-06) & visible
103             if h['measure_result'] == 'no_signal':
104                 good &= ~heard
105             else:
106                 good &= heard
107                 if h['measure_result'] == 'near':
108                     good &= d <= 5.001
109                 else:
110                     good &= d > 4.999
111         return (positions[good], radii[good], normals[good])
112
113 class Planner:
114
115     def __init__(self, robot, variant, sites=None):
116         self.sites = SITES if sites is None else np.asarray(sites, float)
117         self.robot = robot
118         self.variant = variant
119         self.tracks = {}
120         self.negatives = {ch: [] for ch in range(1, 21)}
121         self.visited = set()
122         self.scan_history = []
123         self.hyp_empty = 0
124         self.fallbacks = 0
125         self.lost = 0
126
127     def observe(self, q, ch):
128         z = self.robot.measure(q, ch)
129         if ch not in self.tracks and z['measure_result'] != 'no_signal':
130             t = Track(ch)
131             t.history = list(self.negatives[ch])
132             self.tracks[ch] = t
133         if ch in self.tracks:
134             self.tracks[ch].update(q, z)
135             t = self.tracks[ch]
136             self.robot.record(dict(kind='region', channel=ch, wkt=t.region.wkt,
137                 ↪ radius_m=t.radius))
138         elif z['measure_result'] == 'no_signal':
139             self.negatives[ch].append(dict(point=np.asarray(q).tolist(), *z))

```

```

136         return z
137
138     def try_clear(self, t, q, reason):
139         self.robot.record(dict(kind='clear_reason', channel=t.channel, reason=reason,
140                               ↪ radius_m=t.radius))
141         ok = self.robot.clear(q, t.channel)
142         if not ok:
143             t.exclude(q)
144         return ok
145
146     def candidates(self, t):
147         p = self.robot.position
148         c = t.center
149         (_, (i, j)) = diameter(t.poly)
150         u = t.poly[j] - t.poly[i]
151         u = u / max(np.linalg.norm(u), 1e-12)
152         v = np.array([-u[1], u[0]])
153         off = max(25.0, min(220.0, t.radius * 0.5))
154         raw = [c + off * v, c - off * v, c]
155         for f in [0.4, 0.7]:
156             mid = p + f * (c - p)
157             raw += [mid, mid + 0.6 * off * v, mid - 0.6 * off * v]
158         raw += [c + 0.25 * off * u, c - 0.25 * off * u]
159         return [q for q in raw if all((math.dist(q, h['point']) > 2 for h in
160                                     ↪ t.history))]
161
162     def select(self, t):
163         candidates = self.candidates(t)
164         if not candidates:
165             return None
166         p = self.robot.position
167         if self.variant in ['staged', 'interleave']:
168             q = min(candidates[:2], key=lambda q: np.linalg.norm(q - p))
169             return q
170         (g, rad, n) = heading_hypotheses(t)
171         if not len(g):
172             self.hyp_empty += 1
173             return min(candidates[:2], key=lambda q: np.linalg.norm(q - p))
174         scores = []
175         for q in candidates:
176             delta = q - g
177             visible = (np.linalg.norm(delta, axis=1) <= rad) & ((delta * n).sum(axis=1)
178                       ↪ >= -1e-08)
179             prob = float(visible.mean())
180             vg = g[visible]
181             if len(vg):
182                 (ug, counts) = np.unique(np.round(vg, 8), axis=0, return_counts=True)
183                 sizes = []
184                 for x in ug:
185                     d = np.linalg.norm(x - q)
186                     if d <= 5:
187                         sizes.append(5.0)
188                     continue
189                     angle = math.degrees(math.atan2(x[1] - q[1], x[0] - q[0]))
190                     pp = clip_halfplanes(t.poly, *wedge(q, angle, EPS))
191                     sizes.append(enclosing_circle(pp)[1] if len(pp) else t.radius)
192                 after = float(np.average(sizes, weights=counts))
193             else:
194                 after = t.radius
195             cost = np.linalg.norm(q - p) / 5 + 5 + max(0, np.linalg.norm(q - t.center)
196               ↪ - 19) / 5
197             cost += prob * (after / 5 + 8 * max(0, math.log(max(after / 19, 1), 2)))
198             cost += (1 - prob) * (35 + min(t.radius, 350) / 2.5)
199             scores.append(float(cost))
200         best = int(np.argmin(scores))
201         self.robot.record(dict(kind='scenario_choice', channel=t.channel,
202                               ↪ scenarios=len(g), candidates=[dict(point=q.tolist(), cost_s=sc) for (q, sc)
203                               ↪ in zip(candidates, scores)], selected=best))
204         return candidates[best]
205
206     def optical_cover(self, t):
207         self.fallbacks += 1
208         step = 25.0
209         (xmin, ymin, xmax, ymax) = t.region.bounds
210         nodes = []

```

```

205     for ix in range(math.floor(xmin / step), math.floor(xmax / step) + 1):
206         for iy in range(math.floor(ymin / step), math.floor(ymax / step) + 1):
207             square = Polygon([(ix * step, iy * step), ((ix + 1) * step, iy * step),
208                               ↪ ((ix + 1) * step, (iy + 1) * step), (ix * step, (iy + 1) * step)])
209             if square.intersects(t.region):
210                 nodes.append(np.array([(ix + 0.5) * step, (iy + 0.5) * step]))
211     while nodes:
212         p = self.robot.position
213         i = min(range(len(nodes)), key=lambda i: np.linalg.norm(nodes[i] - p))
214         q = nodes.pop(i)
215         if not t.region.intersects(Point(*q).buffer(20.01)):
216             continue
217         if self.try_clear(t, q, 'finite optical cover'):
218             return
219         raise ArithmeticError('exhausted continuous optical covering without success')
220
221 def solve(self, ch):
222     t = self.tracks[ch]
223     tries = 0
224     for k in range(10):
225         if t.radius <= 19.5:
226             if not self.try_clear(t, t.center, 'enclosing circle certificate'):
227                 raise ArithmeticError('certified clear failed')
228             return
229         if self.variant == 'optical' and t.radius <= 85 and (tries < 2):
230             tries += 1
231             q = np.array(t.region.representative_point().coords[0])
232             if self.try_clear(t, q, 'optical shortcut'):
233                 return
234             continue
235         q = self.select(t)
236         if q is None:
237             break
238         z = self.observe(q, ch)
239         if z['measure_result'] == 'no_signal':
240             self.lost += 1
241         self.optical_cover(t)
242
243 def run(self):
244     remaining = set(range(len(self.sites)))
245     while remaining:
246         p = self.robot.position
247         i = min(remaining, key=lambda i: (np.linalg.norm(self.sites[i] - p), i))
248         remaining.remove(i)
249         q = self.sites[i]
250         to_scan = [ch for ch in range(1, 21) if ch not in self.robot.cleared and
251                   ↪ (ch not in self.tracks or (self.variant == 'staged' and
252                   ↪ self.tracks[ch].radius > 19.5))]
253         for ch in to_scan:
254             self.observe(q, ch)
255             self.visited.add(i)
256             self.scan_history.append(dict(site=i, point=q.tolist(), channels=to_scan))
257             self.robot.record(dict(kind='scan_site', site=i, channels=to_scan))
258             if self.variant != 'staged':
259                 while any((ch not in self.robot.cleared for ch in self.tracks)):
260                     ch = min((ch for ch in self.tracks if ch not in
261                               ↪ self.robot.cleared), key=lambda ch:
262                               ↪ np.linalg.norm(self.tracks[ch].center - self.robot.position))
263                     self.solve(ch)
264             if len(self.robot.cleared) == 16:
265                 return 'count upper bound'
266         while any((ch not in self.robot.cleared for ch in self.tracks)):
267             ch = min((ch for ch in self.tracks if ch not in self.robot.cleared),
268                       ↪ key=lambda ch: np.linalg.norm(self.tracks[ch].center -
269                       ↪ self.robot.position))
270             self.solve(ch)
271         for ch in range(1, 21):
272             if ch not in self.tracks:
273                 assert len(self.negatives[ch]) == len(self.sites)
274         return 'mesh certificate and all discovered cleared'

```

文件 q4/M32/bearing_geometry.py

本文件与 q3/v6/bearing_geometry.py 内容完全相同, 使用前述代码。

文件 q4/M32/coverage.py

```
1 """Continuous position/heading certificate; no simulator or truth access."""
2 import math
3 import numpy as np
4 from scipy.spatial import ConvexHull, QhullError
5 from optimize import RING
6
7 class Coverage:
8     def __init__(self):
9         self.points=np.array(RING['points'],dtype=float)
10        self.corners=np.array([[r['lo'],r['hi'][0],r['lo'][1]],r['hi'],r['lo'][0],r[
11        ↪ 'hi'][1]] for r in RING['proof'])
12        self.witness=[list(r['stations']) for r in RING['proof']]
13        self.active=set(range(len(self.points)))
14        self.distances=np.linalg.norm(self.corners[:,None,:]-self.points[None,:,None]
15        ↪ :,1,axis=3).max(axis=2)
16
17    def proposal(self,remove,q=None):
18        """Certify every affected square using any remaining nearby viewpoints."""
19        if q is None:
20            points=self.points;distances=self.distances;active=sorted(self.active-{rem
21            ↪ ove})
22        else:
23            points=np.vstack([self.points,q]);extra=np.linalg.norm(self.corners-q,axis
24            ↪ =2).max(axis=1)
25            distances=np.column_stack([self.distances,extra]);active=sorted(self.activ
26            ↪ e-{remove})+[len(self.points)]
27        updates={}
28        for ci,old in enumerate(self.witness):
29            if remove not in old:continue
30            ids=[i for i in active if distances[ci,i]<=999.99]
31            if len(ids)<3:return None
32            try:h=ConvexHull(points[ids])
33            except QhullError:return None
34            if np.max(self.corners[ci]@h.equations[:,2].T+h.equations[:,2])>1e-8:retu
35            ↪ rn None
36            updates[ci]=[ids[i] for i in h.vertices]
37        return updates
38
39    def commit(self,remove,q,updates):
40        self.active.remove(remove)
41        if q is not None:
42            self.active.add(len(self.points));self.points=np.vstack([self.points,q])
43            extra=np.linalg.norm(self.corners-q,axis=2).max(axis=1)
44            self.distances=np.column_stack([self.distances,extra])
45            for ci,ids in updates.items():self.witness[ci]=ids
46
47    def with_points(self,qs):
48        clone=Coverage.__new__(Coverage)
49        clone.corners=self.corners;clone.witness=self.witness.copy()
50        clone.points=np.vstack([self.points,qs])
51        clone.active=self.active|set(range(len(self.points),len(clone.points)))
52        extra=np.linalg.norm(self.corners[:,None,:]-np.asarray(qs)[None,:,None,:],ax
53        ↪ is=3).max(axis=2)
54        clone.distances=np.column_stack([self.distances,extra])
55        return clone
56
57    def certificate(self):
58        return dict(points=self.points.tolist(),active=sorted(self.active),witness=self
59        ↪ f.witness)
```

文件 q4/M32/joint_planner.py

```

1  """M16 with certified projection of joint radius/heading constraints."""
2  from research_strategy import Planner as M16
3  from joint_region import tighten
4
5  class Planner(M16):
6      def __init__(self, robot, policy='joint'):
7          if policy not in ['baseline', 'joint']: raise ValueError(policy)
8          super().__init__(robot, 'both'); self.joint_policy=policy
9
10     def observe(self, q, ch):
11         z=super().observe(q, ch)
12         if self.joint_policy=='baseline' or ch not in self.tracks: return z
13         t=self.tracks[ch]
14         if not any(h['measure_result']=='no_signal' for h in t.history): return z
15         old=t.region; old_radius=t.radius
16         new, proof=tighten(old, t.history)
17         if not proof['rejected']: return z
18         t.region=new; t.refresh()
19         self.robot.record(dict(kind='joint_region_certificate', channel=ch, history=t.history,
20                                ↪ story.copy(), old_region=old.wkt, new_region=new.wkt, old_radius=old_radius, n
21                                ↪ ew_radius=t.radius, proof=proof))
22         self.robot.record(dict(kind='region', channel=ch, wkt=t.region.wkt, radius_m=t.ra
23                                ↪ dius))
24         return z

```

文件 q4/M32/joint_region.py

```

1  """Conservative cell rejection for joint position/radius/heading feasibility.
2  Closed relaxed arcs retain zero-measure and boundary possibilities.
3  """
4  import math
5  import numpy as np
6  from shapely.geometry import box
7  from shapely.affinity import affine_transform
8  from shapely.ops import unary_union
9  TAU=2*math.pi
10
11  def arc_intersection(arcs, angle, width):
12      if width>=math.pi: return arcs
13      lo=(angle-width)%TAU; hi=lo+2*width
14      pieces=[(lo, min(hi, TAU))]
15      if hi>=TAU: pieces.append((0., hi-TAU))
16      return [(max(a, c), min(b, d)) for a, b in arcs for c, d in pieces if
17              ↪ max(a, c)<=min(b, d)]
18
19  def reject(center, rho, history):
20      positive=[h for h in history if h['measure_result']!='no_signal']
21      negative=[h for h in history if h['measure_result']=='no_signal']
22      pd=[(np.asarray(h['point'])-center) for h in positive]
23      lower=max([1000.]+[float(np.linalg.norm(d))-rho for d in pd])
24      if lower>1500.+1e-5: return dict(reason='radius impossible', lower_radius=lower)
25      forced=[j for j, h in enumerate(negative) if
26              ↪ np.linalg.norm(np.asarray(h['point'])-center)+rho<lower-1e-5]
27      if not forced: return None # Omnidirectional compatibility not ruled out.
28      arcs=[(0., TAU)]
29      for d in pd+[center-np.asarray(negative[j]['point']) for j in forced]:
30          distance=float(np.linalg.norm(d))
31          if distance<=rho+1e-8: continue
32          width=math.pi/2+math.asin(min(1., rho/distance))+1e-9
33          arcs=arc_intersection(arcs, math.atan2(d[1], d[0]), width)
34          if not arcs: return dict(reason='joint heading
35              ↪ impossible', lower_radius=lower, forced_negative_indices=forced)
36      return None
37
38  def tighten(region, history, resolution=10., max_boxes=4096):
39      if not any(h['measure_result']=='no_signal' for h in history): return
40      ↪ region, dict(boxes=0, rejected=[])
41      rectangle=np.array(region.minimum_rotated_rectangle.exterior.coords[:-1]); u=rectangle
42      ↪ gle[1]-rectangle[0]; u/=max(np.linalg.norm(u), 1e-12); v=np.array([-u[1], u[0]])

```

```

38     transformed=affine_transform(region,[u[0],u[1],v[0],v[1],0,0]);queue=[transformed.
    ↪ bounds];removed=[];certificates=[];count=0
39     while queue and count<max_boxes:
40         bounds=queue.pop();g=transformed.intersection(box(*bounds))
41         if g.is_empty or g.area<=1e-12:continue
42         a,b,c,d=g.bounds;center=(np.array([(a+c)/2,(b+d)/2])@np.array([u,v]));rho=math.
    ↪ .hypot(c-a,d-b)/2+2e-6;count+=1
43         proof=reject(center,rho,history)
44         if proof is not None:
45             # The padded rectangle is INSIDE the certified disk. Its overlap
46             # avoids coincident polygon boundary fragments from separate clips.
47             world=affine_transform(box(a-1e-6,b-1e-6,c+1e-6,d+1e-6),[u[0],v[0],u[1],v[
    ↪ 1],0,0]);removed.append(world)
48             certificates.append(dict(center=center.tolist(),radius=rho,wkt=world.wkt,*
    ↪ *proof));continue
49         if max(c-a,d-b)<=resolution:continue
50         if c-a>=d-b:
51             middle=(a+c)/2;queue.extend([(a,b,middle,d),(middle,b,c,d)])
52         else:
53             middle=(b+d)/2;queue.extend([(a,b,c,middle),(a,middle,c,d)])
54         result=region.difference(unary_union(removed)) if removed else region
55         if result.is_empty:raise ArithmeticError('joint feasible region unexpectedly
    ↪ empty')
56         if not result.is_valid:raise ArithmeticError('joint feasible region invalid
    ↪ topology')
57         return result,dict(boxes=count,budget_remaining_boxes=len(queue),rejected=certific
    ↪ ates)

```

文件 q4/M32/m06.py

```

1  """M06: certified dynamic discovery coverage and service-cost routing."""
2  import math
3  import numpy as np
4  import base
5  from previous import Planner as Previous
6  from coverage import Coverage
7  from optimize import open_route,length
8
9  class Planner(Previous):
10     def __init__(self,robot,policy='coverage'):
11         if policy not in ['baseline','coverage','service','combined','pool','pool_serv
    ↪ ice','count','count_service','area_service']:raise ValueError(policy)
12         super().__init__(robot,'route_opt')
13         self.new_policy=policy;self.coverage=Coverage() if policy in
    ↪ ['coverage','combined','pool','pool_service'] else None
14         self.service=policy in ['service','combined','pool_service','count_service','a
    ↪ rea_service'];self.first_choice={};self.service_choices={}
15         self.next_waypoint=None
16         self.replacements=0;self.removed=0
17
18     def select(self,t):
19         if t.channel in self.first_choice:return self.first_choice.pop(t.channel)
20         return super().select(t)
21
22     def try_clear(self,t,q,reason):
23         if self.new_policy=='area_service' and reason=='enclosing circle certificate':
24             # Inner disk of the exact clearance region, with 0.5m safety margin.
25             p=self.robot.position;c=t.center;r=max(0.,19.5-t.radius)
26             d=math.dist(p,c);near=c+(p-c)*min(1.,r/max(d,1e-12))
27             options=[c,near]+[c+r*np.array([math.cos(a),math.sin(a)]) for a in
    ↪ np.arange(32)*2*math.pi/32]
28             def cost(x):return math.dist(p,x)+(math.dist(x,self.next_waypoint) if
    ↪ self.next_waypoint is not None else 0)
29             q=min(options,key=cost)
30             assert np.linalg.norm(t.poly-q,axis=1).max()<=19.501
31             self.robot.record(dict(kind='service_region_clear',channel=t.channel,cente
    ↪ r=c.tolist(),radius=t.radius,point=np.asarray(q).tolist(),next=np.asar
    ↪ ray(self.next_waypoint).tolist() if self.next_waypoint is not None else
    ↪ None))
32         return super().try_clear(t,q,reason)
33

```

```

34 def service_options(self,t):
35     if t.radius<=19.5:return [(t.center,5.)]
36     qs=self.candidates(t)
37     if not qs:return [(t.center,5.+t.radius/5)]
38     g,rad,n=base.heading_hypotheses(t)
39     out=[]
40     for q in qs:
41         if not len(g):after=t.radius;prob=0.
42         else:
43             d=q-g;vis=(np.linalg.norm(d,axis=1)<=rad)&((d*n).sum(axis=1)>=-1e-8)
44             prob=float(vis.mean());vg=g[vis];sizes=[];weights=[]
45             if len(vg):
46                 ug,counts=np.unique(np.round(vg,8),axis=0,return_counts=True)
47                 for x,count in zip(ug,counts):
48                     if math.dist(x,q)<=5:r=5.
49                     else:
50                         a=math.degrees(math.atan2(x[1]-q[1],x[0]-q[0]))
51                         pp=base.clip_halfplanes(t.poly,*base.wedge(q,a,base.EPS))
52                         r=base.enclosing_circle(pp)[1] if len(pp) else t.radius
53                         sizes.append(r);weights.append(count)
54                 after=float(np.average(sizes,weights=weights)) if sizes else t.radius
55                 residual=5.+max(0,math.dist(q,t.center)-19)/5
56                 residual+=prob*(after/5+8*max(0,math.log(max(after/19,1),2)))
57                 residual+=(1-prob)*(35+min(t.radius,350)/2.5)
58                 out.append((q,float(residual)))
59     return out
60
61 def joint_order(self,remaining):
62     if not self.service:return super().joint_order(remaining)
63     keys=[('site',i) for i in sorted(remaining)]+[('target',ch) for ch in
64     ↪ sorted(self.tracks) if ch not in self.robot.cleared]
65     if not keys:return []
66     exits=[self.robot.position]+[self.sites[k] if kind=='site' else
67     ↪ self.tracks[k].center for kind,k in keys]
68     options=[[(self.sites[k],6.*(20-len(self.tracks))) if kind=='site' else
69     ↪ self.service_options(self.tracks[k]) for kind,k in keys]
70     cost=np.zeros((len(exits),len(exits)));choice={}
71     for a,p in enumerate(exits):
72         for b,opts in enumerate(options,1):
73             vals=[math.dist(p,q)/5+r for q,r in opts];j=int(np.argmin(vals));cost[
74             ↪ a,b]=vals[j];choice[a,b]=opts[j][0]
75     todo=set(range(1,len(exits)));order=[0]
76     while todo:
77         b=min(todo,key=lambda
78         ↪ b:(cost[order[-1],b],b));order.append(b);todo.remove(b)
79     def value(rr):return sum(cost[a,b] for a,b in zip(rr[:-1],rr[1:]))
80     best=value(order)
81     for _ in range(30):
82         winning=None;score=best
83         for i in range(1,len(order)-1):
84             for j in range(i+1,len(order)):
85                 rr=order[:i]+order[i:j+1][::-1]+order[j+1:];v=value(rr)
86                 if v<score-1e-7:score=v;winning=rr
87             if winning is None:break
88         order=winning;best=score
89     first=keys[order[1]-1]
90     if first[0]=='target' and self.tracks[first[1]].radius>19.5:self.first_choice[
91     ↪ first[1]]=choice[0,order[1]].copy()
92     self.robot.record(dict(kind='service_route',nodes=[dict(kind=keys[b-1][0],id=k
93     ↪ eys[b-1][1],entry=choice[a,b].tolist(),exit=np.asarray(exits[b]).tolist())
94     ↪ for a,b in zip(order[:-1],order[1:]),estimated_s=float(best)])
95     return [keys[b-1] for b in order[1:]]
96
97 def scan(self,q,site=None):
98     channels=[ch for ch in range(1,21) if ch not in self.tracks]
99     channels += [ch for ch,t in sorted(self.tracks.items()) if ch not in
100     ↪ self.robot.cleared and self.share_worth(t,q)]
101     for ch in sorted(channels):
102         if ch in self.tracks:self.shared_measures+=1
103         self.observe(q,ch)
104     self.robot.record(dict(kind='scan_site',site=site,point=np.asarray(q).tolist()
105     ↪ ,channels=sorted(channels)))
106
107 def adapt(self,remaining):
108     if not remaining:return

```

```

99     if len(self.tracks)==16 and (self.coverage or self.new_policy in
100     ↪ ['count','count_service','area_service']):
101         self.robot.record(dict(kind='skip_search_count_bound',sites=sorted(remaini
102         ↪ ng)))
103         remaining.clear();return
104     if not self.coverage:return
105     if self.new_policy in ['pool','pool_service']:
106         self.pool_adapt(remaining);return
107     # Search all pending stations; certify any replacement before paying for it.
108     p=self.robot.position.copy()
109     # Large savings first, but no unproved station is ever removed.
110     for idx in sorted(remaining,key=lambda i:-math.dist(p,self.sites[i])):
111         updates=self.coverage.proposal(idx)
112         if updates is not None:
113             self.coverage.commit(idx,None,updates);remaining.remove(idx);self.remo
114             ↪ ved+=1
115             self.robot.record(dict(kind='coverage_remove',site=idx,updated_cells=l
116             ↪ en(updates)))
117     for idx in sorted(remaining,key=lambda i:math.dist(p,self.sites[i])):
118         if math.dist(p,self.sites[idx])<2:continue
119         updates=self.coverage.proposal(idx,p)
120         if updates is None:continue
121         # Same unknown-channel scan count as a station, now at the current pose.
122         self.scan(p);self.coverage.commit(idx,p,updates);remaining.remove(idx);sel
123         ↪ f.replacements+=1
124         self.robot.record(dict(kind='coverage_replace',site=idx,point=p.tolist(),u
125         ↪ pdated_cells=len(updates)))
126         break
127
128 def pool_adapt(self,remaining):
129     inner=[i for i in remaining if i<8]
130     if not inner:return
131     p=self.robot.position.copy()
132     pending=sorted([t.center for ch,t in self.tracks.items() if ch not in
133     ↪ self.robot.cleared],key=lambda q:math.dist(p,q))
134     qs=[]
135     for q in [p]+pending[:2]:
136         if all(math.dist(q,self.coverage.points[i])>2 for i in
137         ↪ self.coverage.active) and all(math.dist(q,x)>2 for x in
138         ↪ qs):qs.append(q)
139     if not qs:return
140     proposed=self.coverage.with_points(qs);removed=[]
141     for idx in sorted(inner,key=lambda i:-math.dist(p,self.sites[i])):
142         updates=proposed.proposal(idx)
143         if updates is not
144         ↪ None:proposed.commit(idx,None,updates);removed.append(idx)
145     if not removed:return
146     newids=list(range(len(self.coverage.points),len(proposed.points)))
147     def route_length(ids,points):
148         pts=np.vstack([points[sorted(ids)],pending]) if pending else
149         ↪ points[sorted(ids)]
150         rr=open_route(p,set(range(len(pts))),pts,True)
151         return length(p,rr,pts)
152     old=route_length(remaining,self.sites);new=route_length((remaining-set(removed
153     ↪ ))|set(newids),proposed.points)
154     extra_scans=(len(newids)-len(removed))*(20-len(self.tracks))
155     saving=(old-new)/5-6*extra_scans
156     self.robot.record(dict(kind='pool_proposal',added=len(newids),removed=removed,
157     ↪ estimated_saving_s=float(saving),accepted=saving>20))
158     if saving<=20:return
159     self.coverage=proposed;self.sites=proposed.points.copy();remaining.difference_
160     ↪ update(removed);remaining.update(newids)
161     self.removed+=len(removed);self.replacements+=len(newids)
162     for i in newids:
163         if math.dist(p,self.sites[i])<1e-7:self.scan(p,i);remaining.remove(i);self
164         ↪ .visited.add(i)
165     self.robot.record(dict(kind='pool_commit',removed=removed,added=[dict(id=i,poi
166     ↪ nt=self.sites[i].tolist()) for i in newids]))
167
168 def run(self):
169     if self.new_policy=='baseline':return super().run()
170     remaining=set(range(len(self.sites)))
171     while remaining or any(ch not in self.robot.cleared for ch in self.tracks):
172         self.adapt(remaining)

```

```

157         order=self.joint_order(remaining)
158         if not order:break
159         self.next_waypoint=None
160         if len(order)>1:
161             nk,ni=order[1];self.next_waypoint=self.sites[ni] if nk=='site' else
             ↪ self.tracks[ni].center
162         kind,k=order[0]
163         if kind=='target':self.solve(k)
164         else:self.scan(self.sites[k],k);remaining.remove(k);self.visited.add(k)
165         if len(self.robot.cleared)==16:break
166         self.robot.record(dict(kind='m06_certificate',policy=self.new_policy,coverage=
             ↪ self.coverage.certificate() if self.coverage else
             ↪ None,visited=sorted(self.visited),replacements=self.replacements,removed=s
             ↪ elf.removed))
167         return 'continuous coverage and all discovered cleared'

```

文件 q4/M32/m07.py

```

1  """M07: action substitution, guaranteed-visible recovery, route portfolios."""
2  import math
3  import numpy as np
4  from shapely.geometry import Point
5  from m06 import Planner as Parent
6
7  class Planner(Parent):
8      def __init__(self,robot,policy='baseline'):
9          if policy not in ['baseline','probe65','probe85','hull','portfolio','sticky','
             ↪ fusion','probe_hull','sticky_probe']:raise ValueError(policy)
10         super().__init__(robot,'count');self.m07=policy
11         self.threshold=.85 if policy=='probe85' else .65
12         self.probe=policy in ['probe65','probe85','fusion','probe_hull','sticky_probe']
13         self.recover=policy in ['hull','fusion','probe_hull']
14         self.multi=policy in ['portfolio','sticky','fusion','sticky_probe']
15         self.previous_order=[];self.recovery_count={};self.recovery_candidates=None
16
17         def candidates(self,t):
18             return self.recovery_candidates if self.recovery_candidates is not None else
             ↪ super().candidates(t)
19
20         def select(self,t):
21             if not self.recover or t.history[-1]['measure_result']!='no_signal' or
             ↪ self.recovery_count.get(t.channel,0)>=2:return super().select(t)
22             pts=np.array([h['point'] for h in t.positive])
23             if len(pts)<2:return super().select(t)
24             dm=np.linalg.norm(pts[:,None,:]-pts[None,:, :],axis=2);i,j=np.unravel_index(np.
             ↪ argmax(dm),dm.shape)
25             raw=[pts.mean(axis=0)]+[(1-f)*pts[i]+f*pts[j] for f in
             ↪ [.25,.5,.75]]+[(pts[-1]+pts[-2])/2]
26             qs=[]
27             for q in raw:
28                 if all(math.dist(q,h['point'])>2 for h in t.history) and
                 ↪ all(math.dist(q,x)>1e-6 for x in qs):qs.append(q)
29             if not qs:return super().select(t)
30             self.recovery_candidates=qs
31             try:q=super().select(t)
32             finally:self.recovery_candidates=None
33             self.recovery_count[t.channel]=self.recovery_count.get(t.channel,0)+1
34             self.robot.record(dict(kind='visible_hull_recovery',channel=t.channel,positive
             ↪ _points=pts.tolist(),point=q.tolist()))
35             return q
36
37         def solve(self,ch):
38             if not self.probe:return super().solve(ch)
39             t=self.tracks[ch];probed=False
40             for _ in range(10):
41                 if t.radius<=19.5:
42                     if not self.try_clear(t,t.center,'enclosing circle certificate'):raise
                     ↪ ArithmeticError('certified clear failed')
43                     return
44                 q=self.select(t)
45                 if q is None:break
46                 if not probed:

```

```

47         fraction=t.region.intersection(Point(*q).buffer(19.999,quad_segs=24)).
↪ area/max(t.region.area,1e-12)
48     if fraction>=self.threshold:
49         probed=True
50         self.robot.record(dict(kind='pre_radio_probe',channel=ch,point=q,t
↪ olist(),fraction=float(fraction),region=t.region.wkt,threshold
↪ =self.threshold))
51         if self.try_clear(t,q,'one optical attempt at planned radio
↪ viewpoint'):return
52     # After failed probe, keep the originally selected paid radio viewpoint.
53     z=self.observe(q,ch)
54     if z['measure_result']=='no_signal':self.lost+=1
55     self.optical_cover(t)
56
57 def joint_order(self,remaining):
58     baseline=super().joint_order(remaining)
59     if not self.multi or len(baseline)<2:return baseline
60     keys=[('site',i) for i in sorted(remaining)]+[('target',ch) for ch in
↪ sorted(self.tracks) if ch not in self.robot.cleared]
61     pts=np.vstack([self.robot.position]+[self.sites[k] if kind=='site' else
↪ self.tracks[k].center for kind,k in keys])
62     dm=np.linalg.norm(pts[:,None,:]-pts[None,:,:],axis=2);index={key:i+1 for i,key
↪ in enumerate(keys)}
63     def cost(order):return float(sum(dm[a,b] for a,b in zip(order[:-1],order[1:])))
64     def insert(order):
65         todo=set(range(1,len(pts)))-set(order)
66         while todo:
67             best=None
68             for k in sorted(todo):
69                 for pos in range(1,len(order)+1):
70                     a=order[pos-1];delta=dm[a,k]
71                     if pos<len(order):b=order[pos];delta+=dm[k,b]-dm[a,b]
72                     choice=(delta,k,pos)
73                     if best is None or choice<best:best=choice
74             _,k,pos=best;order.insert(pos,k);todo.remove(k)
75     return order
76     def improve(order):
77         for _ in range(100):
78             best=-1e-7;change=None
79             for i in range(1,len(order)-1):
80                 a,b=order[i-1],order[i]
81                 for j in range(i+1,len(order)):
82                     c=order[j];delta=dm[a,c]-dm[a,b]
83                     if j+1<len(order):d=order[j+1];delta+=dm[b,d]-dm[c,d]
84                     if delta<best:best=delta;change=(i,j)
85             if change is None:break
86             i,j=change;order[i:j+1]=order[i:j+1][::-1]
87     return order
88     orders=[[0]+[index[k] for k in baseline]]
89     warm=[0]+[index[k] for k in self.previous_order if k in index]
90     orders.append(improve(insert(warm)))
91     far=int(np.argmax(dm[0,1:]))+1;orders.append(improve(insert([0,far])))
92     costs=[cost(o) for o in orders];chosen=int(np.argmin(costs))
93     if self.m07 in ['sticky','sticky_probe'] and costs[1]<=min(costs)+25:chosen=1
94     answer=[keys[i-1] for i in
↪ orders[chosen][1:]];self.previous_order=answer.copy()
95     self.robot.record(dict(kind='routing_portfolio',lengths_m=costs,chosen=chosen,
↪ order=[dict(kind=k,id=i) for k,i in answer]))
96     return answer

```

文件 q4/M32/m13.py

```

1  """Change only target representation used in global route ordering."""
2  import numpy as np
3  from m07 import Planner as Parent
4  from posterior import position_cloud,median
5
6  class Planner(Parent):
7      def __init__(self,robot,policy='baseline'):
8          if policy not in
↪ ['baseline','centroid','posterior_mean','posterior_median']:raise
↪ ValueError(policy)

```

```

9         super().__init__(robot, 'portfolio'); self.m13=policy; self.representatives={}
10
11     def representative(self, t):
12         signature=(len(t.history), t.region.wkb)
13         old=self.representatives.get(t.channel)
14         if old is not None and old[0]==signature: return old[1]
15         if self.m13=='centroid': p=np.array(t.region.centroid.coords[0]); count=0
16         else:
17             pts,w=position_cloud(t); p=w@pts if self.m13=='posterior_mean' else
18             ↪ median(pts,w); count=len(pts)
19         self.representatives[t.channel]=(signature,p)
20         self.robot.record(dict(kind='route_representative', channel=t.channel, method=se
21         ↪ lf.m13, mec_center=t.center.tolist(), representative=p.tolist(), mec_radius=t
22         ↪ .radius, points=count))
23         return p
24
25     def joint_order(self, remaining):
26         if self.m13=='baseline': return super().joint_order(remaining)
27         saved={ch:t.center.copy() for ch,t in self.tracks.items() if ch not in
28         ↪ self.robot.cleared}
29         # MEC centers are restored before observing, selecting or clearing.
30         try:
31             for ch in saved: self.tracks[ch].center=self.representative(self.tracks[ch])
32             return super().joint_order(remaining)
33         finally:
34             for ch,c in saved.items(): self.tracks[ch].center=c

```

文件 q4/M32/optimize.py

```

1  """Round 2: route and search/localization integration; frozen M03 base."""
2  from pathlib import Path
3  import json, math, time, argparse, hashlib, sys, platform
4  import numpy as np
5  import base
6  MODEL = Path(__file__).resolve().parents[1]
7  RING = json.loads(Path(__file__).with_name('ring.json').read_text(encoding='utf8'))
8  SITES = np.array(RING['points'])
9  NAMES = {'E0': '原E最近邻', 'R': '开放路线优化', 'J': '顺路定位', 'RJ': '路线与定位融合',
10 ↪ 'JD': '只推迟不顺路补测', 'J100': '顺路定位100m', 'J500': '顺路定位500m', 'K':
11 ↪ '可见性检查后顺路定位'}
12
13 def length(p, route, sites):
14     return float(np.linalg.norm(np.diff(np.vstack([p, sites[route]]), axis=0),
15     ↪ axis=1).sum()) if route else 0.0
16
17 def open_route(p, ids, sites, opt=True):
18     todo = set(ids)
19     r = []
20     q = p
21     while todo:
22         i = min(todo, key=lambda i: (np.linalg.norm(sites[i] - q), i))
23         r.append(i)
24         todo.remove(i)
25         q = sites[i]
26     if not opt:
27         return r
28     best = length(p, r, sites)
29     for repeat in range(30):
30         improved = False
31         for i in range(len(r) - 1):
32             for j in range(i + 1, len(r)):
33                 rr = r[:i] + r[i:j + 1][::-1] + r[j + 1:]
34                 value = length(p, rr, sites)
35                 if value < best - 1e-07:
36                     r = rr
37                     best = value
38                     improved = True
39         if not improved:
40             break
41     return r
42
43 class Integrated(base.Planner):

```

```

41
42 def __init__(self, robot, variant):
43     super().__init__(robot, 'heading', SITES.copy())
44     self.name = variant
45     self.deferred = 0
46     self.shared_measures = 0
47
48 def route(self, remaining):
49     return open_route(self.robot.position, remaining, self.sites, self.name in
    ↪ ['R', 'RJ'])
50
51 def share_worth(self, t, q):
52     if t.radius <= 19.5:
53         return False
54     if any((math.dist(q, h['point']) <= 2 for h in t.history)):
55         return False
56     d = math.dist(q, t.center)
57     if d > 1200:
58         return False
59     old = np.array(t.positive[-1]['point'])
60     u = old - t.center
61     v = q - t.center
62     angle = math.degrees(math.acos(float(np.clip(u @ v / max(np.linalg.norm(u) *
    ↪ np.linalg.norm(v), 1e-10), -1, 1))))
63     return angle >= 15 or d < 0.7 * np.linalg.norm(u)
64
65 def future_useful(self, t, route):
66     (g, r, n) = base.heading_hypotheses(t)
67     if not len(g):
68         return False
69     for i in route[:3]:
70         q = self.sites[i]
71         if not self.share_worth(t, q):
72             continue
73         delta = q - g
74         visible = (np.linalg.norm(delta, axis=1) <= r) & ((delta * n).sum(axis=1)
    ↪ >= -1e-08)
75         if float(visible.mean()) >= 0.7:
76             return True
77     return False
78
79 def run(self):
80     remaining = set(range(len(self.sites)))
81     joint = self.name in ['J', 'RJ', 'JD', 'J100', 'J500', 'K']
82     detour_limit = {'J100': 100, 'J500': 500}.get(self.name, 250)
83     while remaining:
84         route = self.route(remaining)
85         i = route[0]
86         q = self.sites[i]
87         remaining.remove(i)
88         channels = [ch for ch in range(1, 21) if ch not in self.tracks]
89         if joint and self.name != 'JD':
90             channels += [ch for (ch, t) in sorted(self.tracks.items()) if ch not in
    ↪ self.robot.cleared and self.share_worth(t, q)]
91         channels = sorted(channels)
92         for ch in channels:
93             if ch in self.tracks:
94                 self.shared_measures += 1
95                 self.observe(q, ch)
96             self.visited.add(i)
97             self.scan_history.append(dict(site=i, point=q.tolist(), channels=channels))
98             self.robot.record(dict(kind='scan_site', site=i, channels=channels))
99             while any((ch not in self.robot.cleared for ch in self.tracks)):
100                 p = self.robot.position
101                 pending = [ch for ch in self.tracks if ch not in self.robot.cleared]
102                 next_route = self.route(remaining)
103                 next_q = self.sites[next_route[0]] if next_route else None
104                 costs = {}
105                 for ch in pending:
106                     t = self.tracks[ch]
107                     d = math.dist(p, t.center)
108                     detour = d if next_q is None else d + math.dist(t.center, next_q) -
    ↪ math.dist(p, next_q)
109                     costs[ch] = (detour, d)

```

```

110         ch = min(pending, key=lambda ch: (costs[ch][0], costs[ch][1], ch)) if
        ↪ joint else min(pending, key=lambda ch: (math.dist(p,
        ↪ self.tracks[ch].center), ch))
111         (detour, d) = costs[ch]
112         if joint and remaining and (len(self.tracks) < 16) and (detour >
        ↪ detour_limit) and (d > 350):
113             if self.name != 'K' or self.future_useful(self.tracks[ch],
        ↪ next_route):
114                 self.deferred += len(pending)
115                 self.robot.record(dict(kind='defer', channels=pending,
        ↪ next_site=next_route[0], detour_m=detour, distance_m=d))
116                 break
117                 self.robot.record(dict(kind='defer_rejected', channel=ch,
        ↪ reason='no useful visible station in next three'))
118             self.solve(ch)
119             if len(self.robot.cleared) == 16:
120                 return 'count upper bound'
121         while any((ch not in self.robot.cleared for ch in self.tracks)):
122             ch = min((ch for ch in self.tracks if ch not in self.robot.cleared),
        ↪ key=lambda ch: math.dist(self.robot.position, self.tracks[ch].center))
123             self.solve(ch)
124         for ch in range(1, 21):
125             if ch not in self.tracks:
126                 assert len(self.negatives[ch]) == len(self.sites)
127         return 'mesh certificate and all discovered cleared'

```

文件 q4/M32/posterior.py

```

1  """Exact heading/radius prior mass at each position, area quadrature of P.
2
3  Used only for route ranking. Uniform area/R/heading and a half-type mixture
4  are modeling assumptions, not official generator facts or hard certificates.
5  """
6  import math
7  import numpy as np
8  from scipy.stats import qmc
9  from shapely.geometry import Point
10 TAU=2*math.pi
11
12 def intersect_arc(intervals,angle):
13     lo=(angle-math.pi/2)%TAU;hi=lo+math.pi
14     pieces=[(lo,min(hi,TAU))]+[(0.,hi-TAU)] if hi>TAU else []
15     return [(max(a,c),min(b,d)) for a,b in intervals for c,d in pieces if
        ↪ min(b,d)>max(a,c)]
16
17 def prior_mass(x,history):
18     positive=[h for h in history if h['measure_result']!='no_signal']
19     negative=[h for h in history if h['measure_result']=='no_signal']
20     angles=[(0.,TAU)];lo=1000.
21     for h in positive:
22         d=np.asarray(h['point'])-x;distance=float(np.linalg.norm(d));lo=max(lo,distance)
        ↪ e)
23         if h['measure_result']=='near' and distance>5.000001:return 0.
24         if h['measure_result']=='direction' and distance<4.999999:return 0.
25         if distance>1e-8:angles=intersect_arc(angles,math.atan2(d[1],d[0]))
26     if lo>=1500:return 0.
27     negatives=[]
28     for h in negative:
29         d=np.asarray(h['point'])-x;distance=float(np.linalg.norm(d))
30         if distance<1e-8:return 0.
31         negatives.append((distance,math.atan2(d[1],d[0])+math.pi))
32     negatives.sort()
33     omni=max(0,min(1500.,negatives[0][0] if negatives else 1500.))-lo)/500.
34     current=lo;mass=0.
35     for radius,back_angle in negatives:
36         stop=min(radius,1500.)
37         if stop>current:mass+=(stop-current)*sum(b-a for a,b in
        ↪ angles)/TAU;current=stop
38         if radius>=1500:break
39         angles=intersect_arc(angles,back_angle)
40         if not angles:break
41     if angles and current<1500:mass+=(1500-current)*sum(b-a for a,b in angles)/TAU

```

```

42     return .5*omni+.5*mass/500.
43
44 def position_cloud(track,power=7):
45     poly=track.poly;c=np.asarray(track.region.convex_hull.centroid.coords[0]);a=poly;b_
46     ↪ =np.roll(poly,-1,axis=0)
47     area=np.abs((a[:,0]-c[0])*(b[:,1]-c[1])-(a[:,1]-c[1])*(b[:,0]-c[0]))/2
48     if area.sum()<1e-10:return np.array([track.center]),np.ones(1)
49     distribution=np.cumsum(area/area.sum());z=qmc.Sobol(3,scramble=False).random_base2_
50     ↪ (power)
51     idx=np.minimum(np.searchsorted(distribution,z[:,0],side='right'),len(poly)-1);f=np_
52     ↪ .sqrt(z[:,1])
53     pts=(1-f[:,None])*c+f[:,None]*((1-z[:,2,None])*a[idx]+z[:,2,None]*b[idx])
54     pts=np.array([p for p in pts if np.linalg.norm(p)<=1800+1e-8 and
55     ↪ track.region.covers(Point(*p))])
56     if not len(pts):return np.array([track.center]),np.ones(1)
57     weights=np.array([prior_mass(x,track.history) for x in pts]);keep=weights>0
58     if not keep.any():return np.array([track.region.centroid.coords[0]]),np.ones(1)
59     pts=pts[keep];weights=weights[keep];weights/=weights.sum();return pts,weights
60
61 def median(points,weights):
62     x=weights@points
63     for _ in range(100):
64         d=np.maximum(np.linalg.norm(points-x,axis=1),1e-7);w=weights/d;y=(w@points)/w._
65         ↪ sum()
66         if np.linalg.norm(y-x)<1e-5:return y
67     x=y
68     return x

```

文件 q4/M32/previous.py

```

1  """Candidate policy only; all paid physics comes from shared Robot."""
2  import math,json
3  from pathlib import Path
4  import numpy as np
5  from shapely.geometry import Point,Polygon
6  from scipy.spatial import ConvexHull
7  import base
8  from optimize import Integrated,RING
9
10 class Planner(Integrated):
11     def __init__(self,robot,policy='control'):
12         if policy not in ['control','recovery','optcover','mobile','combined','route',_
13         ↪ 'route_opt','greedyopt']:raise ValueError(policy)
14         super().__init__(robot,'J500');self.policy=policy;self.relocations=0
15         self.leaves=[]
16         if policy=='mobile':
17             for leaf in RING['proof']:
18                 lo,hi=leaf['lo'],leaf['hi'];c=np.array([lo,[hi[0],lo[1]],hi,[lo[0],hi[_
19                 ↪ 1]])
20                 self.leaves.append((c,leaf['stations']))
21         self.checked_moves=set()
22     def candidates(self,t):
23         raw=super().candidates(t)
24         if self.policy not in ['recovery','combined'] or
25         ↪ t.history[-1]['measure_result']!='no_signal':return raw
26         last=np.array(t.positive[-1]['point']);c=t.center
27         # Recover from a known positive viewpoint; hypotheses still rank these.
28         for f in [.25,.5,.75]:
29             q=last+f*(c-last)
30             if all(math.dist(q,h['point'])>2 for h in t.history):raw.append(q)
31         return raw
32     def optical_cover(self,t):
33         if self.policy in ['greedyopt','route_opt']:return self.greedy_optical(t)
34         if self.policy not in ['optcover','combined']:return super().optical_cover(t)
35         self.fallbacks+=1
36         poly=t.poly;_,(i,j)=base.diameter(poly);u=poly[j]-poly[i];u=u/max(np.linalg.no_
37         ↪ rm(u),1e-10);v=np.array([-u[1],u[0]]);basis=np.array([u,v])
38         rotated=poly@basis.T;lo=rotated.min(axis=0);hi=rotated.max(axis=0);width=hi[1]_
39         ↪ -lo[1]
40         # Every rectangle has circumradius <=19.5m; intersecting cells retained.
41         if width<=36:

```

```

37         height=max(width,1e-6);step=2*math.sqrt(19.5**2-(height/2)**2)
38     else:height=27.5;step=27.5
39     nx=max(1,math.ceil((hi[0]-lo[0])/step));ny=max(1,math.ceil((hi[1]-lo[1])/height
    ↪ t));nodes=[]
40     for ix in range(nx):
41         for iy in range(ny):
42             a=lo+np.array([ix*step,iy*height]);b=np.minimum(hi,a+[step,height]);co
    ↪ rners=np.array([a,[b[0],a[1]],b,[a[0],b[1]]])@basis
43             if Polygon(corners).intersects(t.region):nodes.append(((a+b)/2@basis,c
    ↪ orners))
44     self.robot.record(dict(kind='optical_cover_certificate',channel=t.channel,width
    ↪ h_m=float(width),cells=[c.tolist() for q,c in nodes],centers=[q.tolist()
    ↪ for q,c in nodes]))
45     while nodes:
46         k=min(range(len(nodes)),key=lambda
    ↪ k:math.dist(nodes[k][0],self.robot.position));q,c=nodes.pop(k)
47         if not Polygon(c).intersects(t.region):continue
48         if t.radius<=19.5:
49             if not self.try_clear(t,t.center,'enclosing circle certificate'):raise
    ↪ ArithmeticError('certified clear failed')
50             return
51             if self.try_clear(t,q,'oriented rectangle optical cover'):return
52     raise ArithmeticError('exhausted certified optical rectangles')
53 def greedy_optical(self,t):
54     self.fallbacks+=1;step=25.;xmin,ymin,xmax,ymax=t.region.bounds;nodes=[]
55     for ix in range(math.floor(xmin/step),math.floor(xmax/step)+1):
56         for iy in range(math.floor(ymin/step),math.floor(ymax/step)+1):
57             corners=np.array([[ix*step,iy*step],[(ix+1)*step,(iy+1)*step],[(ix+1)*step
    ↪ ,(iy+1)*step],[ix*step,(iy+1)*step]])
58             if Polygon(corners).intersects(t.region):nodes.append((corners.mean(ax
    ↪ is=0),corners))
59     self.robot.record(dict(kind='optical_cover_certificate',channel=t.channel,cell
    ↪ s=[c.tolist() for q,c in nodes],centers=[q.tolist() for q,c in nodes]))
60     while nodes:
61         if t.radius<=19.5:
62             if not self.try_clear(t,t.center,'enclosing circle certificate'):raise
    ↪ ArithmeticError('certified clear failed')
63             return
64         candidates=sorted(range(len(nodes)),key=lambda
    ↪ i:math.dist(nodes[i][0],self.robot.position))[:48]
65         scores=[]
66         for i in candidates:
67             q,c=nodes[i];area=t.region.intersection(Point(*q).buffer(19.999,quad_s
    ↪ egs=16)).area
68             scores.append(area/(3+math.dist(q,self.robot.position)/5))
69         k=candidates[int(np.argmax(scores))];q,c=nodes.pop(k)
70         if not Polygon(c).intersects(t.region):continue
71         if self.try_clear(t,q,'optical area per paid second'):return
72     raise ArithmeticError('exhausted optical covering')
73 def route(self,remaining):
74     route=super().route(remaining)
75     if self.policy!='mobile' or not route:return route
76     idx=route[0];q=self.robot.position
77     key=(idx,round(float(q[0]),3),round(float(q[1]),3))
78     if idx==0 or idx>7 or key in self.checked_moves or not
    ↪ 2<math.dist(q,self.sites[idx])<=250:return route
79     self.checked_moves.add(key);new=self.sites.copy();new[idx]=q
80     for corners,ids in self.leaves:
81         if idx not in ids:continue
82         pts=new[ids]
83         if np.linalg.norm(pts[:,None,:]-corners[None,:,:],axis=2).max(>999.99:ret
    ↪ urn route
84         try:h=ConvexHull(pts)
85         except Exception:return route
86         if (corners@h.equations[:,2].T+h.equations[:,2]).max(>1e-8:return route
87     self.robot.record(dict(kind='relocate',site=idx,old=self.sites[idx].tolist(),n
    ↪ ew=q.tolist()))
88     self.sites=new;self.relocations+=1
89     return route
90 def run(self):
91     reason=self.joint_route_run() if self.policy in ['route','route_opt'] else
    ↪ super().run()

```

```

92     self.robot.record(dict(kind='final_certificate', reason=reason, sites=self.sites
    ↪ .tolist(), visited=sorted(self.visited), relocations=self.relocations, policy
    ↪ =self.policy))
93     return reason
94 def joint_order(self, remaining):
95     keys=[('site',i) for i in sorted(remaining)]+[('target',ch) for ch in
    ↪ sorted(self.tracks) if ch not in self.robot.cleared]
96     pts=[self.sites[k] if kind=='site' else self.tracks[k].center for kind,k in
    ↪ keys]
97     if not pts: return []
98     pp=np.vstack([self.robot.position,pts]);dm=np.linalg.norm(pp[:,None,:]-pp[None,
    ↪ :,:],axis=2);todo=set(range(1,len(pp)));order=[0]
99     while todo:
100         k=min(todo,key=lambda k:(dm[order[-1],k],k));order.append(k);todo.remove(k)
101     for _ in range(100):
102         change=None;best=-1e-7
103         for i in range(1,len(order)-1):
104             a,b=order[i-1],order[i]
105             for j in range(i+1,len(order)):
106                 c=order[j];delta=dm[a,c]-dm[a,b]
107                 if j+1<len(order):
108                     d=order[j+1];delta+=dm[b,d]-dm[c,d]
109                 if delta<best:best=delta;change=(i,j)
110             if change is None:break
111             i,j=change;order[i:j+1]=order[i:j+1][::-1]
112     self.robot.record(dict(kind='joint_route', nodes=[dict(kind=keys[k-1][0], id=key
    ↪ s[k-1][1], point=pp[k].tolist()) for k in
    ↪ order[1:]], length_m=float(sum(dm[a,b] for a,b in
    ↪ zip(order[:-1],order[1:])))
113     return [keys[k-1] for k in order[1:]]
114 def joint_route_run(self):
115     remaining=set(range(len(self.sites)))
116     while remaining or any(ch not in self.robot.cleared for ch in self.tracks):
117         order=self.joint_order(remaining);kind,k=order[0]
118         if kind=='target':self.solve(k)
119         else:
120             q=self.sites[k];remaining.remove(k)
121             channels=[ch for ch in range(1,21) if ch not in self.tracks]
122             channels += [ch for ch,t in sorted(self.tracks.items()) if ch not in
    ↪ self.robot.cleared and self.share_worth(t,q)]
123             for ch in sorted(channels):
124                 if ch in self.tracks:self.shared_measures+=1
125                 self.observe(q,ch)
126             self.visited.add(k);self.scan_history.append(dict(site=k, point=q.tolis
    ↪ t(), channels=sorted(channels)))
127             self.robot.record(dict(kind='scan_site', site=k, channels=sorted(channel
    ↪ s)))
128             if len(self.robot.cleared)==16: return 'count upper bound'
129         for ch in range(1,21):
130             if ch not in self.tracks: assert len(self.negatives[ch])==len(self.sites)
131     return 'mesh certificate and all discovered cleared'

```

文件 q4/M32/research_strategy.py

```

1  """Ablate posterior-based routing and posterior-centered radio candidates.
2
3  MEC geometry is restored before any paid action or safety certificate.
4  """
5  from m07 import Planner as M07
6  from m13 import Planner as Parent
7
8  class Planner(Parent):
9      def __init__(self, robot, policy='baseline'):
10         if policy not in ['baseline', 'route', 'local', 'both']: raise ValueError(policy)
11         super().__init__(robot, 'posterior_mean'); self.m16=policy
12
13     def joint_order(self, remaining):
14         if self.m16 in ['baseline', 'local']: return M07.joint_order(self, remaining)
15         return super().joint_order(remaining)
16
17     def select(self, t):
18         if self.m16 not in ['local', 'both']: return super().select(t)

```

```

19         representative=self.representative(t);original=t.center.copy()
20     try:
21         t.center=representative
22         q=super().select(t)
23     finally:t.center=original
24     if q is not None:self.robot.record(dict(kind='posterior_radio_candidate',chann
        ↪ el=t.channel,mec_center=original.tolist(),representative=representative.to
        ↪ list(),point=q.tolist()))
25     return q

```

文件 q4/M32/run_robot.py

```

1  """ 问题四 M32 joint 正式测试入口; 冻结策略已完成 30 场官方演练。"""
2  import argparse
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import sys
9  import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14 def main():
15     if hasattr(sys.stdout, 'reconfigure'):
16         sys.stdout.reconfigure(encoding='utf-8')
17     parser = argparse.ArgumentParser(description=__doc__)
18     parser.add_argument('--policy', choices=['joint'], default='joint', help='第四问M32
        ↪ joint联合可行域策略')
19     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
20     parser.add_argument('--port', type=int, help='默认复用旧配置, 缺省端口 2026')
21     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
22     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置,
        ↪ 不连接模拟器')
23     args = parser.parse_args()
24     try:
25         import numpy
26         import scipy
27         import shapely
28         from api_client import Robot, BudgetReached
29         from strategy import Planner, VERSION, POLICIES
30     except ImportError as error:
31         print(f'缺少依赖: {error}\n 请安装 requirements.txt 中的依赖后重试。')
32         return 2
33     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT.parent /
        ↪ 'simple-strategy' / 'config.json']
34     config_path = next((p for p in paths if p and p.is_file()), None)
35     config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path
        ↪ else {}
36     team, port = args.team or config.get('team_no'), args.port or config.get('port',
        ↪ 2026)
37     if not team:
38         print('找不到队号配置, 请使用 --config 指定旧 config.json, 或使用 --team 参数。')
39         return 2
40     if args.check:
41         print(f'策略 {args.policy}: 第四问 M32 joint; 已完成 30 场官方演练。')
42         print(f'自检通过: Python {sys.version.split()[0]} / numpy {numpy.__version__} /
        ↪ scipy {scipy.__version__}')
43         print(f'配置可读: 端口 {port}; 未连接模拟器。')
44         return 0
45     print(f'问题 4 研究策略 {VERSION} / {args.policy} | 端口 {port}')
46     print('这是已完成 30 场官方演练的冻结 M32 joint 策略。')
47     print('请在模拟器选择【问题 4 正式测试】, 开始后等待至少 5 秒, 确认“等待机器狗进入”。')
48     print('接口无法查询演练/正式模式, 程序按问题 4 混合全向/定向干扰器规则工作。')
49     input('界面就绪后按 Enter 开始: ')
50     folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' +
        ↪ uuid.uuid4().hex[:6])
51     robot = Robot(team, f'http://127.0.0.1:{port}', folder)
52     planner = Planner(robot, policy=args.policy)
53

```

```

54 snapshot = folder / 'source_snapshot'
55 snapshot.mkdir()
56 hashes = {}
57 for p in list(ROOT.glob('*.py')) + [p for p in ROOT.glob('*.json') if p.name !=
    ↳ 'config.json']:
58     shutil.copy2(p, snapshot / p.name)
59     hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60 (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION,
    ↳ policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__,
    ↳ shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path), mode='user_selected_P4_formal_not_API_verified'),
63     ensure_ascii=False, indent=2), encoding='utf-8')
64 outcome, error_text, entered = 'completed', None, False
65 robot.started = time.monotonic()
66 try:
67     robot.call('/enter')
68     entered = True
69     planner.run()
70 except BudgetReached as error:
71     outcome, error_text = 'budget_stop', str(error)
72 except (Exception, KeyboardInterrupt) as error:
73     outcome, error_text = 'error', str(error) or '用户中断'
74     # Interruption during HTTP could leave execution state unknown.
75     if isinstance(error, KeyboardInterrupt):
76         robot.transport_uncertain = True
77 finally:
78     if entered and not robot.transport_uncertain:
79         try:
80             robot.call('/exit')
81         except Exception as error:
82             outcome = 'exit_unconfirmed'
83             error_text = (error_text or '') + '\n 退出未确认: ' + str(error)
84     count = len(robot.cleared)
85     summary = dict(strategy=VERSION, policy=args.policy,
    ↳ policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
86     exit_confirmed=robot.exit_confirmed,
    ↳ transport_uncertain=robot.transport_uncertain,
87     cleared_count=count, cleared_channels=sorted(robot.cleared),
88     observed_but_uncleared=sorted(robot.discovered - robot.cleared),
89     measure_count=robot.measure_count, clear_failures=robot.clear_failures,
90     scanned_sites=robot.scanned_sites, distance_m=robot.distance_m,
    ↳ switches=robot.switches,
91     virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if
    ↳ count else None,
92     local_elapsed_s=time.monotonic() - robot.started,
93     actual_jammer_count=None, completion=planner.certificate(),
94     note='实际目标总数和正式成绩以模拟器为准;
    ↳ 完整性证书以问题4半圆定向/全向与R≥1000规则为前提。')
95     (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False,
    ↳ indent=2), encoding='utf-8')
96     robot.log.close()
97     print(f'\n结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动
    ↳ {robot.distance_m:.1f} 米。')
98     print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据
    ↳ {planner.stop_reason}')
99     if error_text:
100         print(error_text)
101     print(f'运行记录: {folder}')
102     print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
103     return 0 if outcome == 'completed' and robot.exit_confirmed else 1
104
105
106 if __name__ == '__main__':
107     sys.exit(main())

```

文件 q4/M32/strategy.py

```

1 """Frozen J500 planner with an official HTTP-compatible lifecycle wrapper."""
2 import json
3 from pathlib import Path
4 from joint_planner import Planner as Integrated

```

```

5
6 VERSION = 'p4-m32-frozen-official-20260912'
7 POLICIES =
  ↳ json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-8'))
8
9
10 class Planner(Integrated):
11     def __init__(self, robot, *, policy='joint'):
12         if policy != 'joint':
13             raise ValueError(policy)
14         super().__init__(robot, 'joint')
15         self.stop_reason = None
16
17     def run(self):
18         try:
19             self.stop_reason = super().run()
20         finally:
21             self.robot.scanned_sites = len(self.visited)
22         return self.stop_reason
23
24     def certificate(self):
25         return dict(problem=4, reason=self.stop_reason,
26             ↳ complete=bool(self.stop_reason),
27                 cleared_channels=sorted(self.robot.cleared),
28                 observed_but_uncleared=sorted(set(self.tracks) - set(self.robot.cleared)),
29                 mesh_points=self.sites.tolist(), visited_sites=sorted(self.visited),
30                 negative_counts={str(ch):len(points) for ch,points in
31                     ↳ self.negatives.items()}),
32                 deferred=self.deferred, shared_measures=self.shared_measures,
33                 optical_fallbacks=self.fallbacks, known_channel_radio_losses=self.lost,
34                 assumptions='m32/joint: frozen research policy; original certified coverage
35                     ↳ and finite optical clearance')

```